INGENIERÍA DE SOFTWARE II

Material de repaso

Las diapositivas de clase de los 8 temas, en orden, con diapositivas extra de ejemplos y ejercicios resueltos.

01 Gestión de la configuración
02 Construcción de sistemas
03 Integración continua
04 Prácticas y estándares
05 Inspección de código
06 Pruebas de software
07 Valor límite
08 Clases de equivalencia

Tema 1

Gestión de la configuración y gestión de versiones

Políticas, procesos y herramientas para gestionar sistemas que cambian

INGENIERÍA DE SOFTWARE II

Gestión de la Configuración

Políticas, procesos y herramientas para gestionar sistemas de software cambiantes

Agenda



Gestión de la configuración



Gestión de versiones

01

GESTIÓN DE LA CONFIGURACIÓN

Gestión de la Configuración

Gestión de la configuración (CM)

Los sistemas de software cambian constantemente durante el desarrollo y el uso:



Se descubren errores y deben solucionarse.



Los requisitos del sistema cambian y hay que implementarlos en una nueva versión.



Se lanzan nuevas versiones de hardware y plataformas, y los sistemas deben adaptarse.



Los competidores introducen nuevas características que hay que igualar.

Gestión de la configuración (CM)



Se ocupa de las políticas, los procesos y las herramientas para gestionar los sistemas de software cambiantes.



A medida que se realizan cambios en el software, se crea una nueva versión del sistema.



Un sistema puede considerarse como un conjunto de versiones, cada una de las cuales puede requerir mantenimiento y gestión.

Cuatro actividades relacionadas

La gestión de la configuración de un producto de software implica:



Gestión de versiones



Construcción de sistemas

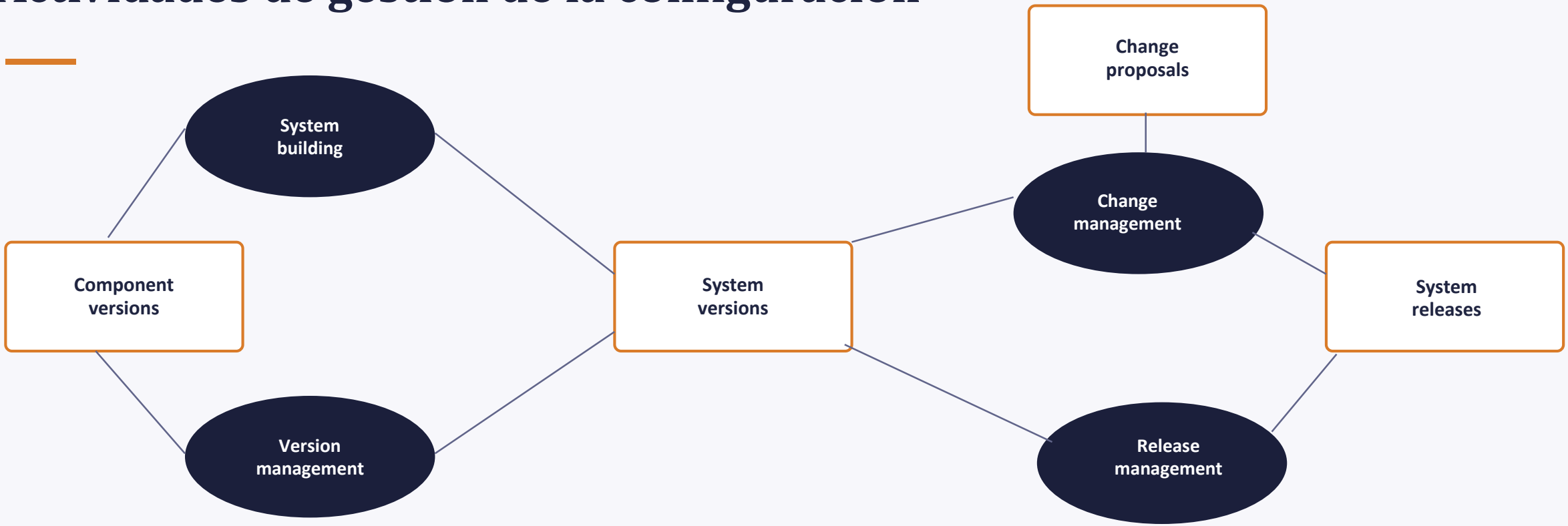


Gestión del cambio



Gestión de lanzamiento

Actividades de gestión de la configuración



Actividades de gestión de la configuración



Gestión de versiones

Implica realizar un seguimiento de las múltiples versiones de los componentes del sistema y garantizar que los cambios realizados por diferentes desarrolladores no interfieran entre sí.



Construcción del sistema

Proceso de ensamblar componentes de programa, datos y bibliotecas, para luego compilarlos y vincularlos y crear un sistema ejecutable.

Actividades de gestión de la configuración

Gestión del cambio

Seguimiento de las solicitudes de cambio al software entregado por clientes y desarrolladores; cálculo de costos e impacto; decisión de si implementar los cambios y cuándo.

Gestión de lanzamiento

Preparación del software para su lanzamiento externo, y seguimiento de las versiones del sistema que se han lanzado para uso del cliente.

Las cuatro actividades en un caso real

EJEMPLO

Una app de delivery recibe un reporte: **«el total no incluye el costo de envío»**. Así participa cada actividad de la gestión de la configuración:

1 · Gestión del cambio

Se registra la **solicitud de cambio**, se estima costo e impacto y se decide arreglarlo en la próxima versión.

2 · Gestión de versiones

Ana crea una **rama** desde la línea principal, corrige `Pedido.java` y lo integra. El historial registra quién cambió qué.

3 · Construcción del sistema

El servidor **compila y vincula** el código, las librerías y la configuración, y ejecuta las pruebas.

4 · Gestión de lanzamiento

Se prepara y entrega la **versión 2.3.1** a los usuarios, y se registra qué componentes la forman.

> Sin gestión de la configuración: nadie sabe qué versión tiene el cliente, dos personas pisan el mismo archivo y no se puede reconstruir la versión que falló.

Terminología de gestión de la configuración

Línea base	Colección de versiones de componentes que conforman un sistema. Están controladas: sus componentes no pueden cambiar, y siempre se pueden recrear a partir de ellos.
Ramificación	Creación de una nueva línea de código a partir de una versión en una línea existente. Ambas podrán desarrollarse independientemente.
Línea de código	Conjunto de versiones de un componente de software y otros elementos de configuración de los que depende.
Control de configuración	Proceso que garantiza que las versiones de sistemas y componentes se registren y mantengan, identificando y almacenando todas las versiones durante la vida del sistema.
Elemento de configuración (SCI)	Cualquier artefacto de un proyecto de software (diseño, código, datos de prueba, documento) puesto bajo control de configuración, con identificador único.

Terminología de gestión de la configuración

Línea principal	Secuencia de líneas base que representan diferentes versiones de un sistema.
Fusión	Creación de una nueva versión de un componente al fusionar versiones separadas en distintas líneas de código.
Lanzamiento	Versión de un sistema que se ha entregado a los clientes u otros usuarios para su uso.
Repositorio	Base de datos compartida de versiones de componentes de software y metainformación sobre sus cambios.
Construcción del sistema	Creación de una versión ejecutable compilando y vinculando las versiones apropiadas de componentes y bibliotecas.
Versión	Instancia de un elemento de configuración que difiere de otras instancias de ese elemento; siempre con identificador único.
Espacio de trabajo	Área de trabajo privada donde se puede modificar software sin afectar a otros desarrolladores.

La terminología, traducida a Git

EJEMPLO

Término	Qué es	En Git / GitHub
Repositorio	Base de datos compartida de versiones	El proyecto en GitHub o GitLab
Espacio de trabajo	Área privada donde modificas sin afectar a otros	Tu carpeta local (working directory)
Versión	Instancia con identificador único	Un commit: `a3f9c1e`
Línea de código	Secuencia de versiones de un componente	El historial de un archivo o rama
Ramificación	Nueva línea a partir de una existente	`git branch envio`
Fusión	Unir versiones de líneas distintas	`git merge envio`
Línea principal	Secuencia de líneas base	La rama `main`
Línea base	Conjunto controlado de versiones que se puede recrear	Una etiqueta: `git tag v1.0`
Lanzamiento	Versión entregada a los clientes	Un *release* publicado: v1.0

Elemento de configuración (SCI): cualquier artefacto bajo control, con identificador único: código, diseño, datos de prueba o documentos.

02

GESTIÓN DE VERSIONES

Gestión de Versiones

Gestión de versiones



Proceso de realizar seguimiento de las diferentes versiones de los componentes de software y los sistemas en los que se utilizan.



Garantiza que los cambios realizados por diferentes desarrolladores no interfieran entre sí.

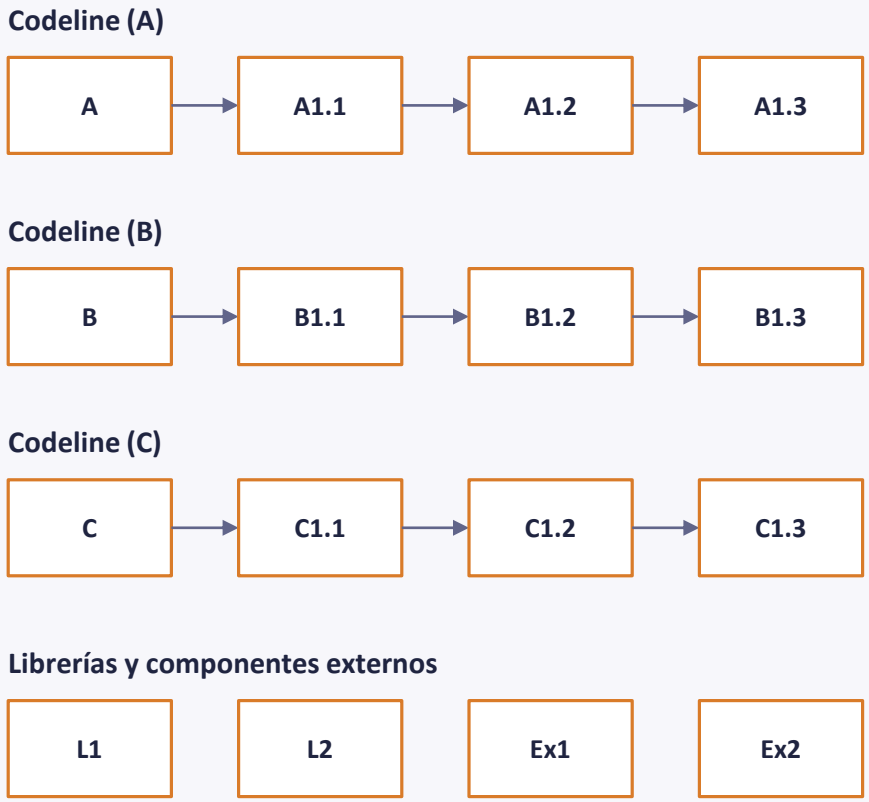


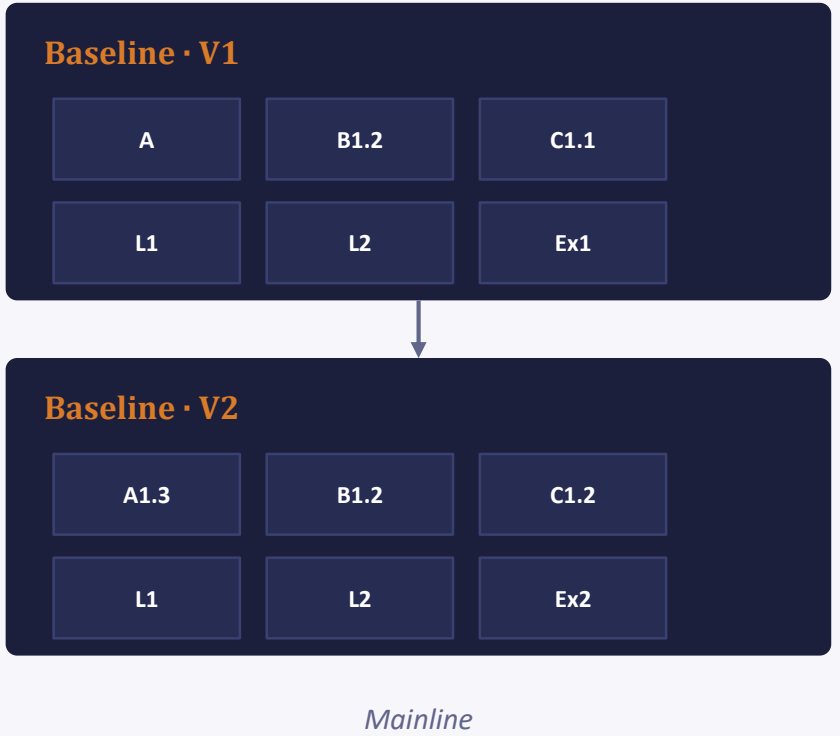
En otras palabras, es el proceso de administrar líneas de código y líneas base.



Para respaldarla, necesitamos sistemas de control de versiones (VC).

Líneas de código y líneas base





Mainline

Ejercicio: ¿qué cambió entre dos líneas base?

EJERCICIO

Componente	Línea base V1	Línea base V2
A (interfaz)	A	A1.3
B (lógica)	B1.2	B1.2
C (datos)	C1.1	C1.2
L1, L2 (librerías)	L1, L2	L1, L2
Ex (externo)	Ex1	Ex2

Mismo esquema de la diapositiva «Líneas de código y líneas base».

Preguntas

1. ¿Qué componentes cambiaron de V1 a V2?
2. ¿Se puede reconstruir hoy la V1?
3. ¿Qué es la línea principal aquí?

1. Cambiaron **A** (A → A1.3), **C** (C1.1 → C1.2) y el componente externo (**Ex1 → Ex2**). B y las librerías se reutilizan.
2. **Sí**: una línea base está controlada; sus versiones no cambian y siempre se puede recrear.
3. La secuencia **V1 → V2** de líneas base.

Características de los sistemas de control de versiones (VC)



Identificación de versión y lanzamiento: a las versiones administradas de un componente se les asignan identificadores únicos al enviarse al sistema.



Registro del historial de cambios: se mantienen registros de los cambios realizados para crear una nueva versión a partir de una anterior.



Desarrollo independiente: diferentes desarrolladores pueden trabajar en el mismo componente al mismo tiempo.

Características de los sistemas de control de versiones (VC)



Soporte de proyectos

Un sistema de control de versiones puede soportar el desarrollo de varios proyectos que comparten componentes.



Gestión de almacenamiento

En lugar de mantener copias separadas de todas las versiones, se utilizan mecanismos eficientes que evitan duplicar archivos idénticos.

Tipos de sistemas de control de versiones

Los sistemas VC identifican, almacenan y controlan el acceso a las diferentes versiones de los componentes. Existen dos tipos modernos:

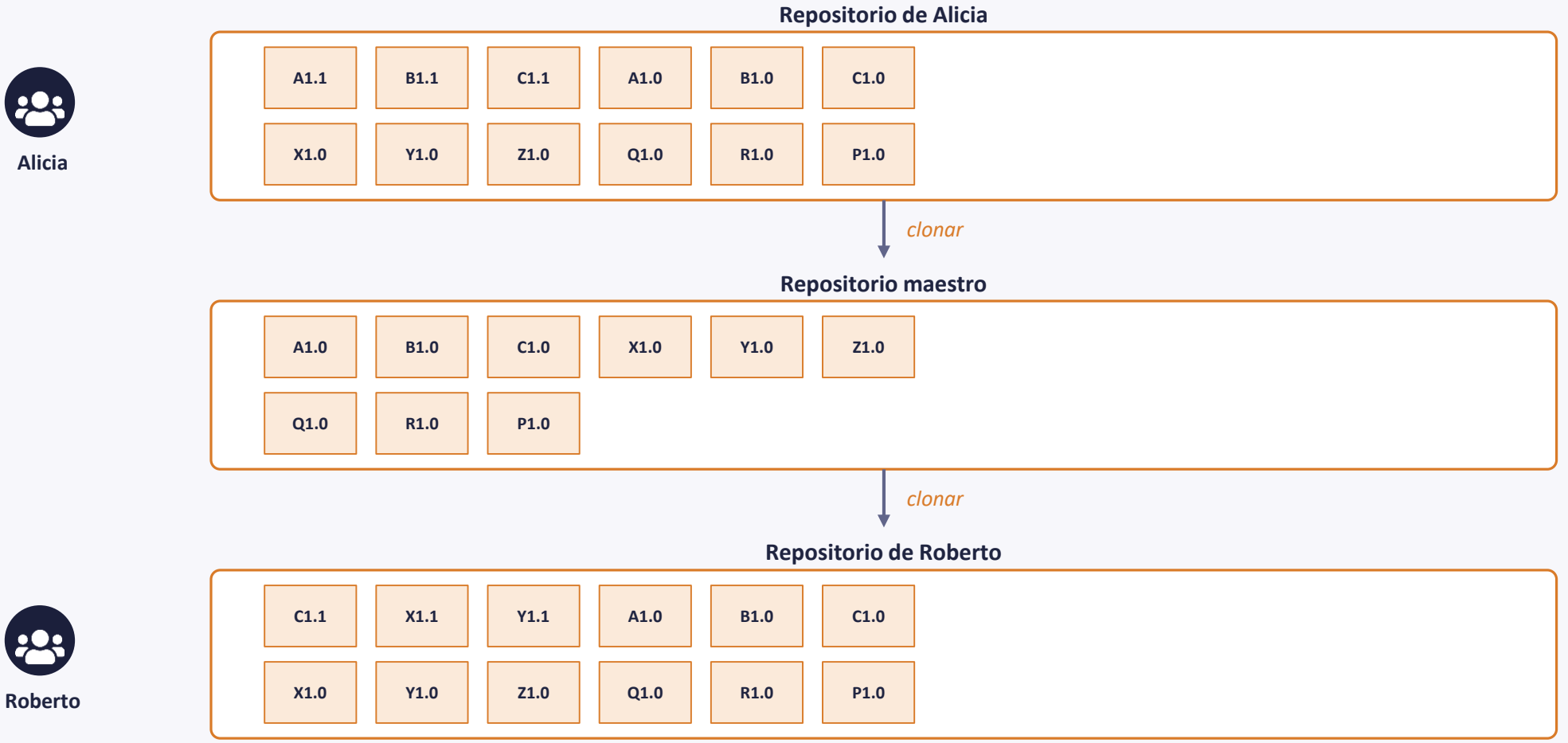
Sistemas centralizados

- Un único repositorio maestro mantiene todas las versiones de los componentes en desarrollo.
- Subversion es un ejemplo común de sistema centralizado.

Sistemas distribuidos

- Existen múltiples versiones del repositorio de componentes de forma simultánea.
- Git es un ejemplo común de sistema distribuido.

Sistemas de VC distribuidos: clonación de repositorios



Ventajas de los sistemas VC distribuidos sobre los centralizados



Proporciona un mecanismo de respaldo: si el repositorio se daña, se puede restaurar desde copias locales sin detener el trabajo.



Permite trabajar sin conexión, de modo que los desarrolladores pueden confirmar cambios sin una conexión de red.



El soporte de proyectos es la forma predeterminada de trabajar: se puede compilar y probar todo el sistema en el equipo local.

Centralizado vs. distribuido: el día que se cae el servidor

EJEMPLO

Centralizado (Subversion)

Hay un solo repositorio maestro. Cada desarrollador solo tiene una copia de trabajo.

- ✕ Si el servidor cae, nadie puede confirmar (commit) cambios.
- ✕ Si el disco del servidor se daña sin respaldo, se pierde el historial.
- ✕ Para ver el historial necesitas red.

Distribuido (Git)

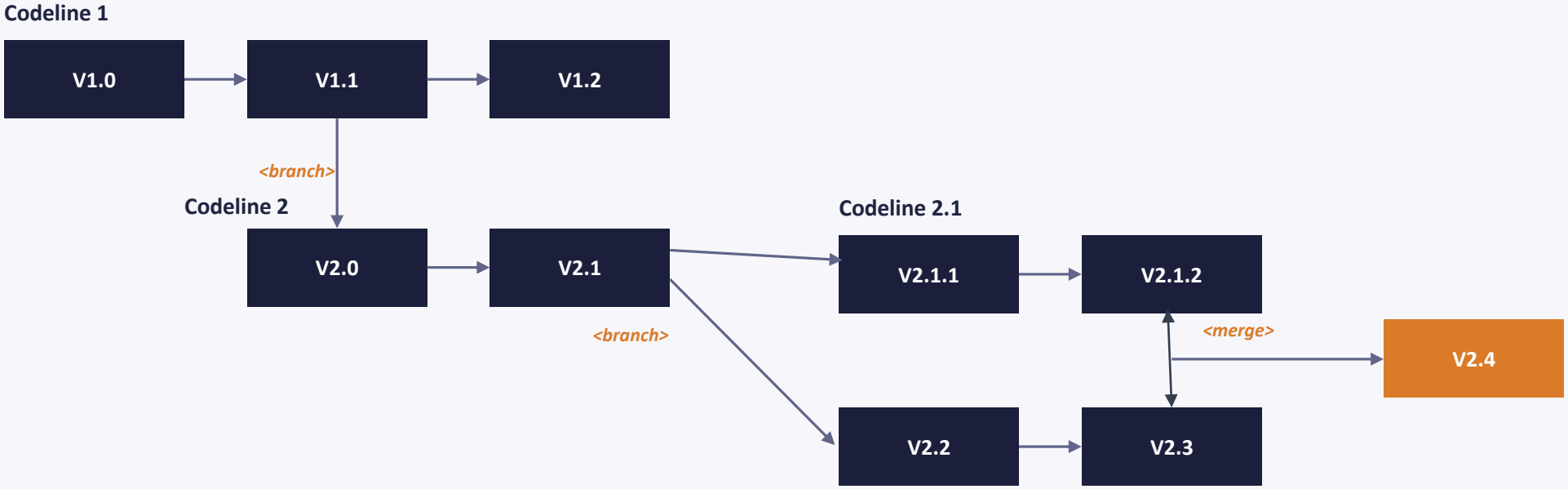
Cada desarrollador **clona** el repositorio completo, con todo su historial.

- ✓ Si el servidor cae, sigues haciendo commits locales y subes después.
- ✓ Cualquier clon sirve de **respaldo** para restaurar el maestro.
- ✓ Trabajas sin conexión y compilas y pruebas todo en tu equipo.

Son justo las tres ventajas de la diapositiva: **respaldo**, **trabajo sin conexión** y **soporte de proyectos** en el equipo local.

Ramificación y fusión en sistemas VC distribuidos

Las líneas de código pueden ramificarse por el desarrollo independiente del mismo componente; en algún momento puede ser necesario fusionar ramas para crear una nueva versión.



Ramificar y fusionar con Git, paso a paso

EJEMPLO

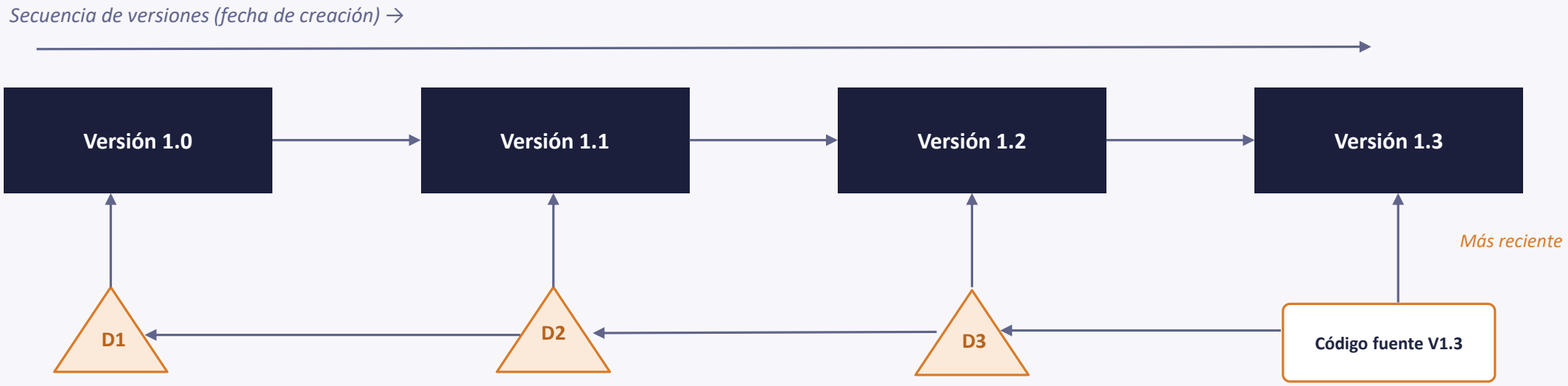
```
git clone https://github.com/equipo/delivery.git
git switch -c envio          # nueva rama (ramificación)
# ...editas Pedido.java...
git add Pedido.java
git commit -m "Incluye costo de envío"  # nueva versión
git switch main
git pull                      # trae cambios de otros
git merge envio               # fusión → nueva versión
git tag v2.3.1                # línea base / lanzamiento
git push --follow-tags
```

1. El **clon** trae todo el repositorio a tu espacio de trabajo.
2. La **rama** permite desarrollar sin afectar la línea principal.
3. Cada **commit** es una versión con identificador único y queda en el historial.
4. El **merge** crea una versión nueva que combina las dos líneas.
5. La **etiqueta** marca la línea base que se entrega.

> Si dos personas cambian las mismas líneas, Git reporta un **conflicto** y alguien decide cómo fusionar.

Gestión del almacenamiento en sistemas VC distribuidos

Al crear una nueva versión, el sistema almacena un delta —una lista de diferencias— entre la nueva versión y la anterior. Git usa deltas dentro de archivos empaquetados para reducir aún más su tamaño.



Estructura de almacenamiento

Ejercicio: ¿cuánto espacio ahorran los deltas?

EJERCICIO

Un archivo de **100 KB** tiene 4 versiones (1.0 a 1.3). En cada versión cambian unos **2 KB**.

Estrategia	Qué guarda	Espacio
Copias completas	4 copias de 100 KB	**400 KB**
Deltas	La versión más reciente completa (100 KB) + 3 deltas de 2 KB	**106 KB**

Ahorro ≈ **73 %**. Para obtener la versión 1.0 se parte de la 1.3 y se aplican los deltas D3, D2 y D1 hacia atrás.



Así funciona la «estructura de almacenamiento» de la diapositiva: la versión más usada (la última) se lee directo; las antiguas se reconstruyen. Git, además, comprime los deltas en archivos empaquetados.

Es la característica de **gestión de almacenamiento**: no duplicar archivos idénticos.

Tema 2

Construcción de sistemas, gestión del cambio y gestión de versiones

Compilar el ejecutable, controlar los cambios y preparar lanzamientos



INGENIERÍA DE SOFTWARE II

Construcción de sistemas

Gestión del cambio · Gestión de versiones

CONTENIDO DEL MÓDULO

Agenda

01 

Construcción de sistemas

Compilar y vincular componentes en un ejecutable

02 

Gestión del cambio

Controlar la evolución del sistema

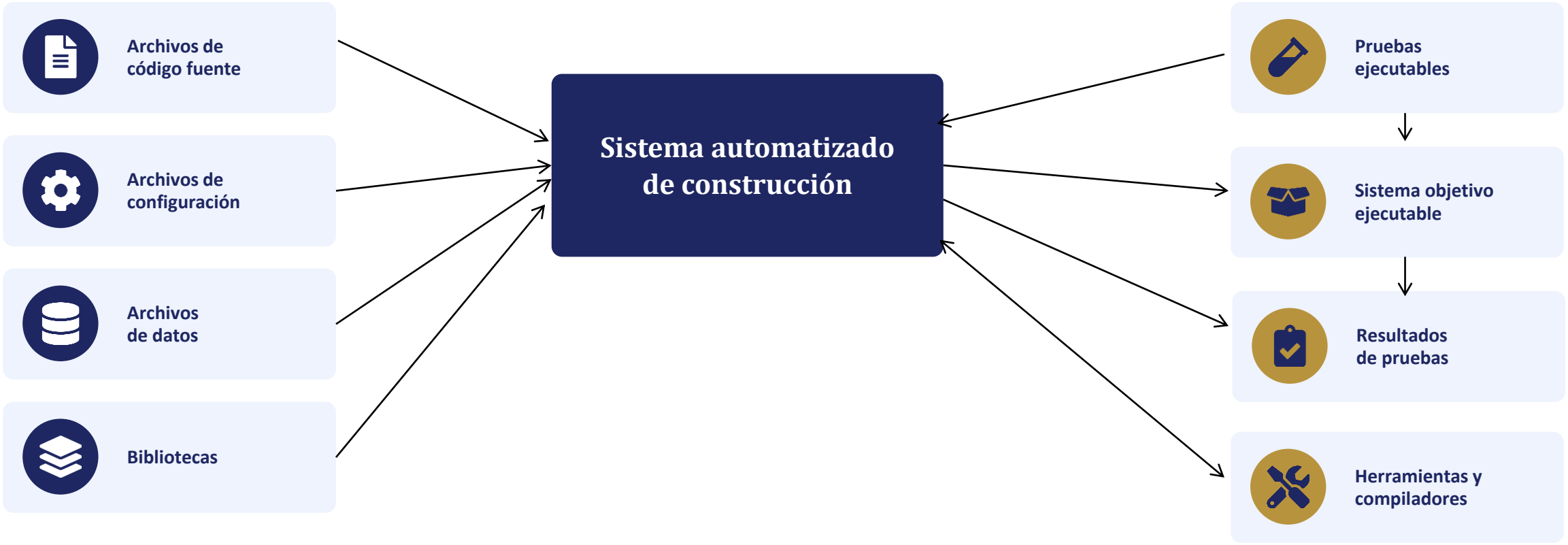
03 

Gestión de versiones

Preparar y distribuir lanzamientos

¿Qué es construir un sistema?

El proceso de crear un sistema completo y ejecutable mediante la compilación y vinculación de los componentes del sistema, bibliotecas externas, archivos de configuración y otra información.



Construir un sistema con un solo comando

EJEMPLO

```
$ mvn package
[INFO] Compiling 42 source files     # código fuente
[INFO] Copying 3 resources           # configuración y datos
[INFO] Downloading junit-5.10.jar    # bibliotecas externas
[INFO] Running tests...
[INFO] Tests run: 118, Failures: 0   # pruebas automáticas
[INFO] Building jar: tienda-2.3.1.jar
[INFO] BUILD SUCCESS
```

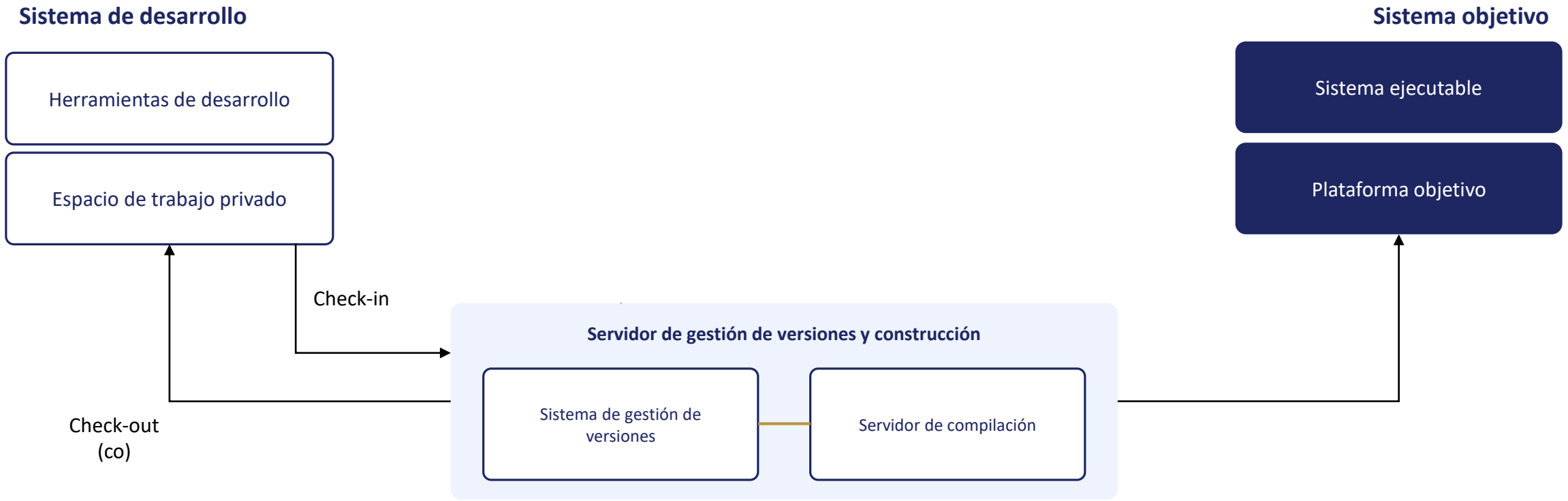
Una herramienta de construcción (Maven, Gradle, npm, make) lee un **script de compilación** y hace todo el trabajo: lo ideal de la diapositiva, «un solo comando o clic».

Entrada	Salida
Archivos de código fuente	Sistema ejecutable (.jar)
Archivos de configuración y de datos	Resultados de las pruebas
Bibliotecas (JUnit, drivers…)	Informe de éxito o fracaso
Pruebas ejecutables	Documentación generada

Recompilación mínima: si solo cambiaste `Pedido.java`, la herramienta compara fechas o firmas y recompila solo ese archivo y lo que depende de él, no los 42.

Plataformas del proceso de construcción

En el proceso de construcción pueden intervenir tres plataformas de sistemas diferentes:



Construir versiones ejecutables del sistema

Las tres plataformas, con un ejemplo

EJEMPLO

Sistema de desarrollo

La laptop de Ana: IDE, compilador y su **espacio de trabajo privado**. Hace *check-out* del código, lo modifica y prueba.

Servidor de versiones y construcción

GitHub + un servidor de CI. Recibe el *check-in*, guarda la versión y **construye** el ejecutable oficial.

Sistema objetivo

Donde corre el software: un servidor Linux, un celular Android o un cajero automático.

¿Por qué separarlos? Porque el sistema objetivo puede ser muy distinto al de desarrollo. Una app de Android se programa en Windows, pero se construye en el servidor y se ejecuta en el teléfono.

Pregunta típica: ¿dónde se genera la versión que se entrega al cliente? En el **servidor de construcción**, no en la máquina de un desarrollador: así se construye siempre igual y con las versiones registradas.

Integración continua

Los métodos ágiles recomiendan realizar compilaciones del sistema muy frecuentes y utilizar pruebas automatizadas para descubrir problemas de software.

Las compilaciones frecuentes son parte de un proceso de integración continua.

Herramientas de integración y construcción

Lo ideal sería poder construir un sistema completo con un solo comando o clic del mouse.

- **Generación de scripts de compilación**
- **Integración con el sistema de control de versiones**
- **Recompilación mínima del código fuente**
- **Creación de un sistema ejecutable**
- **Automatización de pruebas**
- **Informes de éxito o fracaso**
- **Generación de documentación**

MÓDULO 02



Gestión del cambio

Controlar la evolución del sistema a lo largo del tiempo

Una constante en todo sistema

Cambios inevitables

Las necesidades y requisitos organizacionales cambian durante la vida útil de un sistema; los errores deben repararse y los sistemas deben adaptarse a su entorno.

Un objetivo claro

Garantizar que se controle la evolución del sistema y que se prioricen los cambios más urgentes y rentables.

Un proceso definido

Analizar los costos y beneficios de los cambios propuestos, aprobar los rentables y rastrear qué componentes se han modificado.

El proceso de gestión del cambio

Cliente

- Envía solicitud de cambio (CR)

Soporte al cliente

- Revisa la CR
- Registra o cierra la CR

CCB · Desarrollo del producto

- Evalúa las CR
- Selecciona las CR
- Cierra las CR

Desarrollo

- Analiza implementación e impacto
- Modifica y prueba el software
- Cierra la CR

CCB: junta de control de cambios (Change Control Board)

La solicitud de cambio

Este proceso se inicia cuando una parte interesada del sistema completa y envía una solicitud de cambio que describe el cambio requerido para el sistema.

Formulario estándar de solicitud de cambio (CR)

Change Request Form

Proyecto:	SICSA/AppProcessing
Número:	23/02
Solicitante:	I. Sommerville
Fecha:	20/07/12
Cambio solicitado:	Mostrar visualmente el estado de los solicitantes (rechazado, aceptado, etc.)
Prioridad:	Media
Esfuerzo estimado:	2 horas
Decisión CCB:	Aceptar cambio · Release 1.2

Una solicitud de cambio (CR), llenada

EJEMPLO

Campo	Valor
Proyecto	Portal de matrícula UEES
Número	CR-2026-041
Solicitante	Secretaría académica
Cambio solicitado	Mostrar un aviso cuando una materia se cruza de horario con otra ya inscrita
Prioridad	Alta (en semana de matrícula)
Esfuerzo estimado	6 horas + pruebas
Componentes afectados	Módulo de horarios y pantalla de inscripción
Decisión del CCB	Aceptar · Release 3.4

Recorrido de la CR

1. **Cliente**: envía la CR.
2. **Soporte**: revisa que sea válida y la registra (o la cierra si está duplicada).
3. **Desarrollo**: analiza el impacto y el costo.
4. **CCB**: evalúa y selecciona las CR que se harán.
5. **Desarrollo**: modifica, prueba y cierra la CR.

Factores que influyen en la decisión

- Las consecuencias de no hacer el cambio
- Los beneficios del cambio
- El número de usuarios afectados
- Los costos de hacer el cambio
- El ciclo de lanzamiento del producto

Ejercicio: el CCB tiene que elegir

EJERCICIO

Solo queda tiempo para **una** CR antes del lanzamiento del viernes. Usa los factores de la diapositiva:

Factor	CR-A: el botón «Pagar» falla en iPhone	CR-B: modo oscuro
Consecuencias de no hacerlo	Clientes que no pueden comprar	Ninguna grave
Beneficios	Recuperar ventas	Comodidad visual
Usuarios afectados	≈ 35 % (usuarios de iPhone)	Quienes lo prefieran
Costo	4 horas	30 horas
Ciclo de lanzamiento	Cabe en este lanzamiento	Mejor en el siguiente

Se aprueba la **CR-A**: más usuarios afectados, consecuencias graves y bajo costo. La CR-B queda para una versión futura. El objetivo es **priorizar los cambios más urgentes y rentables**.

Cuatro clases de cambio

Clase 1

Corrección local

Corrige un error o mejora el contenido o la funcionalidad local.

Clase 2

Impacto funcional

Tiene impacto en otros objetos o componentes funcionales.

Clase 3

Impacto amplio

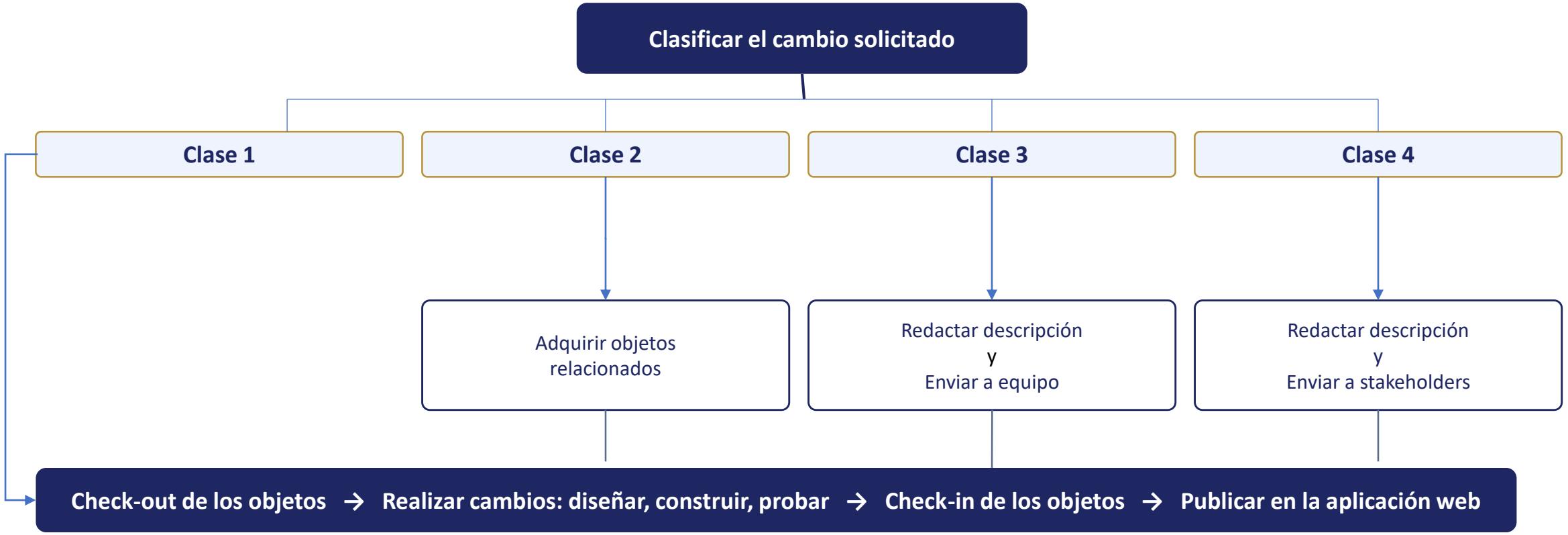
Impacto amplio en toda la aplicación: ampliación importante de la funcionalidad, mejora o reducción significativa del contenido, cambios en la navegación.

Clase 4

Cambio de diseño

Cambio de diseño importante (interfaz o navegación) inmediatamente perceptible para los usuarios.

Flujo de clasificación del cambio



Ejercicio: clasifica cada cambio de una aplicación web

EJERCICIO

Cambio solicitado	Clase	Por qué
Corregir una falta de ortografía en la página «Contacto»	**1** · Corrección local	Solo afecta ese contenido
Cambiar el cálculo del IVA, que usan el carrito, la factura y los reportes	**2** · Impacto funcional	Afecta a otros componentes
Agregar pagos con tarjeta y un historial de compras	**3** · Impacto amplio	Gran ampliación de funcionalidad y nueva navegación
Rediseñar el menú y toda la navegación del sitio	**4** · Cambio de diseño	Cambio de interfaz que el usuario nota de inmediato
Actualizar la foto de un producto	**1** · Corrección local	Mejora de contenido local

Según la clase cambia el flujo: la clase 1 va directo a check-out → cambio → check-in; la 2 primero reúne los objetos relacionados; las clases 3 y 4 necesitan una descripción escrita y la revisión del equipo o de los stakeholders.

MÓDULO 03



Gestión de versiones

Preparar, documentar y distribuir lanzamientos del sistema

Más que el código ejecutable

Una versión del sistema es una versión de un sistema de software que se distribuye a los clientes. Una versión de software no es solo el código ejecutable: también puede incluir:

Archivos de configuración

Definen cómo debe configurarse la versión para las instalaciones

Archivos de datos

Mensajes de error en distintos idiomas necesarios para el funcionamiento

Programa de instalación

Ayuda a instalar el sistema en el hardware de destino

Documentación

Electrónica y en papel que describe el sistema

Embalaje y publicidad

Diseñados específicamente para ese lanzamiento

Documentar un lanzamiento



Registrar las versiones

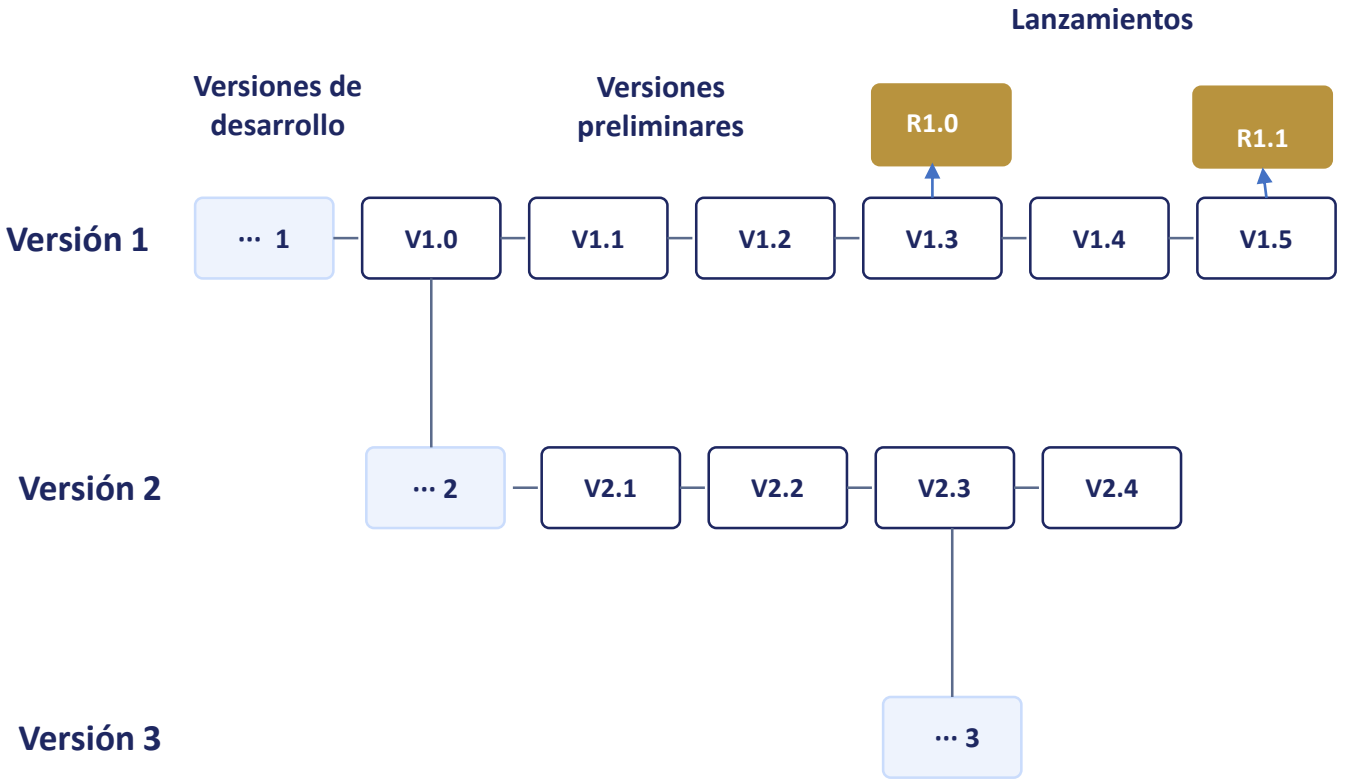
Para documentar un lanzamiento, debe registrar las versiones específicas de los componentes del código fuente que se utilizaron para crear el código ejecutable.



Conservar copias

Debe conservar copias de los archivos de código fuente, los ejecutables correspondientes y todos los archivos de datos y configuración. Puede ser necesario conservar copias de sistemas operativos antiguos y otro software de soporte, ya que podrían seguir en funcionamiento.

Versiones y lanzamientos



Versión vs. lanzamiento, con números

EJEMPLO

V1.0 | V1.1 | V1.2 | **V1.3 = R1.0** | V1.4 | **V1.5 = R1.1**

El equipo genera muchas **versiones** internas (desarrollo y preliminares). Solo algunas se convierten en **lanzamientos** que reciben los clientes.

> Toda versión entregada es un lanzamiento, pero no toda versión se lanza.

¿Qué incluye el lanzamiento R1.1?

- El ejecutable
- Archivos de configuración
- Archivos de datos (mensajes en varios idiomas)
- El instalador
- La documentación
- Embalaje y publicidad, si aplica

Para documentarlo se registran las versiones exactas de los componentes (por ejemplo, la etiqueta `v1.5`) y se guardan copias de fuentes, ejecutables y configuración.

Factores que influyen en un lanzamiento



Competencia

Para software de mercado masivo, puede ser necesario un nuevo lanzamiento si un producto competidor introdujo nuevas funciones y se corre el riesgo de perder cuota de mercado.



Requisitos de marketing

El departamento de marketing puede haberse comprometido a que los lanzamientos estén disponibles en una fecha concreta, o a incluir nuevas funciones para incentivar la actualización.



Cambios de plataforma

Puede ser necesario crear un nuevo lanzamiento cuando se libera una nueva versión de la plataforma del sistema operativo.



Calidad técnica del sistema

Si se reportan fallas graves que afectan a muchos clientes, puede ser necesario corregirlas en un nuevo lanzamiento; las fallas menores se reparan con parches.

Tema 3

Integración continua: conceptos y herramientas

Prácticas, beneficios, herramientas y conceptos relacionados



INGENIERÍA DE SOFTWARE II

Integración continua: conceptos y herramientas

Prácticas, beneficios, herramientas y conceptos relacionados

CONTENIDO DEL MÓDULO

Agenda

01 **Integración continua (CI) e implementación continua**

Conceptos, pasos, prácticas y herramientas

02 **Vídeos**

Recursos audiovisuales complementarios

03 **Reunión y proyecto Scrum**

Seguimiento del avance del equipo

PARTE 01



Integración continua

Terminología, pasos, prácticas y beneficios

Terminología

Construir

Un conjunto de actividades realizadas para generar, probar, inspeccionar e implementar software.

Continuo

En el contexto de la IC, no significa algo ininterrumpido, sino algo que ocurre repetidamente.

Integración

El acto de combinar artefactos de código fuente separados para determinar cómo funcionan como un todo.

Integración Continua (IC)

Una práctica de equipo

Los miembros de un equipo integran su trabajo con frecuencia.

Múltiples veces al día

Generalmente cada persona integra al menos una vez al día, dando lugar a múltiples integraciones diarias.

Verificación automatizada

Cada integración se verifica mediante una compilación automatizada (incluida una prueba) para detectar errores lo más rápido posible.

Un día con integración continua

EJEMPLO

Hora	Qué pasa	Servidor de CI
09:10	Ana integra su cambio en el formulario de registro	✓ **compila · 118 pruebas OK**
11:30	Luis integra un cambio en el cálculo de descuentos	× **falla 1 prueba**
11:45	Luis corrige de inmediato (práctica 6: reparar la compilación rota)	✓ **todo OK**
15:00	Marta integra la nueva pantalla de pagos	✓ **todo OK**
17:20	Cualquiera puede descargar el último ejecutable y probarlo	Versión lista para entregar

El error de Luis se descubrió **15 minutos** después de introducirlo, cuando el cambio era pequeño y estaba fresco.

Sin CI, se habría descubierto días después, al juntar el trabajo de todos: el temido «infierno de la integración».

Los pasos de la integración continua

1. Extraer el sistema principal del sistema VC (control de versiones) en el espacio de trabajo privado del desarrollador.

2. Construir el sistema y ejecutar pruebas automatizadas para garantizar que el sistema compilado supere todas las pruebas. De lo contrario, la compilación falla y el/los problema(s) deben solucionarse.

3. Realizar los cambios en los componentes del sistema.

4. Crear el sistema en un espacio de trabajo privado y volver a ejecutar las pruebas. Si las pruebas fallan, se continúa editando.

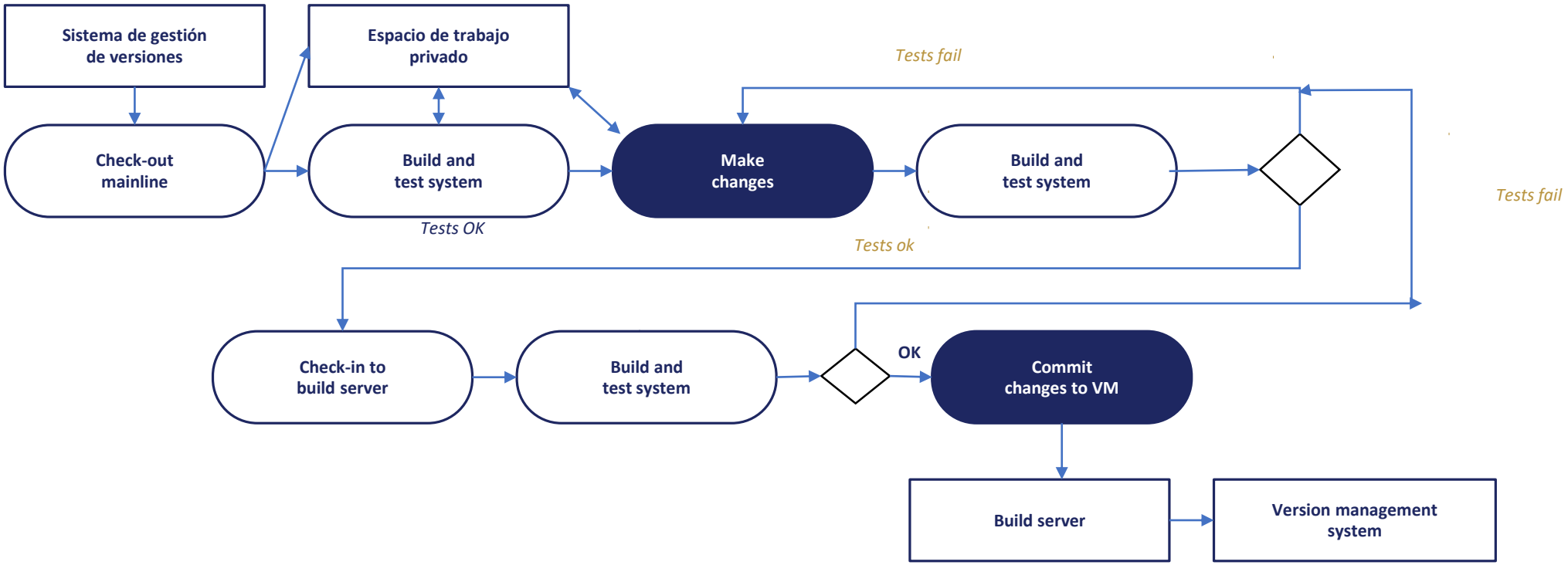
Pasos en la Integración Continua

5 Una vez que el sistema haya pasado sus pruebas, se registra en el servidor del sistema de compilación.

6 Se compila el sistema en el servidor de compilación y se ejecutan las pruebas. Si usa Git, también se pueden extraer los cambios recientes del servidor al espacio de trabajo privado, por si otros usuarios han modificado componentes desde el check-out. En ese caso, se revisan los componentes fallidos y se editan para que las pruebas se realicen correctamente.

7 Si el sistema pasa las pruebas en el sistema de compilación, se confirman los cambios realizados como una nueva línea base en la línea principal del sistema.

Pasos en la Integración Continua



Ejercicio: ordena los pasos de la integración continua

EJERCICIO

Desordenados

A. Hacer los cambios en los componentes.

B. Confirmar los cambios como nueva línea base.

C. Extraer la línea principal al espacio privado.

D. Construir en el servidor y ejecutar las pruebas.

E. Construir y probar en el espacio privado.

F. Registrar (check-in) en el servidor de compilación.

G. Construir y probar de nuevo en el espacio privado.

Orden correcto: C → E → A → G → F → D → B

1. C · check-out de la línea principal
2. E · construir y probar *antes* de tocar nada (¿la base está sana?)
3. A · hacer los cambios
4. G · construir y probar otra vez (si falla, se sigue editando)
5. F · check-in al servidor
6. D · construir y probar en el servidor
7. B · commit como nueva línea base

Prácticas clave de la integración continua

1. Mantener un único repositorio de fuentes.
2. Automatizar la compilación. Mantener la compilación rápida.
3. Hacer que la compilación se pruebe automáticamente.
4. Todos contribuyen a la línea principal todos los días.
5. Cada contribución debe construir la línea principal.
6. Reparar las compilaciones rotas inmediatamente.
7. Probar en un clon del entorno de producción.
8. Facilitar que cualquier persona obtenga el último ejecutable.
9. Todo el mundo puede ver lo que está pasando.
10. Automatizar la implementación.

Ejercicio: ¿qué práctica se está violando?

EJERCICIO

Situación	Práctica violada
El código está en tres repositorios y en una memoria USB	1 · Mantener un único repositorio de fuentes
Para compilar hay que seguir un documento de 20 pasos manuales	2 · Automatizar la compilación
Pedro integra su trabajo una vez al mes	4 · Todos contribuyen a la línea principal todos los días
La compilación lleva rota dos días y nadie la arregla	6 · Reparar las compilaciones rotas inmediatamente
Se prueba en Windows, pero producción corre en Linux	7 · Probar en un clon del entorno de producción
Solo el líder sabe si la última compilación pasó	9 · Todo el mundo puede ver lo que está pasando

Un pipeline de CI real (GitHub Actions)

EJEMPLO

```yaml
name: CI
on: [push, pull_request]        # en cada integración
jobs:
  build:
    runs-on: ubuntu-latest      # clon del entorno
    steps:
      - uses: actions/checkout@v4   # extrae el código
      - uses: actions/setup-java@v4
        with: { java-version: '21', distribution: 'temurin' }
      - run: mvn -B verify      # compila + pruebas
      - uses: actions/upload-artifact@v4
        with: { name: app, path: target/*.jar }
```

Línea del pipeline	Práctica clave
`on: push`	Cada contribución construye la línea principal
`mvn verify`	Compilación automatizada y que se prueba sola
`ubuntu-latest`	Probar en un entorno parecido a producción
`upload-artifact`	Cualquiera obtiene el último ejecutable
Pestaña «Actions»	Todo el mundo ve lo que está pasando

Beneficios de la Integración Continua

Reducir riesgos

Integrar muchas veces al día facilita una detección más temprana de defectos.

Reducir los procesos manuales repetitivos

Los procesos se ejecutarán cada vez que se produzca una confirmación en el repositorio de control de versiones.

Generar software implementable en cualquier momento

Realizar pequeños cambios en el código fuente e integrarlos con el resto de la base del código de manera regular.

Beneficios de la Integración Continua

Permitir una mejor visibilidad del proyecto

Un sistema CI puede proporcionar información en tiempo real sobre el estado reciente de la compilación y las métricas de calidad.

Generar mayor confianza en el producto de software

Con cada compilación, el equipo sabe que se ejecutan pruebas, que se cumplen los estándares de diseño y codificación, y que el resultado es un producto funcionalmente comprobable.

Limitaciones de la integración continua





Sistemas muy grandes

Si el sistema es muy grande, su desarrollo y pruebas pueden llevar mucho tiempo, especialmente si implica integración con otros sistemas de aplicaciones. Puede resultar poco práctico desarrollar el sistema varias veces al día.



Diferencias de plataforma

Si la plataforma de desarrollo es diferente a la de destino, es posible que no se puedan ejecutar pruebas en el espacio de trabajo privado del desarrollador. Puede haber diferencias en hardware, sistema operativo o software instalado, por lo que se requiere más tiempo para probar el sistema.

Algunas herramientas de CI

Jenkins

Herramienta CI multiplataforma y de código abierto, escrita en Java. Ofrece configuración vía GUI y comandos de consola.

Código abierto

Buddy

Herramienta de integración continua moderna e intuitiva; su interfaz sencilla minimiza el tiempo de aprendizaje.

Prueba gratuita / planes de pago

Bamboo

Servidor de compilación de integración continua que realiza compilaciones, pruebas y lanzamientos automáticos en un solo lugar.

Pagado con prueba gratuita

TeamCity

Servidor CI maduro de los laboratorios de JetBrains.

Gratis (3 agentes / 100 configs) · edición empresarial paga

Beneficios y limitaciones, con casos

EJEMPLO

Beneficios

- **Menos riesgo**: los defectos aparecen el mismo día.
- **Menos trabajo manual**: compilar y probar ocurre solo en cada commit.
- **Software implementable** en cualquier momento.
- **Visibilidad**: estado de la compilación y métricas en tiempo real.
- **Confianza**: cada compilación pasó las pruebas y los estándares.

Limitaciones

- **Sistemas muy grandes**: si compilar y probar un sistema bancario completo tarda 6 horas, no se puede hacer varias veces al día.
- **Diferencias de plataforma**: el software de un cajero automático no se puede probar completo en la laptop del desarrollador.

> Herramientas: **Jenkins** (libre, Java), **Bamboo**, **TeamCity** y **Buddy**.

PARTE 02



Conceptos relacionados

Entrega continua, despliegue continuo y sus diferencias

Entrega continua



Una extensión de la CI

Garantiza que se puedan lanzar nuevos cambios a los clientes rápidamente y de manera sostenible.



Lanzamiento automatizado

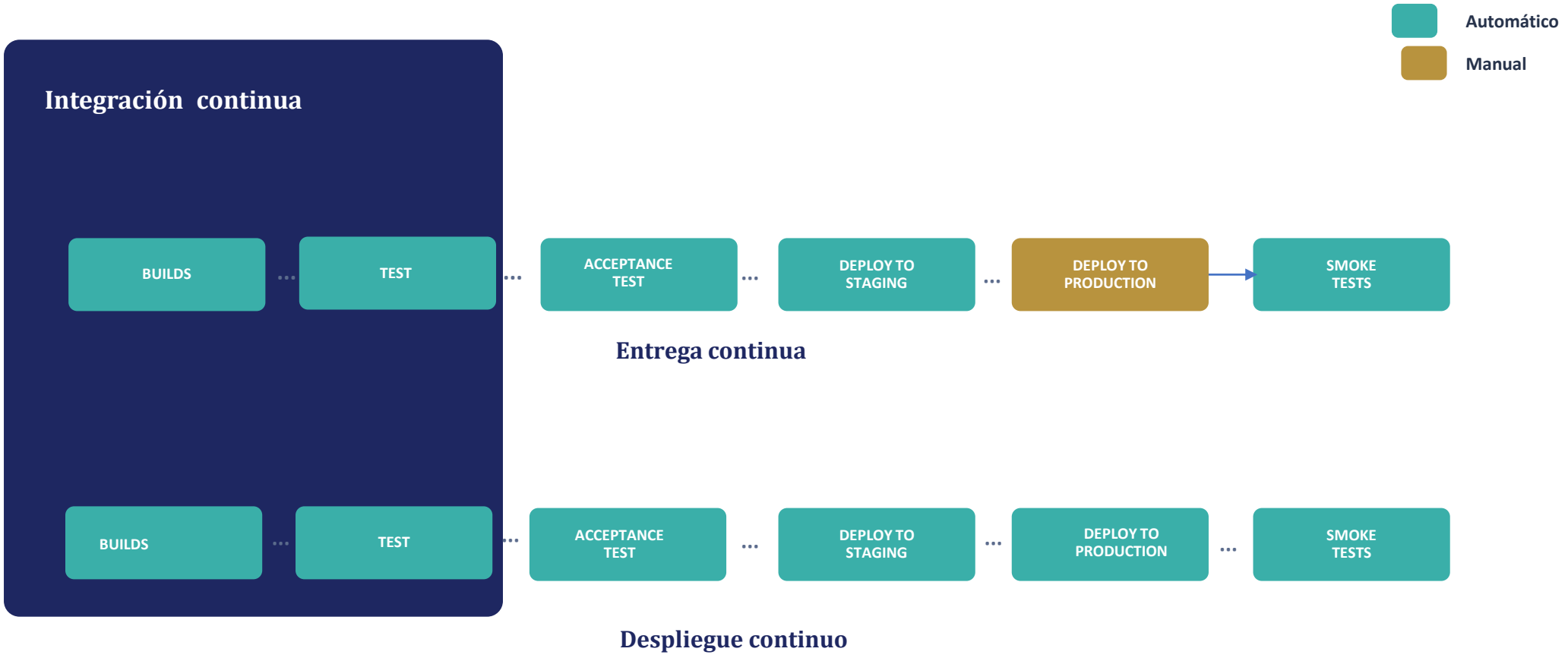
Además de automatizar las pruebas, se automatiza el proceso de lanzamiento: se puede implementar la aplicación en cualquier momento con un clic.



Ritmo flexible

Con la entrega continua se puede decidir lanzar diaria, semanal o quincenalmente, según lo que mejor se adapte al negocio.

Integración, entrega e implementación continuas



Cortesía de: atlassian.com/continuous-delivery/principles/continuous-integration-vs-delivery-vs-deployment

CONCEPTOS RELACIONADOS

Despliegue continuo



Un paso más allá de la entrega continua

Cada cambio que pasa por todas las etapas del flujo de producción se publica automáticamente para los clientes.



Sin intervención humana

Solo una prueba fallida impedirá que se implemente un nuevo cambio en producción.



Publicación inmediata

Los desarrolladores se centran en crear software y ven su trabajo publicado minutos después de terminarlo.

Integración, entrega y despliegue continuos

EJERCICIO

Integración continua

Cada cambio se integra y se verifica con compilación y pruebas automáticas.

Llega hasta: software probado.

Entrega continua

Además, el lanzamiento está automatizado: se puede publicar **en cualquier momento con un clic**. Una persona decide cuándo.

Despliegue continuo

Todo cambio que pasa todas las etapas se publica **automáticamente**. Solo una prueba fallida lo detiene.

Ejercicio: ¿cuál es cada caso?

Caso	Respuesta
Un banco tiene todo automatizado, pero un gerente aprueba cada publicación los martes	**Entrega continua**
Netflix publica en producción, sin intervención humana, cada cambio que pasa las pruebas	**Despliegue continuo**
El equipo compila y prueba en cada push, pero instala a mano en el servidor	**Integración continua**

Vídeos — ver individualmente (30 minutos)

Continuous Integration y Continuous Deployment: CI/CD

https://youtu.be/g0qSsex0Reg

¿Qué es CI/CD?

https://www.youtube.com/watch?v=N9KbmHhesmE

DevOps: Introducción a CI/CD (Integración Continua, Entrega Continua, Despliegue Continuo)

https://www.youtube.com/watch?v=AlrImm1T8Wg

Tema 4

Prácticas y estándares de codificación

Código limpio y el estándar de codificación Java

INGENIERÍA DE SOFTWARE II

Prácticas y estándares de codificación



Agenda

01



Prácticas para escribir código limpio

02



Estándares de codificación

OBJETIVO

Utilizar el estándar de codificación para mejorar la calidad del código en un proyecto determinado.



CALIDAD

Construcción de software

Calidad del software

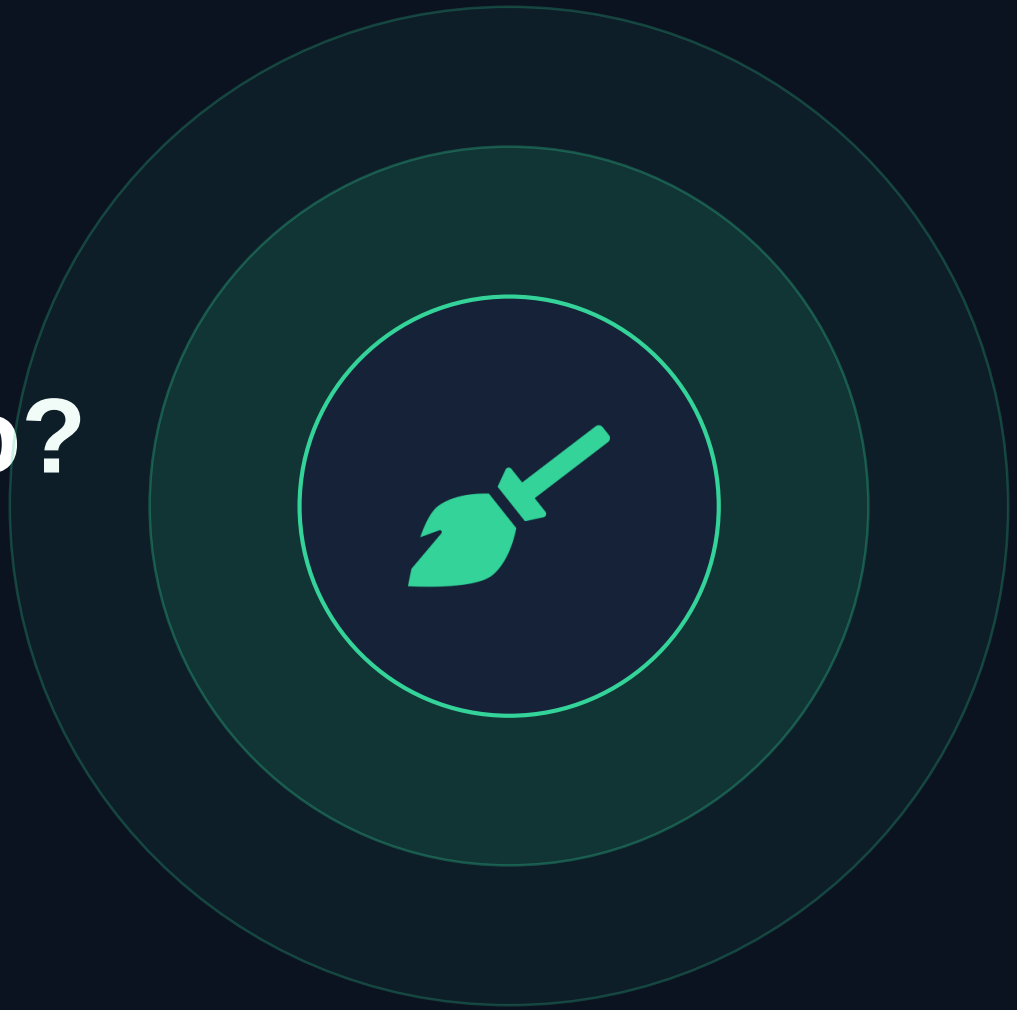
Mantenibilidad, extensibilidad

- **Legibilidad del código, Estilo de codificación**
- **Introducción de un nuevo comportamiento mediante la adición de código**

> **La calidad de la construcción afecta sustancialmente la calidad del software.**

INTRODUCCIÓN

¿Qué significa código limpio?



Habitación desordenada

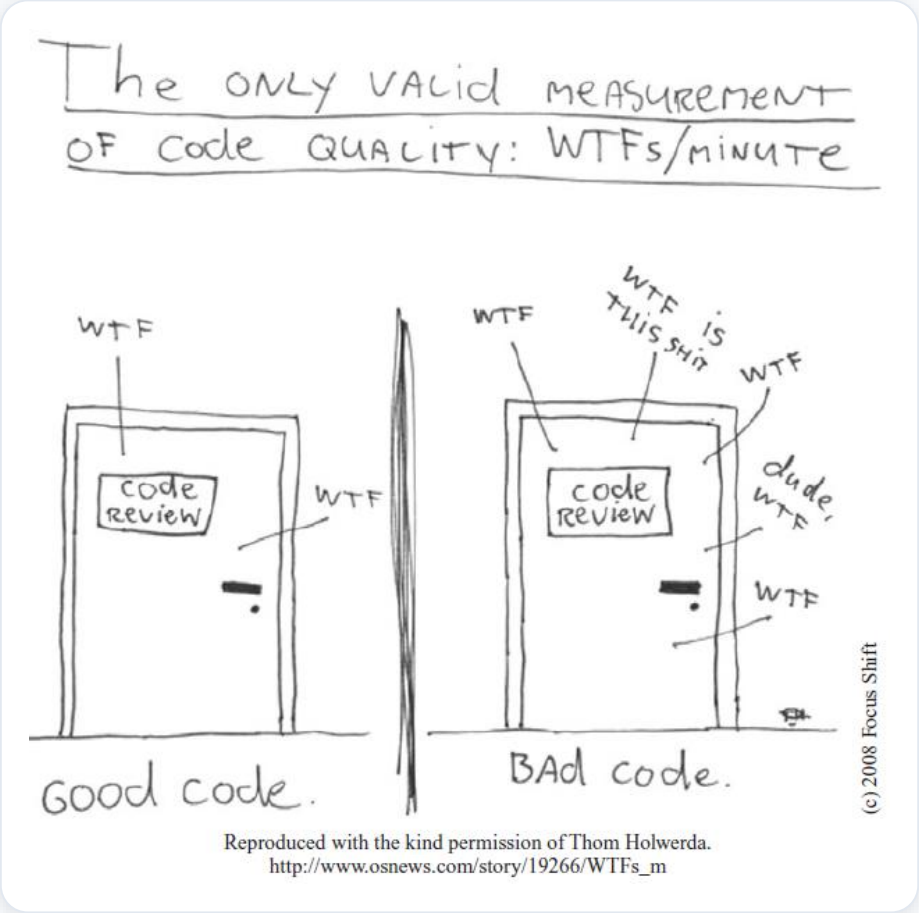




Calidad del código

WTF:

work that frustrate



Reproduced with the kind permission of Thom Holwerda.
http://www.osnews.com/story/19266/WTFs_m

Extractos de varias definiciones de código limpio



“El código limpio puede ser leído y **mejorado por un desarrollador distinto de su autor original**”.

Dave Thomas

Fundador de OTI, padrino de la estrategia Eclipse



“La lógica debe ser sencilla para dificultar **que los errores se oculten**, las dependencias mínimas para **facilitar el mantenimiento**, **el manejo total de errores** de acuerdo con una **estrategia articulada** y **un rendimiento cercano al óptimo**”.

Bjarne Stroustrup

Inventor de C++ y autor de El lenguaje de programación C++



“Sabes que estás trabajando en código limpio cuando **cada rutina** que lees resulta ser **prácticamente lo que esperabas**”.

Ward Cunningham

PARTE 1

Prácticas para escribir código limpio



Categorías de prácticas de escritura de código limpio



01

Nombres con significado

variables, funciones, argumentos, clases, paquetes, directorios de archivos fuente, etc.



02

Funciones

Escribiéndolas bien.



03

Comentarios

Uso adecuado de los comentarios.

Categorías de prácticas de escritura de código limpio



04

Formato

Elija un conjunto de **reglas simples** que rijan el formato de su código y aplíquelas (el equipo debe estar de acuerdo)



05

Objetos y estructuras de datos

Mantener las variables privadas.



06

Manejo de errores

Categorías de prácticas de escritura de código limpio



07

Límites

Debemos integrar limpiamente el código ajeno con el nuestro.



08

Pruebas unitarias

Escribiendo buenas pruebas



09

Clases

No tenemos código limpio hasta que hayamos **prestado atención** a niveles superiores de organización del código, como las clases.

Nombres con significado

Utilice nombres que revelen intenciones

```
int elapsedTimeInDays;
int daysSinceCreation;
int daysSinceModification;
int fileAgeInDays;
```

Evite la desinformación o las confusiones

No utilice nombres que varíen ligeramente

```
XYZControllerForEfficientHandlingOfStrings
```

```
XYZControllerForEfficientStorageOfStrings
```

Nombres con significado

Utilice nombres pronunciables

EVITAR

```
class DtaRcrd102 {
  private Date genymdhms;
  private Date modymdhms;
  private final String pszqint = "102";
  /* ... */
};
```

PREFERIR

```
class Customer {
  private Date generationTimestamp;
  private Date modificationTimestamp;
  private final String recordId = "102";
  /* ... */
};
```

Utilice nombres que permitan realizar búsquedas

Nombres con significado

Evite las codificaciones

Nombre **sin prefijos**. Por ejemplo, en lugar de llamar a una interfaz *IShapeFactory*, llámela *ShapeFactory*.

```
IShapeFactory  →  ShapeFactory
```

Elija una palabra por concepto

Elige **una palabra** para un concepto abstracto y **apéguese a ella**.

Ejemplo: cliente, consumidor ---- elija uno

```
cliente  o  consumidor
```

Nombres con significado: antes y después

EJEMPLO

Antes: ¿qué calcula? ¿qué es t == 1?

```
public double calc(List<Double> l, int t) {
    double d = 0;
    for (double x : l) d += x;
    if (t == 1) d = d * 0.88;
    return d;
}
```

- `calc`, `l`, `d`: no revelan la intención.
- `1` y `0.88`: números mágicos imposibles de buscar.

Después: se lee como una frase

```
public double totalConDescuento(List<Double> precios,
                                TipoCliente tipo) {
    double total = sumar(precios);
    if (tipo == TipoCliente.FRECUENTE) {
        total = total * (1 - DESCUENTO_FRECUENTE);
    }
    return total;
}
```

> Nombres que revelan la intención, pronunciables y que se pueden buscar (`DESCUENTO_FRECUENTE`), y la suma extraída a su propia función.

Funciones

Las funciones siempre deben ser pequeñas.

Que haga una sola cosa.

Argumentos de función

0 1 2 3

Prefiera 0, 1 o 2 argumentos. Se deben evitar tres argumentos siempre que sea posible.

Es difícil escribir todos los casos de prueba para garantizar que todas las distintas combinaciones de argumentos funcionen correctamente.

Funciones



Sin efectos secundarios

Su función debe hacer una cosa, pero a veces también hace otras cosas ocultas (por ejemplo, cambios inesperados en las variables de su propia clase).



Comando - Separación de consultas

Las funciones deben cambiar el estado de un objeto o devolver alguna información sobre él, pero no ambas cosas.



Preferir excepciones a la devolución de códigos de error

Reemplazar el código de error con una excepción

Funciones



No repita código



La repetición de código es un problema, ya que requerirá múltiples modificaciones si el algoritmo requiere un cambio.



También representa una oportunidad para replicar un error.

Funciones pequeñas que hacen una sola cosa

EJEMPLO

Una función que hace cuatro cosas

```
void procesarPedido(Pedido p) {
    // 1. validar
    if (p.getItems().isEmpty()) ...
    // 2. calcular total
    for (Item i : p.getItems()) ...
    // 3. guardar en BD
    db.insert(p) ...
    // 4. enviar correo
    mail.send(p.getCliente()) ...
}
```

Cada paso es una función con nombre

```
void procesarPedido(Pedido p) {
    validar(p);
    double total = calcularTotal(p);
    guardar(p, total);
    notificarCliente(p);
}
```

Si cambia la forma de enviar correos, solo tocas `notificarCliente`. Cada función se prueba por separado.

Argumentos

`crearUsuario(nombre, apellido, correo, edad, ciudad)` → agrupa los datos en un objeto: `crearUsuario(datos)`. Prefiere 0, 1 o 2 argumentos.

No repitas código

Si el cálculo del IVA está copiado en 5 lugares, un cambio de tasa obliga a editar 5 veces, y basta olvidar una para crear un error.

Separación comando-consulta y excepciones

EJEMPLO

Viola la separación comando-consulta

```
// Cambia el estado Y devuelve información
boolean setNombre(String nombre) {
    if (nombre.isBlank()) return false;
    this.nombre = nombre;
    return true;
}
if (setNombre("Ana")) { ... }  // confuso
```

Mejor dos funciones: una **consulta** `esNombreValido(n)` que solo responde, y un **comando** `setNombre(n)` que solo cambia el estado.

Preferir excepciones a códigos de error

```
// Excepción en lugar de código de error
void retirar(double monto) {
    if (monto > saldo)
        throw new SaldoInsuficienteException(
            "Saldo " + saldo + ", pedido " + monto);
    saldo -= monto;
}
```

En vez de devolver `-1` (que el llamador puede ignorar), se lanza una excepción con **contexto**: el saldo y el monto pedido. Así se sabe dónde y por qué falló.

Comentarios



Los comentarios no compensan el código malo.

Una de las motivaciones más comunes para escribir comentarios es el código incorrecto.

Buenos comentarios

- Comentarios legales
- Comentarios informativos
- Explicación de la intención
- Clarificación
- Advertencia de consecuencias

Comentarios

Malos – comentarios innecesarios

Comentarios sobre el ruido

```
/**
 * Default constructor.
 */
protected AnnualDateRule() {
}
```

Marcadores de posición

```
// Actions ////////////////////////////
```

Comentarios de llave de cierre

```
  while ((line = in.readLine()) != null) {
    lineCount++;
    charCount += line.length();
    String words[] = line.split("\\W");
    wordCount += words.length;
  } //while
  System.out.println("wordCount = " + wordCount);
  System.out.println("lineCount = " + lineCount);
  System.out.println("charCount = " + charCount);
} // try
catch (IOException e) {
  System.err.println("Error:" + e.getMessage());
} //catch
} //main
```

Ejercicio: ¿comentario bueno o malo?

EJERCICIO

Comentario	Veredicto
`// Copyright 2026 UEES. Licencia MIT.`	**Bueno** · legal
`i++; // incrementa i`	**Malo** · ruido: repite el código
`// Formato esperado: dd/MM/yyyy (ej. 05/09/2026)`	**Bueno** · informativo
`// Se ordena primero para usar búsqueda binaria`	**Bueno** · explica la intención
`// ¡Cuidado! Esta prueba tarda 10 minutos`	**Bueno** · advierte consecuencias
`} // fin del while`	**Malo** · llave de cierre
`// ////////// Acciones //////////`	**Malo** · marcador de posición
`// x es el total menos el descuento` sobre `double x = a - b;`	**Malo** · compensa un mal nombre: renombra

Regla: los comentarios no compensan el código malo. Si necesitas explicar qué hace, primero intenta que el código lo diga.

Formato

Formato vertical

- En las partes más altas del archivo fuente: conceptos y algoritmos de alto nivel.
- A medida que avanzamos hacia abajo, los detalles deberían aumentar.

Formato horizontal

- ¿Qué ancho debe tener una línea?
- El antiguo límite de Hollerith es 80. Las líneas que se extienden hasta 100 o incluso 120 pueden ser aceptables.

Sangría

Objetos y estructuras de datos



Abstracción de datos

- Los usuarios pueden manipular la esencia de los datos sin tener que conocer su implementación.
- Una clase **expone** interfaces abstractas

LISTING 6-1 · CONCRETE POINT

```
public class Point {
  public double x;
  public double y;
}
```

LISTING 6-2 · ABSTRACT POINT

```
public interface Point {
  double getX();
  double getY();
  void setCartesian(double x, double y);
  double getR();
  double getTheta();
  void setPolar(double r, double theta);
}
```

Manejo de errores

Utilice excepciones en lugar de devolver códigos de error

Proporcionar contexto con excepciones

Cada excepción que lance debe proporcionar suficiente contexto para determinar el origen y la ubicación de un error.

Definir clases de excepción en términos de las necesidades del llamador

Separe su lógica de negocios de su manejo de errores.

No devolver nulo

No pases nulo

Manejo de errores: no devolver ni pasar null

EJEMPLO

```
// × Devuelve null: cada llamador debe acordarse de verificarlo
List<Pedido> buscar(String cliente) {
    if (!existe(cliente)) return null;
    ...
}
for (Pedido p : buscar("Ana")) ...    // NullPointerException

// ✓ Devuelve una lista vacía
List<Pedido> buscar(String cliente) {
    if (!existe(cliente)) return List.of();
    ...
}
```

Las reglas de la diapositiva

- Excepciones en lugar de códigos de error.
- Excepciones con contexto suficiente.
- Clases de excepción pensadas para el llamador.
- **No devolver null**: devuelve una colección vacía, un objeto por defecto u `Optional`.
- **No pasar null** como argumento.

Un `null` olvidado es la causa del `NullPointerException`, uno de los errores más comunes en Java.

Límites



Explorando y aprendiendo límites

- No es nuestro trabajo probar el código de terceros, pero puede ser de nuestro interés escribir pruebas para el código de terceros que usamos.
- Escribir pruebas es una forma sencilla de adquirir ese conocimiento.



Límites limpios

Gestionamos los límites de terceros al tener muy pocos lugares en el código que hagan referencia a ellos.

Pruebas unitarias

Una afirmación por prueba

Listing 9-7
SerializedPageResponderTest.java (Single Assert)

```
public void testGetPageHierarchyAsXml() throws Exception {
    givenPages("PageOne", "PageOne.ChildOne", "PageTwo");

    whenRequestIsIssued("root", "type:pages");

    thenResponseShouldBeXML();
  }

  public void testGetPageHierarchyHasRightTags() throws Exception {
    givenPages("PageOne", "PageOne.ChildOne", "PageTwo");

    whenRequestIsIssued("root", "type:pages");

    thenResponseShouldContain(
      "<name>PageOne</name>", "<name>PageTwo</name>", "<name>ChildOne</name>"
    );
  }
```

Concepto único por prueba

Listing 9-8

```
/**
 * Miscellaneous tests for the addMonths() method.
 */
public void testAddMonths() {
    SerialDate d1 = SerialDate.createInstance(31, 5, 2004);

    SerialDate d2 = SerialDate.addMonths(1, d1);
    assertEquals(30, d2.getDayOfMonth());
    assertEquals(6, d2.getMonth());
    assertEquals(2004, d2.getYYYY());

    SerialDate d3 = SerialDate.addMonths(2, d1);
    assertEquals(31, d3.getDayOfMonth());
    assertEquals(7, d3.getMonth());
    assertEquals(2004, d3.getYYYY());

    SerialDate d4 = SerialDate.addMonths(1, SerialDate.addMonths(1, d1));
    assertEquals(30, d4.getDayOfMonth());
    assertEquals(7, d4.getMonth());
    assertEquals(2004, d4.getYYYY());
}
```

Clases



Organización de clases

Siga la convención de codificación estándar

Encapsulación: mantener las variables y funciones de utilidad privadas, pero sin fanatismo.



Las clases deben ser pequeñas

El principio de responsabilidad única

Cohesión: una alta cohesión significa que los métodos y las variables de la clase son codependientes y se mantienen unidos como un todo lógico.



Organizarse para el cambio

Clases pequeñas: el principio de responsabilidad única

EJEMPLO

✕ Clase «Dios»: Estudiante

- guarda nombre, cédula y carrera
- calcula el promedio
- genera el certificado en PDF
- envía correos
- se conecta a la base de datos

Tiene **cinco razones para cambiar**: baja cohesión.

✓ Una responsabilidad por clase

- `Estudiante`: datos del estudiante
- `CalculadoraPromedio`
- `GeneradorCertificado`
- `Notificador`
- `EstudianteRepositorio`

Cada clase es pequeña, cohesiva y **organizada para el cambio**.

Alta cohesión: los métodos y variables de la clase dependen entre sí y forman un todo lógico. Encapsulación: lo interno queda privado, «pero sin fanatismo».

PARTE 2

¿Qué es un estándar de codificación?



¿Qué es un estándar de codificación?



Es una guía que define un estilo de programación.



Garantiza que todos los desarrolladores escriban código de acuerdo con las pautas



Proporciona legibilidad (código fácil de entender)



Impacta positivamente en la calidad del sistema



Reduce la posibilidad de introducir vulnerabilidades de seguridad o problemas de rendimiento.

¿Por qué necesitamos un estándar de codificación?

80%

del coste de vida útil de un software se destina a mantenimiento.



Casi ningún software recibe mantenimiento durante toda su vida por parte del autor original.

Aspectos comunes de un estándar de codificación



Convenciones de nomenclatura



Nombre y organización de archivos



Formato y sangría



Comentarios y documentación



Clases, funciones e interfaces



Manejo de excepciones / registro



Etc

EJEMPLO PRÁCTICO

Ejemplo: El estándar de codificación Java



El estándar de codificación Java

La estructura del documento

1 Introduction
- 1.1 Why Have Code Conventions
- 1.2 Acknowledgments

2 File Names
- 2.1 File Suffixes
- 2.2 Common File Names

3 File Organization
- 3.1 Java Source Files
  - 3.1.1 Beginning Comments
  - 3.1.2 Package and Import Statements
  - 3.1.3 Class and Interface Declarations

4 Indentation
- 4.1 Line Length
- 4.2 Wrapping Lines

5 Comments
- 5.1 Implementation Comment Formats
  - 5.1.1 Block Comments
  - 5.1.2 Single-Line Comments
  - 5.1.3 Trailing Comments
  - 5.1.4 End-Of-Line Comments
- 5.2 Documentation Comments

6 Declarations
- 6.1 Number Per Line
- 6.2 Initialization
- 6.3 Placement
- 6.4 Class and Interface Declarations

7 Statements
- 7.1 Simple Statements
- 7.2 Compound Statements
- 7.3 return Statements
- 7.4 if, if-else, if else-if else Statements
- 7.5 for Statements
- 7.6 while Statements
- 7.7 do-while Statements
- 7.8 switch Statements
- 7.9 try-catch Statements

8 White Space
- 8.1 Blank Lines
- 8.2 Blank Spaces

9 Naming Conventions

10 Programming Practices
- 10.1 Providing Access to Instance and Class Variables
- 10.2 Referring to Class Variables and Methods
- 10.3 Constants
- 10.4 Variable Assignments
- 10.5 Miscellaneous Practices
  - 10.5.1 Parentheses
  - 10.5.2 Returning Values
  - 10.5.3 Expressions before '?' in the Conditional Operator
  - 10.5.4 Special Comments

11 Code Examples
- 11.1 Java Source File Example

EJEMPLO PRÁCTICO

El estándar de codificación Java

SECCIONES

3 · File Organization

3 - File Organization

A file consists of sections that should be separated by blank lines and an optional comment identifying each section.

Files longer than 2000 lines are cumbersome and should be avoided.

For an example of a Java program properly formatted, see “Java Source File Example” on page 19.

EJEMPLO PRÁCTICO

El estándar de codificación Java

SECCIONES

- 4 · Indentation
- 4.1 Line Length
- 4.2 Wrapping Lines

4 - Indentation

Four spaces should be used as the unit of indentation. The exact construction of the indentation (spaces vs. tabs) is unspecified. Tabs must be set exactly every 8 spaces (not 4).

4.1 Line Length

Avoid lines longer than 80 characters, since they're not handled well by many terminals and tools.

Note: Examples for use in documentation should have a shorter line length—generally no more than 70 characters.

4.2 Wrapping Lines

When an expression will not fit on a single line, break it according to these general principles:

- Break after a comma.
- Break before an operator.
- Prefer higher-level breaks to lower-level breaks.
- Align the new line with the beginning of the expression at the same level on the previous line.
- If the above rules lead to confusing code or to code that's squished up against the right margin, just indent 8 spaces instead.

EJEMPLO PRÁCTICO

El estándar de codificación Java

SECCIONES

- 5.1.2 Single-Line Comments
- 5.1.3 Trailing Comments

5.1.2 *Single-Line Comments*

Short comments can appear on a single line indented to the level of the code that follows. If a comment can't be written in a single line, it should follow the block comment format (see section 5.1.1). A single-line comment should be preceded by a blank line. Here's an example of a single-line comment in Java code (also see "Documentation Comments" on page 9):

```
if (condition) {

    /* Handle the condition. */
    ...
}
```

5.1.3 *Trailing Comments*

Very short comments can appear on the same line as the code they describe, but should be shifted far enough to separate them from the statements. If more than one short comment appears in a chunk of code, they should all be indented to the same tab setting. Avoid the assembly language style of commenting every line of executable code with a trailing comment.

Here's an example of a trailing comment in Java code (also see "Documentation Comments" on page 9):

```
if (a == 2) {
    return TRUE;            /* special case */
} else {
    return isprime(a);      /* works only for odd a */
}
```

EJEMPLO PRÁCTICO

El estándar de codificación Java



SECCIONES

- 7.2 Compound Statements
- 7.3 return Statements

7.2 Compound Statements

Compound statements are statements that contain lists of statements enclosed in braces "`{ statements }`". See the following sections for examples.

- The enclosed statements should be indented one more level than the compound statement.
- The opening brace should be at the end of the line that begins the compound statement; the closing brace should begin a line and be indented to the beginning of the compound statement.
- Braces are used around all statements, even singletons, when they are part of a control structure, such as a `if-else` or `for` statement. This makes it easier to add statements without accidentally introducing bugs due to forgetting to add braces.

7.3 return Statements

A `return` statement with a value should not use parentheses unless they make the return value more obvious in some way. Example:

```
return;

return myDisk.size();

return (size ? size : defaultSize);
```

EJEMPLO PRÁCTICO

El estándar de codificación Java

SECCIONES

7.4 if, if-else, if-else-if

7.4 if, if-else, if-else-if Statements

The `if-else` class of statements should have the following form:

```
if (condition) {
    statements;
}

if (condition) {
    statements;
} else {
    statements;
}

if (condition) {
    statements;
} else if (condition) {
    statements;
} else if (condition) {
    statements;
}
```

EJEMPLO PRÁCTICO

El estándar de codificación Java

Revisemos algunas secciones…

8.1 Blank Lines

Blank lines improve readability by setting off sections of code that are logically related.

Two blank lines should always be used in the following circumstances:

- Between sections of a source file
- Between class and interface definitions

One blank line should always be used in the following circumstances:

- Between methods
- Between the local variables in a method and its first statement
- Before a block (see section 5.1.1) or single-line (see section 5.1.2) comment
- Between logical sections inside a method to improve readability

8.1 BLANK LINES

8.2 BLANK SPACES

8.2 Blank Spaces

Blank spaces should be used in the following circumstances:

- A keyword followed by a parenthesis should be separated by a space. Example:

```
while (true) {
    ...
}
```

Note that a blank space should not be used between a method name and its opening parenthesis. This helps to distinguish keywords from method calls.

- A blank space should appear after commas in argument lists.
- All binary operators except . should be separated from their operands by spaces. Blank spaces should never separate unary operators such as unary minus, increment ("++"), and decrement ("--") from their operands. Example:

```
a += c + d;
a = (a + b) / (c * d);

while (d++ = s++) {
    n++;
}
printSize("size is " + foo + "\n");
```

- The expressions in a `for` statement should be separated by blank spaces. Example:

```
for (expr1; expr2; expr3)
```

- Casts should be followed by a blank space. Examples:

```
myMethod((byte) aNum, (Object) x);
myMethod((int) (cp + 5), ((int) (i + 3))
                                + 1);
```

EJEMPLO PRÁCTICO

El estándar de codificación Java

SECCIONES

9 · Naming Conventions

9 - Naming Conventions

Identifier Type	Rules for Naming	Examples
Classes	Class names should be nouns, in mixed case with the first letter of each internal word capitalized. Try to keep your class names simple and descriptive. Use whole words—avoid acronyms and abbreviations (unless the abbreviation is much more widely used than the long form, such as URL or HTML).	`class Raster;` `class ImageSprite;`
Interfaces	Interface names should be capitalized like class names.	`interface RasterDelegate;` `interface Storing;`

Methods	Methods should be verbs, in mixed case with the first letter lowercase, with the first letter of each internal word capitalized.	`run();`
		`runFast();`
		`getBackground();`

Variables	Except for variables, all instance, class, and class constants are in mixed case with a lowercase first letter. Internal words start with capital letters.
	
	Variable names should be short yet meaningful. The choice of a variable name should be mnemonic— that is, designed to indicate to the casual observer the intent of its use. One-character variable names should be avoided except for temporary "throwaway" variables. Common names for temporary variables are `i`, `j`, `k`, `m`, and `n` for integers; `c`, `d`, and `e` for characters.
		`int i;`
		`char *cp;`
		`float myWidth;`

Constants	The names of variables declared class constants and of ANSI constants should be all uppercase with words separated by underscores ("_"). (ANSI constants should be avoided, for ease of debugging.)	`int MIN_WIDTH = 4;`
		`int MAX_WIDTH = 999;`
		`int GET_THE_CPU = 1;`

EJEMPLO PRÁCTICO

El estándar de codificación Java

SECCIONES

- 10.5 Miscellaneous Practices
- 10.5.1 Parentheses
- 10.5.2 Returning Values

10.5 Miscellaneous Practices

10.5.1 Parentheses

It is generally a good idea to use parentheses liberally in expressions involving mixed operators to avoid operator precedence problems. Even if the operator precedence seems clear to you, it might not be to others—you shouldn't assume that other programmers know precedence as well as you do.

```
if (a == b && c == d)     // AVOID!

if ((a == b) && (c == d)) // RIGHT
```

10.5.2 Returning Values

Try to make the structure of your program match the intent. Example:

```
if (booleanExpression) {
    return TRUE;
} else {
    return FALSE;
}
```

should instead be written as

```
return booleanExpression;
```

Similarly,

```
if (condition) {
    return x;
}
return y;
```

should be written as

```
return (condition ? x : y);
```

Ejercicio: encuentra las violaciones al estándar Java

EJERCICIO

Código original

```
1 package Ventas;
2 import java.util.*;
3 public class pedido_cliente {
4     public static final double iva = 0.15;
5     private int Cantidad; private double precio;
6     public double CalcularTotal(){ return Cantidad*precio*(1+iva); }
7 }
```

L1 paquete en minúsculas: `ventas` · L2 evitar `*` · L3 clases en *UpperCamelCase* · L4 constantes en MAYÚSCULAS · L5 una declaración por línea y variables en *lowerCamelCase* · L6 métodos en *lowerCamelCase*, espacios alrededor de operadores, línea demasiado larga y cuerpo en varias líneas.

Corregido según las convenciones

```
package ventas;

import java.util.List;

public class PedidoCliente {
    public static final double IVA = 0.15;
    private int cantidad;
    private double precio;

    public double calcularTotal() {
        return cantidad * precio * (1 + IVA);
    }
}
```

Tema 5

Inspección de código y análisis estático

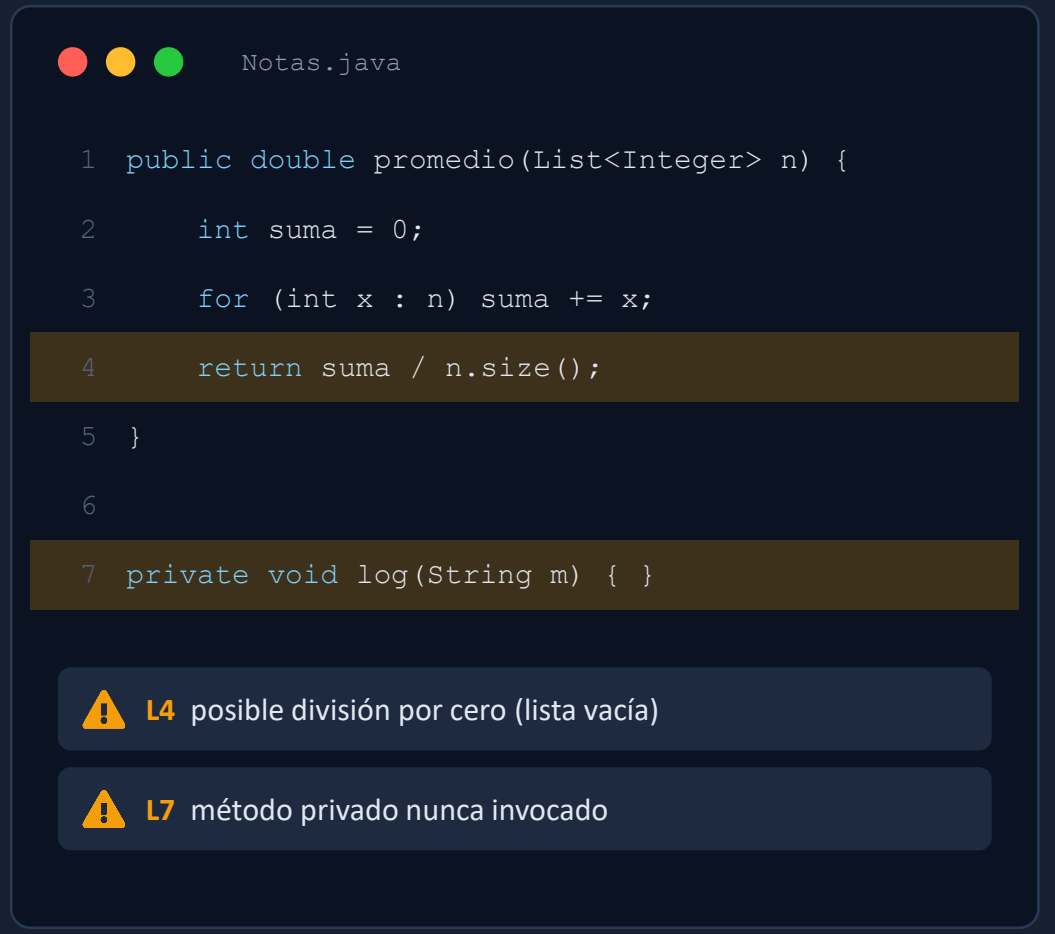
Análisis estático, revisión de código y herramientas

INGENIERÍA DE SOFTWARE II

Code Inspection

Inspección de código y análisis estático de programas

Análisis estático · Revisión de código · Herramientas



CONTENIDO

Agenda

01



Análisis de programas informáticos

Enfoques estático y dinámico

02



Inspección de código

Definición, atributos de calidad y defectos que detecta

03



Ventajas y limitaciones

Beneficios, límites y comparación con las pruebas

04



Herramientas de inspección

Características clave, categorías, PMD y FindBugs

01

Análisis de programas informáticos



Cómo se estudia el comportamiento de un programa: con y sin ejecutarlo

Análisis de programas informáticos

Proceso de analizar **automáticamente** el comportamiento de los programas informáticos.

PERSIGUE DOS OBJETIVOS

- **Optimización**
- **Corrección**

DOS ENFOQUES PRINCIPALES

Análisis estático de programas

Examina el código fuente sin ejecutarlo.

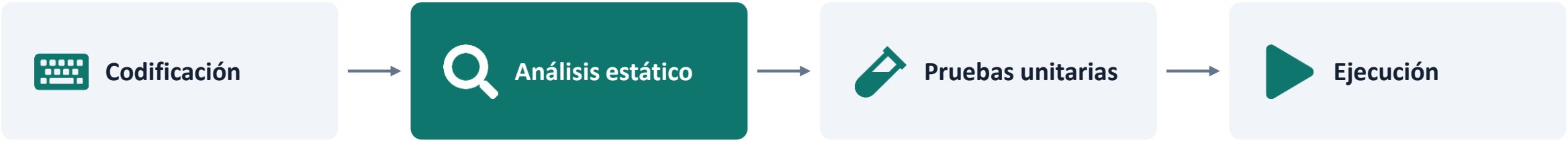
Análisis dinámico de programas

Analiza el programa mientras se ejecuta.

Análisis estático: definición

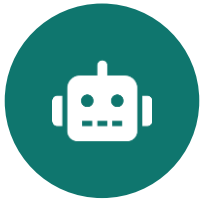
> **Examinar el código fuente** de un programa con la intención de descubrir debilidades **sin ejecutarlo.**

¿CUÁNDO SE REALIZA?



Se realiza **después de la codificación** y **antes de ejecutar las pruebas unitarias**. Por lo tanto, identifica los defectos antes de que el programa se ejecute.

¿Quién realiza el análisis estático?



Una máquina

Análisis automatizado

Puede “recorrer” automáticamente el código fuente y **detectar las reglas que no se cumplen.**

Ejemplos: linters, analizadores integrados en el IDE o en el pipeline de integración continua.



Una persona

Revisión de código

Revisa el código para garantizar que se utilicen los **estándares y convenciones de codificación** adecuados para construir el programa.



La realiza un desarrollador colega: alguien distinto al desarrollador que escribió el código.

Análisis dinámico



Analiza programas ejecutándolos

y generando algún tipo de seguimiento (traza) de su comportamiento.



Identifica vulnerabilidades

en un entorno de ejecución.

Estático vs. dinámico

	Estático	Dinámico
¿Ejecuta el código?	No	Sí
¿Cuándo?	Antes de ejecutar	En tiempo de ejecución
¿Qué obtiene?	Reglas incumplidas y debilidades	Trazas y vulnerabilidades en ejecución

Ejercicio: ¿análisis estático o dinámico?

EJERCICIO

Técnica	Tipo	Por qué
El IDE subraya una variable que no se usa	**Estático** (automatizado)	Lee el código sin ejecutarlo
Un compañero revisa tu pull request	**Estático** (revisión de código)	Una persona distinta al autor lee el código
Ejecutar las pruebas unitarias	**Dinámico**	El programa se ejecuta
Un profiler mide qué método tarda más	**Dinámico**	Genera una traza durante la ejecución
PMD revisa el proyecto en el pipeline de CI	**Estático** (automatizado)	Analiza el código fuente
Medir el consumo de memoria con datos reales	**Dinámico**	Observa el programa corriendo

¿Cuándo va el análisis estático? **Después de codificar y antes de las pruebas unitarias**: encuentra defectos antes de que el programa se ejecute.

02

Inspección de código



Qué es, en qué atributos de calidad se enfoca y qué defectos detecta

Inspección de código: un mismo concepto, varios nombres

Inspección de código = **Análisis de código estático** = **Análisis estático** = **Análisis de programa estático**



Sin ejecución

La inspección de código **no implica la ejecución dinámica** del software bajo prueba.



Detección temprana

Puede detectar posibles defectos **en una etapa temprana**, antes de ejecutar el programa.

Atributos de calidad que puede evaluar

Los atributos de calidad que pueden ser el foco del análisis estático son:



Fiabilidad

El software funciona correctamente de forma consistente.



Mantenibilidad

Facilidad para modificar y corregir el código.



Capacidad de prueba

Facilidad para verificar su comportamiento.



Reutilización

Componentes aprovechables en otros sistemas.



Portabilidad

Capacidad de funcionar en distintas plataformas.



Y otros...

Seguridad, eficiencia, legibilidad, etc.

¿Qué detecta la inspección de código?

Gestión inadecuada de recursos

Fugas de recursos: la memoria no se desasigna correctamente cuando ya no se utiliza.

```
FileReader fr = new FileReader(ruta);
// fr nunca se cierra -> fuga
```

Operaciones ilegales

División por cero, acceso a arreglos fuera de límites, desbordamiento o subdesbordamiento en expresiones aritméticas, etc.

```
int r = total / 0;
v[v.length] = 1;  // fuera de límites
```

Código y datos muertos

Código y datos a los que no se puede acceder o que no se utilizan.

```
return resultado;
contador++;  // inalcanzable
```

Código incompleto

Variables no inicializadas, funciones con valores de retorno no especificados, ramas incompletas, bucles sin terminación, etc.

```
while (true) { procesar(); }
// el bucle nunca termina
```

Ejercicio: inspecciona este código sin ejecutarlo

EJERCICIO

```
1  public class Reporte {
2      public double promedio(int[] notas) {
3          int suma = 0;
4          int max = 0;
5          for (int i = 0; i ⩽ notas.length; i++)
6              suma += notas[i];
7          return suma / notas.length;
8      }
9      public String leer(String ruta) throws IOException {
10         var br = new BufferedReader(new FileReader(ruta));
11         return br.readLine();
12     }
13     private void exportar() { }
14 }
```

Línea	Defecto	Categoría
4	`max` nunca se usa	Datos muertos
5	⩽ accede a `notas[length]`	Operación ilegal (fuera de límites)
7	División por cero si el arreglo está vacío; además es división entera	Operación ilegal
10	El lector nunca se cierra	Gestión inadecuada de recursos (fuga)
13	Método privado que nadie llama	Código muerto

Ninguno necesita ejecutar el programa: se ven leyendo el código. Por eso el análisis estático encuentra el defecto en su **ubicación exacta**.

03

Ventajas y limitaciones



Qué aporta la inspección de código, dónde se queda corta y cómo se compara con las pruebas

Ventajas de la inspección de código

Ubicación exacta

Puede encontrar debilidades en el código en la ubicación exacta.

Realizada por especialistas

Puede ser realizada por desarrolladores de software capacitados.

Código más comprensible

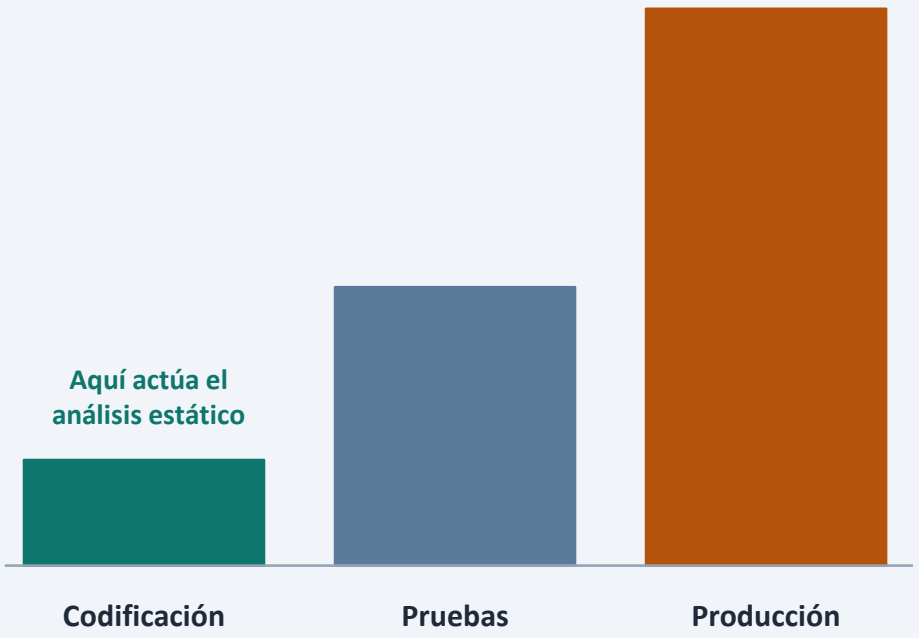
El código fuente puede ser fácilmente entendido por otros desarrolladores o futuros desarrolladores.

Menor costo de corrección

Las debilidades se detectan más temprano en el ciclo de vida del desarrollo (menos defectos en pruebas posteriores).

Detectar temprano cuesta menos

Costo relativo de corregir un defecto según la fase (ilustrativo)



Defectos que las pruebas dinámicas difícilmente detectan

La inspección detecta defectos únicos que no pueden —o apenas pueden— detectarse mediante pruebas dinámicas:



Código inalcanzable

```
throw new Error();
limpiar(); // nunca corre
```



Uso de variables

(no declaradas, no utilizadas)

```
int tmp = calcular();
// tmp no se usa
```



Funciones no llamadas

```
private void exportar()
// sin invocaciones
```

…entre otros.

Limitaciones de la inspección de código



Consume mucho tiempo

Es una tarea que requiere mucho tiempo si se realiza manualmente.



Falta de personal capacitado

No hay suficiente personal capacitado para realizar un análisis exhaustivo del código estático.



Falsos positivos y negativos

Las herramientas automatizadas producen falsos positivos y falsos negativos.



Dependencia de las reglas

Las herramientas automatizadas son tan buenas como las reglas que utilizan para escanear.

Análisis estático vs. pruebas

Criterio	Análisis estático	Pruebas
Ejecución	Se aplica sin ejecutar el código	Solo se aplica ejecutando el código
Momento	Temprano en el proceso de desarrollo	Tarde en el proceso de desarrollo
Entradas	Los resultados no dependen de las entradas	Los resultados dependen de las entradas
Generalización	Resultados generalizables a ejecuciones futuras	Resultados no generalizables a ejecuciones futuras
Costo	Menos costoso	Muy costoso
Velocidad	Proceso muy rápido	Proceso lento
Falsos positivos	Tasa muy alta	Tasa muy baja
Resultados	Usa aproximaciones	Usa resultados exactos
Corrección funcional	No sirve para verificar la corrección funcional	Sirve para verificar la corrección funcional

Fuente: adaptado de la tabla “Comparison of static analysis and testing” del material del curso.

Una crítica del análisis estático

ASPECTOS A CONSIDERAR



Precisión

Puede ser impreciso porque hace aproximaciones sobre el comportamiento en tiempo de ejecución de un programa (muchos falsos positivos).



Eficiencia

Para dar resultados más rápido usa mecanismos de aproximación o abstracción que conducen a resultados menos precisos: sacrifica precisión por eficiencia.

FORTALEZAS



Cobertura

Proporciona una alta cobertura de código para fines de análisis (se analizan rutas de ejecución válidas).



Escalabilidad

Escala bien: se puede ejecutar en muchos programas y de forma repetida.

Falsos positivos y falsos negativos

EJEMPLO

Falso positivo

La herramienta reporta un problema **que no existe**.

```
// Posible falso positivo:
String s = obtener();   // ¿puede ser null?
if (config.modoSeguro()) {
    s = "valor por defecto";
}
System.out.println(s.length()); // ¿NPE?
```

Si `obtener()` nunca devuelve null, la alerta sobra: la herramienta **aproximó** el comportamiento.

Falso negativo

Hay un problema real **que la herramienta no reporta**: por ejemplo, un descuento mal calculado. El código es «correcto» para las reglas, pero no hace lo que pide el cliente.

Por eso el análisis estático **no verifica la corrección funcional** y no reemplaza a las pruebas. Es rápido, barato y generalizable, pero tiene muchos falsos positivos.

Las herramientas son tan buenas como sus **reglas**: si ninguna regla describe el error, no lo encuentran.

04

Herramientas de inspección de código



Características clave, categorías y dos herramientas representativas: PMD y FindBugs

Características clave de las herramientas (1/2)

01



Diseñadas para la seguridad

Herramientas con un módulo de seguridad: el conocimiento de seguridad más crítico viene incorporado.

02



Admiten múltiples niveles

Las aplicaciones rara vez usan un solo lenguaje o plataforma; la herramienta debe cubrir varios lenguajes y funcionar en muchas plataformas.

03



Apoyan los procesos existentes

Debe ser fácil integrarlas con los procesos de compilación existentes y con los entornos de desarrollo integrados (IDE).

Características clave de las herramientas (2/2)

04 

Tienen sentido para múltiples partes interesadas

La herramienta debe respaldar el negocio. Distintas vistas para gerentes de lanzamiento, gerentes de desarrollo y ejecutivos pueden apoyar, por ejemplo, las decisiones de lanzamiento y el control de los costos de retrabajo.

05 

Son extensibles

Arquitectura modular que admite diversas técnicas de análisis. Además, deben permitir que los usuarios agreguen sus propias reglas.

06 

Útiles para analistas de seguridad, equipos de QA y desarrolladores

Deben apoyar no solo a los analistas, sino también a los desarrolladores que necesitan solucionar los problemas descubiertos por la herramienta.

Categorías de herramientas de inspección de código



Comprobación estática

Encontrar errores de codificación comunes.



Detección de errores

Encontrar errores (bug finding) y reportarlos.



Verificación de software

Garantizar la ausencia de errores aportando pruebas (demostraciones).



Inferencia de tipos

Especificar y comprobar las propiedades del programa.

Herramientas de inspección de código por categoría

Herramienta	Comprobación estática	Detección de errores	Verificación de software	Inferencia de tipos
Lint family	●			
JLint	●			
FindBugs	●	●		
PMD	●	●		
Coverity Prevent		●		
KlockWork K7		●		
ASTREE		●	●	
CGS			●	
Polyspace Verifier			●	
TVLA			●	
BANE				●
BANSHEE				●
CQual				●
PREfix		●		
CodeSonar		●		
ESC/Java	●		●	
ESP		●	●	

Resaltadas: PMD y FindBugs, que se detallan a continuación. Fuente: adaptado de la tabla “Summary of static analysis tools” del material del curso.

PMD

PMD es una herramienta de análisis de código estático capaz de detectar automáticamente una amplia gama de defectos potenciales y código inseguro o no optimizado.



Detección preventiva de defectos



Conjunto de reglas altamente configurable



Se integra bien con IDE como Eclipse



Se adapta al proceso de construcción gracias a su integración con Maven

EJEMPLO DE REPORTE (ILUSTRATIVO)

```
$ pmd check -d src -R quickstart

Pedido.java:12
  UnusedLocalVariable
  Variable local 'total' sin usar

Pedido.java:27
  EmptyCatchBlock
  Bloque catch vacío

Pedido.java:41
  UnusedPrivateMethod
  Método privado 'log' sin usar
```

FindBugs

PATRONES DE ERROR QUE BUSCA (EJEMPLOS)

`NP_NULL_ON_SOME_PATH`

Posible desreferencia de null

`ES_COMPARING_STRINGS_WITH_EQ`

Comparar cadenas con ==

`SBSC_USE_STRINGBUFFER_CONCATENATION`

Concatenar cadenas en un bucle (rendimiento)

`DMI_RANDOM_USED_ONLY_ONCE`

Crear un Random para un solo uso

FindBugs es otra herramienta de análisis estático para Java, similar en algunos aspectos a PMD, pero con un enfoque bastante diferente.



Solo le interesan marginalmente las buenas prácticas de codificación.



Se centra en detectar posibles errores y problemas de rendimiento, incluidos muchos errores comunes y difíciles de encontrar.



Detecta un conjunto de problemas bastante diferente al de PMD, con un grado de precisión relativamente alto.



Hoy: su sucesor activo es SpotBugs, que mantiene sus detectores.

PMD vs. FindBugs: herramientas complementarias

	PMD	FindBugs
Enfoque	Defectos potenciales, código inseguro o no optimizado y buenas prácticas	Posibles errores y problemas de rendimiento
Qué analiza	El código fuente	El bytecode compilado (.class)
Reglas	Conjunto de reglas altamente configurable	Catálogo de patrones de error (bug patterns)
Integración	IDE (p. ej., Eclipse) y Maven	IDE, Ant y Maven
Fortaleza	Prevención y estilo del código	Precisión relativamente alta en los errores que reporta



Como detectan conjuntos de problemas distintos, es habitual usarlas juntas en el mismo proyecto.

Ejercicio: ¿qué reportaría PMD y qué FindBugs?

EJERCICIO

```java
public class Pedido {
    public boolean esVip(String tipo) {
        int total = 0;                  // (1)
        try { cargar(); }
        catch (Exception e) { }         // (2)
        return tipo == "VIP";           // (3)
    }
    private void log(String m) { }      // (4)
    public String lista(List<String> xs) {
        String r = "";
        for (String x : xs) r += x;     // (5)
        return r;
    }
}
```

#	Hallazgo	Herramienta
1	Variable local sin usar	**PMD**
2	Bloque catch vacío	**PMD**
3	Comparar cadenas con ==	**FindBugs**
4	Método privado sin usar	**PMD**
5	Concatenar en un bucle (rendimiento)	**FindBugs**

PMD analiza el **código fuente** y se enfoca en buenas prácticas; FindBugs analiza el **bytecode** y busca errores y rendimiento. Detectan cosas distintas: se usan juntas.

IDEAS CLAVE

Para recordar

01

La inspección de código es análisis estático: examina el código fuente sin ejecutarlo.

02

Detecta defectos temprano y en su ubicación exacta, lo que reduce el costo de corregirlos.

03

No reemplaza a las pruebas: tiene falsos positivos y no verifica la corrección funcional.

04

Las herramientas se agrupan en comprobación estática, detección de errores, verificación e inferencia de tipos.

Una lista de verificación para revisar código

EJEMPLO

Fiabilidad

¿Hay divisiones que puedan ser por cero? ¿Índices fuera de límites? ¿Bucles que nunca terminan? ¿Variables sin inicializar?

Recursos

¿Se cierran archivos, conexiones y lectores (por ejemplo, con `try-with-resources`)?

Mantenibilidad

¿Nombres con significado? ¿Funciones pequeñas? ¿Código duplicado o muerto? ¿Sigue el estándar del equipo?

Seguridad y portabilidad

¿Se validan las entradas? ¿Hay rutas o contraseñas escritas en el código? ¿Depende de un sistema operativo?

La revisión la hace un colega distinto al autor. Sus limitaciones: consume tiempo y no siempre hay personal capacitado. Por eso se combina con herramientas automáticas en el IDE y en la CI.

Tema 6

Pruebas de software: definiciones básicas

Casos de prueba · Pruebas funcionales y estructurales



INGENIERÍA DE SOFTWARE II

Pruebas de Software

Definiciones básicas

Casos de prueba · Pruebas funcionales y estructurales

CONTENIDO DEL MÓDULO

Agenda

01 

¿Qué es la prueba?

Concepto y propósito de las pruebas de software

02 

Definiciones básicas

Error, defecto, falla, incidente y caso de prueba

03 

Casos de prueba

Estructura, comportamiento y perspectivas

04 

Pruebas funcionales y estructurales

Caja negra vs. caja blanca

DEFINICIONES BÁSICAS

¿Qué son las pruebas?





Un proceso de búsqueda

Es el proceso de ejecutar un programa con la intención de encontrar errores.



Actividades planificadas

Actividades planificadas que se llevan a cabo sistemáticamente para descubrir errores cometidos durante el diseño y construcción de un software.

Definiciones básicas

Error *(equivocación)*

Las personas cometen errores. Tienden a propagarse: un error en los requisitos puede magnificarse durante el diseño y amplificarse aún más durante la codificación.

Defecto *(fallo)*

Un defecto es el resultado o representación de un error. Puede ser un diagrama UML o código fuente (es estático).

Faltas de comisión

Introducimos algo incorrecto en una representación.

Faltas de omisión

No incluimos información (falta).

Fuente: Junta Internacional de Calificación de Pruebas de Software (ISTQB) e IEEE

Definiciones básicas





Falla

Ocurre cuando se ejecuta el código correspondiente a un evento dinámico, es decir, cuando el sistema no puede realizar parte de su función.



Incidente

Es el síntoma asociado a una falla que alerta al usuario sobre la ocurrencia de esta.



Prueba

El acto de ejercitar el software con casos de prueba para encontrar fallas o demostrar una ejecución correcta.



Caso de prueba

Tiene una identidad y está asociado con el comportamiento del programa. También tiene un conjunto de entradas y salidas esperadas.

De la equivocación al incidente: una sola historia

EJEMPLO

Requisito: **«10 % de descuento en compras desde $100»**.

1 · Error

El analista entiende «más de $100» y lo escribe así en el diseño. Una **persona** se equivoca.

2 · Defecto

Queda en el código: `if (total > 100)`. Es **estático**: está ahí aunque nadie lo ejecute.

3 · Falla

Un cliente compra exactamente **$100**, el código se ejecuta y no aplica el descuento.

4 · Incidente

El cliente ve el total equivocado en su recibo y reclama: el **síntoma** que alerta de la falla.

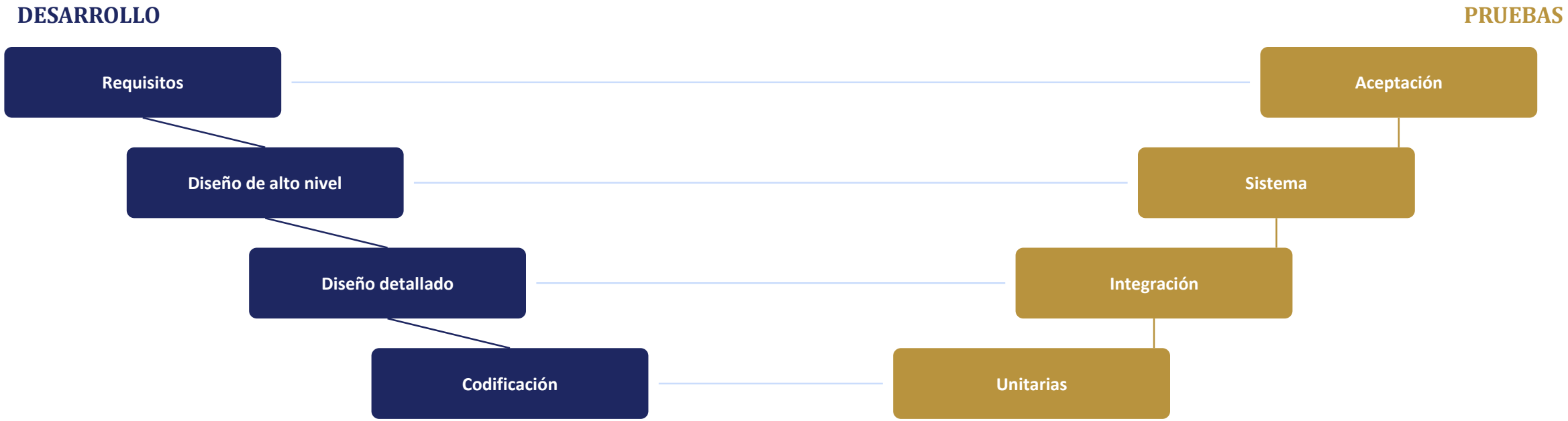
Es una falta de **comisión**: se introdujo algo incorrecto. Si el requisito del descuento nunca se hubiera programado, sería de **omisión**.

El caso de prueba que la encuentra: entrada **total = $100**, salida esperada **$90**. Por eso importan los límites.

El modelo del ciclo de vida para pruebas

Diseño de alto nivel: plataforma tecnológica, implementación física, selección de componentes estructurales, comunicación del sistema, concurrencia, etc.

Diseño detallado: los componentes se refinan en clases, se realizan interfaces, se identifican y aplican patrones de diseño, etc.



Terminología IEEE

Niveles de prueba



Pruebas unitarias

Se prueban unidades/componentes individuales de un software.



Pruebas de integración

Las unidades individuales se combinan y se prueban como un grupo.



Prueba del sistema

Se prueba un software completo e integrado para verificar que cumple con los requisitos especificados.



Prueba de aceptación

Se prueba la cualidad de aceptable de un sistema.

Los cuatro niveles de prueba en una app bancaria

EJEMPLO

Unitarias

Se prueba sola la función `calcularInteres(saldo, tasa)`: con $1000 al 5 % anual debe dar $50.

Integración

Se prueba que el módulo de transferencias, junto con la base de datos, descuenta de una cuenta y acredita en la otra.

Sistema

Se prueba la app completa contra los requisitos: login, consultas, transferencias, rendimiento con 1000 usuarios.

Aceptación

El banco y algunos clientes la usan para decidir si es **aceptable** antes de publicarla.

En el modelo en V, cada nivel de prueba verifica una etapa del desarrollo: unitarias ↔ codificación, integración ↔ diseño detallado, sistema ↔ diseño de alto nivel y aceptación ↔ requisitos.

PARTE 01



Casos de Prueba

Estructura, comportamiento y perspectivas de análisis

Casos de prueba

La esencia de las pruebas de software es determinar un conjunto de casos de prueba para el elemento que se va a probar. Un caso de prueba completo contendrá:

- **Un identificador**
- **Una breve declaración de propósito (p. ej., una regla comercial)**
- **Descripción de condiciones previas y posteriores esperadas**
- **Las entradas**
- **Los resultados esperados**
- **Un historial de ejecuciones**

Un caso de prueba completo

EJEMPLO

Elemento	Ejemplo
Identificador	CP-07
Propósito	Verificar la regla de negocio: 10 % de descuento desde $100
Precondición	Usuario autenticado, carrito vacío
Entradas	Agregar productos por un total de $100.00 y pagar
Resultado esperado	Total a pagar: $90.00
Postcondición	Pedido registrado con el descuento aplicado
Historial de ejecuciones	12/10 · falló (cobró $100) · 14/10 · pasó

Un caso de prueba tiene **identidad** y un conjunto de **entradas** y **salidas esperadas**.

Un buen caso de prueba...

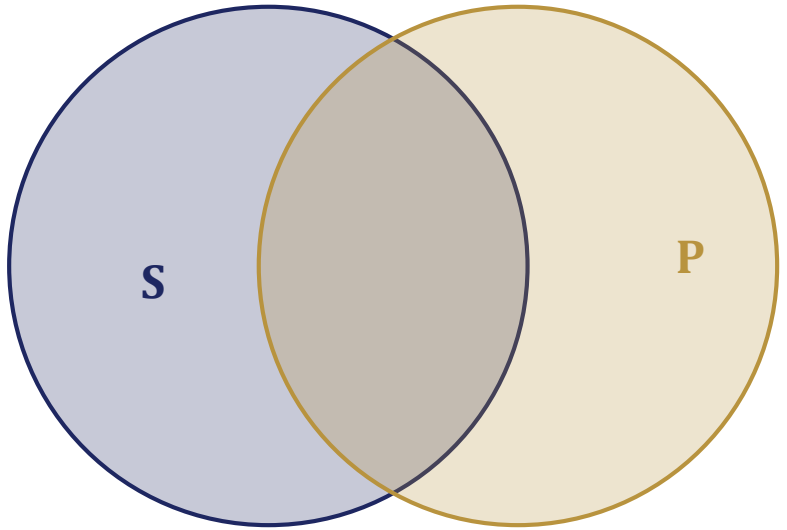
- tiene alta probabilidad de encontrar un error ($100 justo en el límite),
- no es redundante con otros,
- es el mejor de su clase,
- ni muy simple ni muy complejo.

Comportamiento de programas

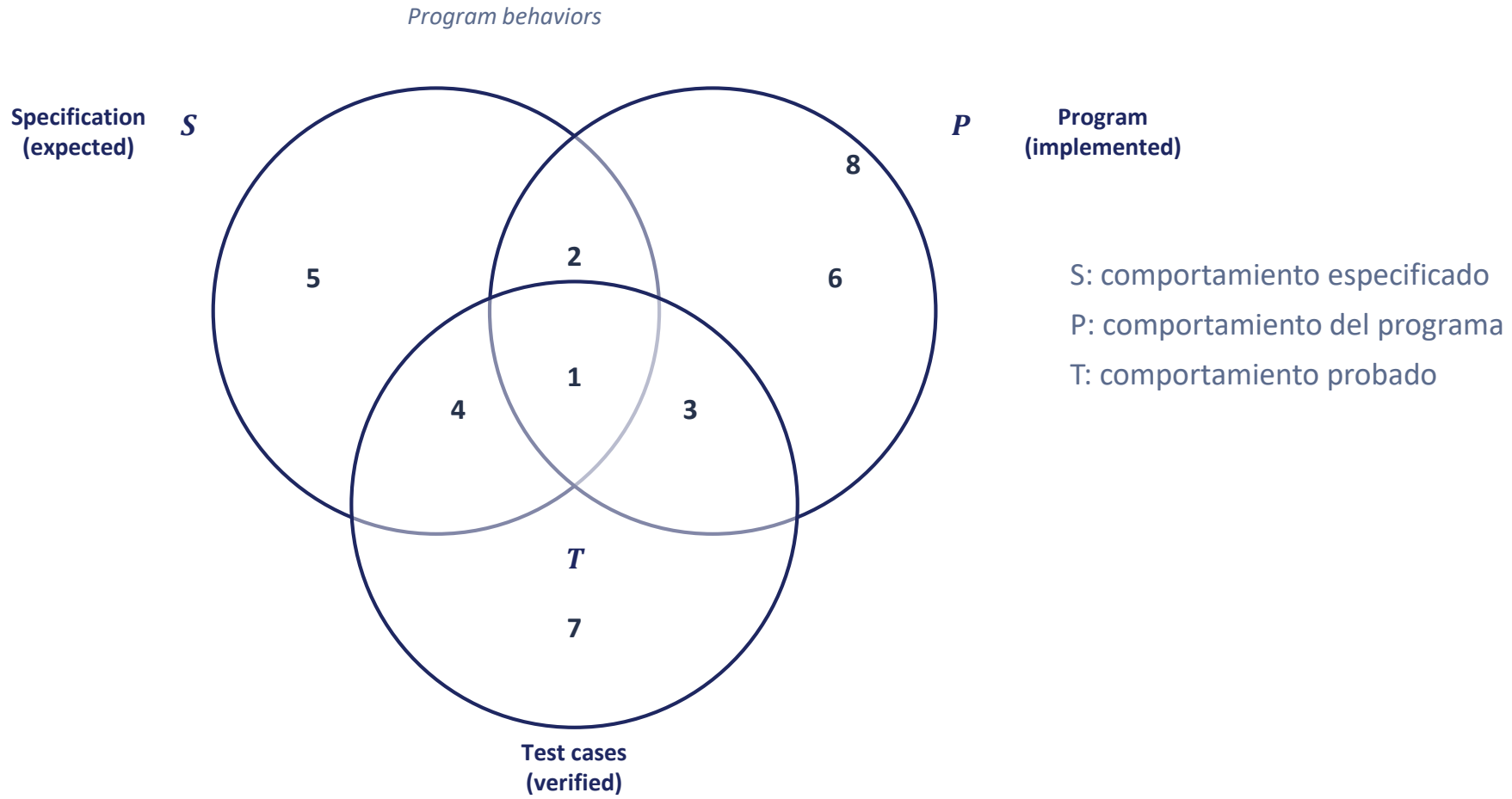
Especificaciones (S): comportamiento esperado del programa (historias de usuario y criterios de aceptación).

Programa (P): comportamiento observado.

S sin P	Comportamientos no especificados
P sin S	Especificaciones no implementadas
S ∩ P	Comportamientos especificados e implementados



Casos más importantes: perspectivas desde un diagrama de Venn

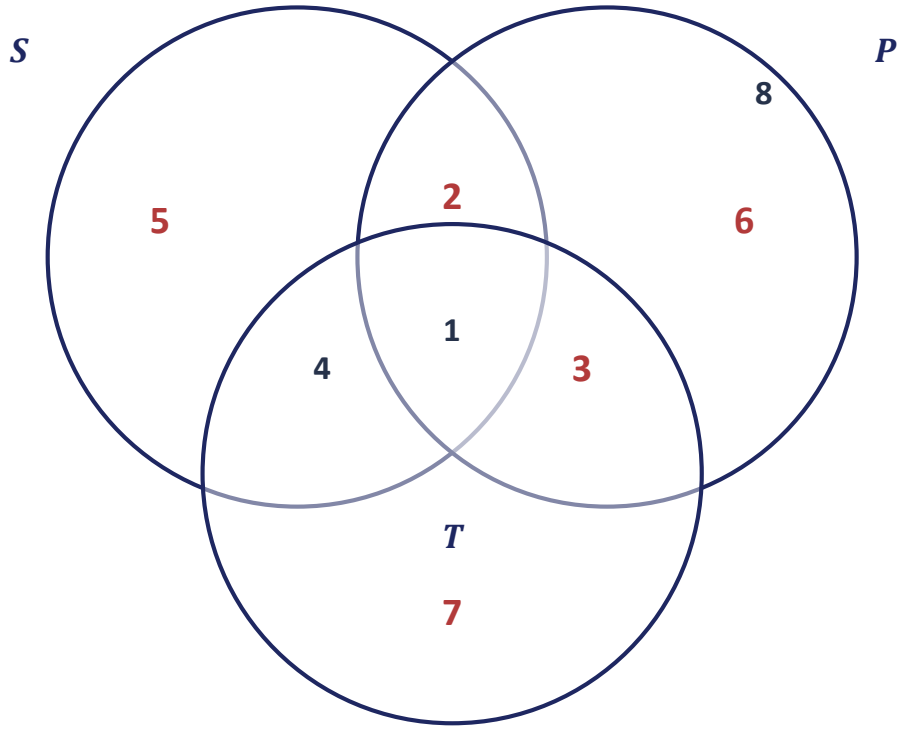


S: comportamiento especificado

P: comportamiento del programa

T: comportamiento probado

Lectura del diagrama de Venn



2 y 5	Requerimientos no probados
1 y 4	Requerimientos con sus casos de prueba
3	Código probado que no fue solicitado
6	Código no solicitado ni probado
1	Requerimientos implementados y probados
7	Casos de prueba no relacionados a los requisitos

Ejercicio: ubica cada comportamiento en el diagrama de Venn

EJERCICIO

Sistema de inicio de sesión. **S** = especificado · **P** = programado · **T** = probado.

Comportamiento	S	P	T	Región
Iniciar sesión con la contraseña correcta	✓	✓	✓	**1** · implementado y probado
Bloquear la cuenta tras 3 intentos fallidos	✓	✓		**2** · requisito no probado
Casilla «Recordarme» que nadie pidió		✓	✓	**3** · código probado no solicitado
Recuperar contraseña: hay prueba, pero no se programó	✓		✓	**4** · requisito con prueba, sin implementar
Cambiar idioma: pedido, sin programar ni probar	✓			**5** · requisito no probado
Modo depuración oculto en el código		✓		**6** · ni solicitado ni probado
Prueba de «exportar a PDF», que no existe en S ni en P			✓	**7** · prueba sin requisito

Lo ideal es que la región 1 sea lo más grande posible. La región 6 es peligrosa: código que nadie pidió y nadie probó.

PARTE 02



Taxonomía de fallas

Solo para conocimiento

Taxonomía de fallas

Basado parcialmente en la Clasificación estándar IEEE para anomalías de software (IEEE, 1993).

Fallas de entrada/salida
Entrada correcta no aceptada
Entrada incorrecta aceptada
Descripción incorrecta o faltante
Parámetros incorrectos o faltantes
Formato de salida incorrecto
Resultado incorrecto
Resultado correcto en momento incorrecto
Resultado incompleto o faltante
Resultado espurio
Errores cosméticos o de ortografía

Fallas de lógica
Caso(s) faltante(s)
Caso(s) duplicado(s)
Condición extrema descuidada
Mala interpretación
Condición faltante
Condición(es) extrañas
Prueba de variable incorrecta
Iteración de bucle incorrecta
Operador incorrecto (p. ej., < en vez de ≤)

Etc, etc, etc.

Ejercicio: nombra la falla según la taxonomía

EJERCICIO

Situación	Tipo de falla
El formulario rechaza un correo válido con «+»: `ana+uees@gmail.com`	Entrada correcta no aceptada
Se acepta una fecha de nacimiento en el año 3025	Entrada incorrecta aceptada
El bucle procesa 9 de los 10 estudiantes	Iteración de bucle incorrecta
Se usó < en lugar de ⩽ en «hasta 18 años»	Operador incorrecto
El reporte muestra 0.30000000000000004 en vez de 0.30	Formato de salida incorrecto
No se consideró qué pasa si el carrito está vacío	Caso faltante / condición extrema descuidada
El botón dice «Envíar»	Error cosmético o de ortografía

PARTE 03

Características de un buen caso de prueba

El problema fundamental de las pruebas de software

Problema: ningún sistema puede probarse completamente (no podemos probarlo todo).

Soluciones

- Priorización de pruebas basada en el análisis de riesgos.
- Lo que pruebas es más importante que cuánto pruebas.
- Priorizar (probabilidad e impacto) para que los errores más importantes se encuentren primero.
- Las pruebas deben diseñarse cuidadosamente (metodología, técnicas).

Características de un buen caso de prueba

Alta probabilidad de encontrar un error

Desarrolla una imagen mental de cómo podría fallar el software.

No redundante

El tiempo y los recursos son limitados; cada prueba debería tener un propósito distinto.

El mejor de su clase

Prioriza las pruebas con mayor probabilidad de descubrir toda una clase de errores.

Ni muy simple ni muy complejo

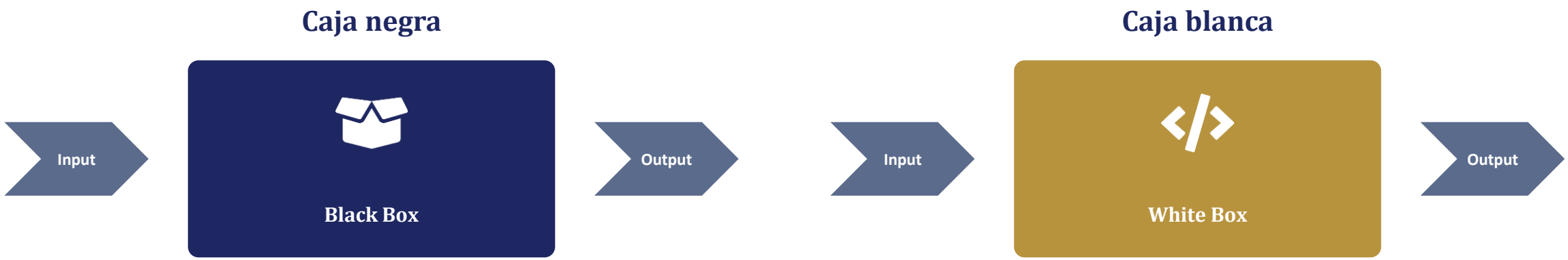
Cada prueba debe ejecutarse por separado, con un propósito claro.

PARTE 04



Enfoques para la identificación de casos de prueba

Dos enfoques: caja negra y caja blanca



Cualquier programa se puede considerar una función: un conjunto de entradas produce un conjunto de salidas.

Caja negra

Conocer la función específica que un producto ha sido diseñado para realizar.



Todo programa se considera una función; la salida depende de las entradas. ¡No es necesario conocer el código!

Caja blanca

Conocer el funcionamiento interno de un producto.



El código es conocido y se utiliza para identificar casos de prueba.

Enfoques para la identificación de casos de prueba

Pruebas funcionales

También llamadas pruebas basadas en especificaciones o de caja negra. Se realizan en la interfaz del software y se centran en los requisitos funcionales.

Pruebas estructurales

También llamadas pruebas basadas en código, de caja blanca o de caja de vidrio. Se basan en el funcionamiento interno de la aplicación y prueban su estructura interna.

PARTE 05



Pruebas funcionales (caja negra)

Pruebas funcionales

Generalmente se utiliza en ingeniería (automóviles, teléfonos, drones, microondas). Solo necesita especificaciones de software.

Pretende encontrar errores en las siguientes categorías:

- **Funcionalidades incorrectas o faltantes**
- **Errores de interfaz**
- **Errores en el acceso**
- **Errores de comportamiento o rendimiento**
- **Errores de inicialización y finalización**

Pruebas funcionales

Ventajas

- Independencia de cómo se implementa el software: si la implementación cambia, los casos de prueba siguen siendo útiles.
- El desarrollo de casos de prueba puede ocurrir en paralelo con la implementación, reduciendo el intervalo general de desarrollo.

Desventajas

Las pruebas funcionales a menudo sufren redundancias significativas entre los casos de prueba, o lagunas de software no probado.

Técnicas de pruebas funcionales

Valor límite

Se prueban los extremos de los rangos de entrada válidos.

Clase de equivalencia

Se agrupan las entradas en clases con comportamiento equivalente.

Tabla de decisiones

Se combinan condiciones y acciones en una tabla estructurada.

PARTE 06



Pruebas estructurales (caja blanca)

Pruebas estructurales



Basado en el código fuente / pseudocódigo del programa o del sistema, y **NO** en su especificación.

Pruebas estructurales

Utilizando métodos de prueba de caja blanca, puede derivar casos de prueba que:

Garanticen rutas independientes

Que todas las rutas independientes dentro de un módulo se hayan ejercitado al menos una vez.

Ejerciten decisiones lógicas

En sus lados verdadero y falso.

Ejecuten todos los bucles

En sus límites y dentro de sus límites operativos.

Ejerciten estructuras de datos internas

Para garantizar su validez.

Técnicas de pruebas estructurales





Prueba de ruta

Se diseñan casos de prueba para ejercitar rutas independientes del código.



Pruebas de flujo de datos

Se analiza el ciclo de vida de las variables (definición y uso) para diseñar casos de prueba.

Técnicas

Pruebas funcionales

- Valor límite
- Clase de equivalencia
- Tabla de decisiones

Pruebas estructurales

- Prueba de ruta
- Pruebas de flujo de datos

Caja negra y caja blanca sobre la misma función

EJEMPLO

Especificación: *«notas de 0 a 10; desde 7 se aprueba; fuera de rango es un error».*

Caja negra (funcional)

Solo con la especificación, sin mirar el código:

Entrada	Esperado
8	Aprobado
5	Reprobado
11	Error

Los casos se pueden escribir antes de programar.

Caja blanca (estructural): mirando el código

```
String clasificar(int nota) {
    if (nota < 0 || nota > 10)      // decisión 1
        return "Error";
    if (nota ⩾ 7)                   // decisión 2
        return "Aprobado";
    return "Reprobado";
}
```

Para ejercitar cada decisión en su lado **verdadero y falso**: nota = **-1** (decisión 1 verdadera), **7** (decisión 2 verdadera) y **6** (decisión 2 falsa).

Ningún enfoque basta solo: la caja negra no ve código extra; la caja blanca no ve requisitos que nunca se programaron.

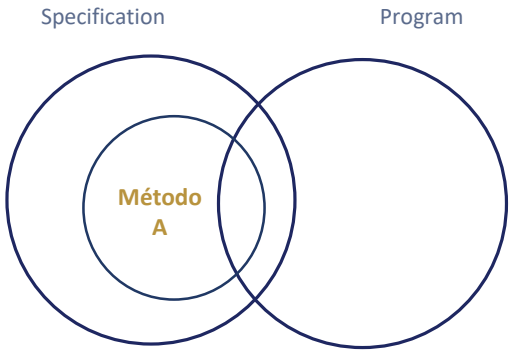
PARTE 07

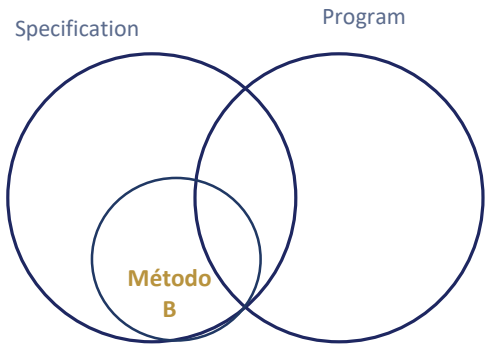
Enfoque funcional versus enfoque estructural

Enfoque funcional versus enfoque estructural

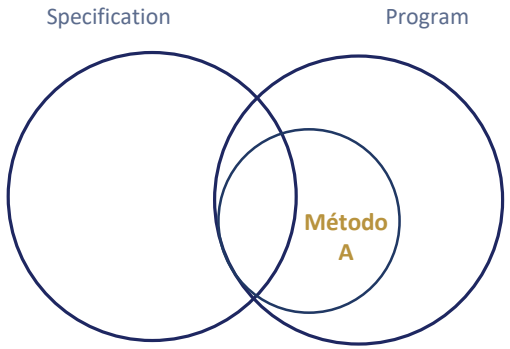
Ningún enfoque por sí solo es suficiente.

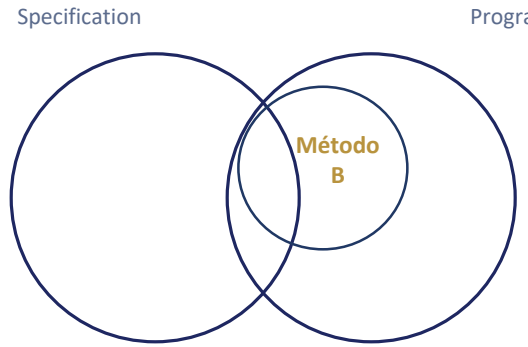
Con técnicas funcionales





Con técnicas estructurales





Distintas técnicas cubren distintas regiones del comportamiento del programa: ninguna, por sí sola, garantiza cobertura completa.



Se necesitan ambos enfoques



Una combinación juiciosa de enfoques proporcionará la confianza de las pruebas basadas en especificaciones y la medición de las pruebas basadas en código.

Tema 7

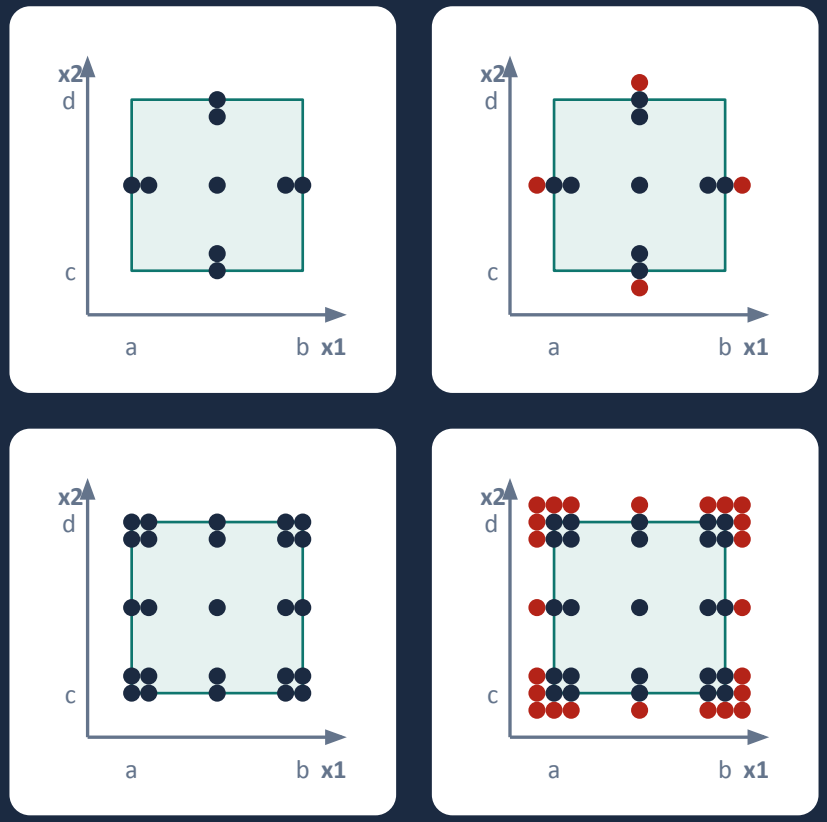
Análisis del valor límite

Casos de prueba funcionales a partir de los límites del dominio de entrada

INGENIERÍA DE SOFTWARE II

Análisis del valor límite

Diseño de casos de prueba funcionales a partir de los límites del dominio de entrada



CONTENIDO DE LA SESIÓN

Agenda

01



Verificación y validación

02



Pruebas funcionales: entradas y salidas

03



Tipos de entrada: ROV, GV y SNM

04



Análisis del valor límite

SECCIÓN 01

Verificación y Validación

01

Dónde encajan las pruebas dentro del aseguramiento de la calidad

Verificación y Validación



Verificación

Proceso de comprobar que el software cumple con los requisitos funcionales y no funcionales establecidos.



Validación

Garantiza que el software cumpla con las expectativas del cliente. Va más allá de la conformidad con la especificación: demuestra que el producto es lo que el cliente necesita.

Dos preguntas que lo resumen

VERIFICACIÓN

"¿Estamos construyendo el producto **correctamente**?"

VALIDACIÓN

"¿Estamos construyendo el producto **correcto**?"

— Barry Boehm (1979)



Las pruebas son parte de un proceso más amplio de verificación y validación de software (V&V).

Ejercicio: ¿verificación o validación?

EJERCICIO

Actividad	Respuesta
Comprobar que el IVA se calcula al 15 %, como dice la especificación	**Verificación**
Usuarios reales prueban la app y dicen que el proceso de pago es confuso	**Validación**
Revisar que la pantalla de login cumple los requisitos de seguridad	**Verificación**
El cliente confirma que el sistema resuelve el problema de su negocio	**Validación**
Ejecutar las pruebas unitarias contra el diseño	**Verificación**

Verificación: ¿estamos construyendo el producto **correctamente**? (contra la especificación)

Validación: ¿estamos construyendo el producto **correcto**? (contra lo que el cliente necesita)

El problema fundamental de las pruebas



Tiempo

Los proyectos tienen plazos limitados.



Dinero

Cada prueba tiene un costo de diseño y ejecución.

No podemos probarlo todo

El número de combinaciones de entrada crece de forma exponencial.



Necesitamos una metodología de pruebas inteligente

Las pruebas deben diseñarse cuidadosamente para encontrar el mayor número de errores con el menor número de casos.

SECCIÓN 02

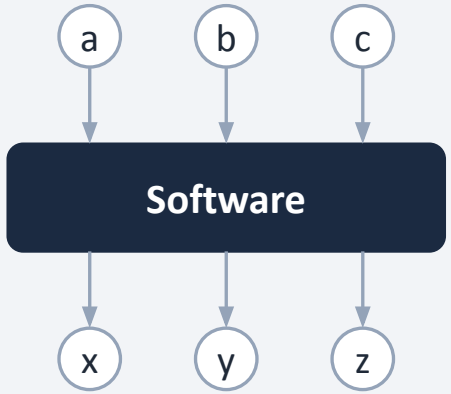
Pruebas funcionales: entradas y salidas

02

Cómo identificar qué entradas afectan a cada salida

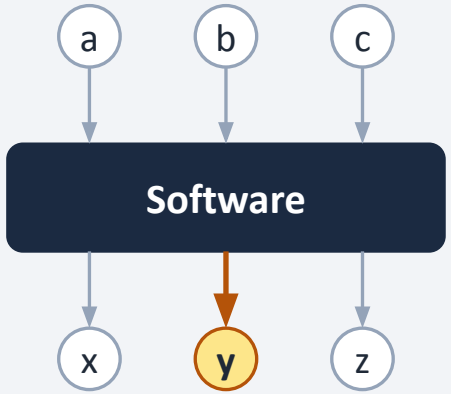
De las salidas a las entradas

1 Partir de resultados críticos



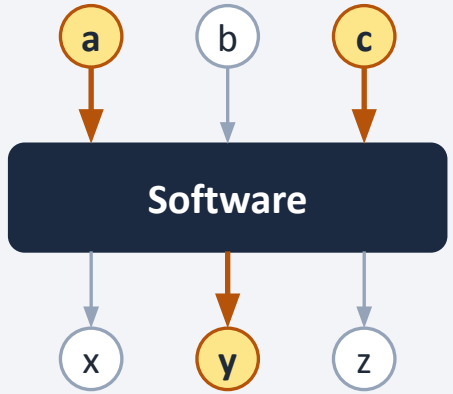
Priorizar las salidas según el análisis de riesgos.

2 Seleccionar una salida



Elegir la salida que se va a probar (por ejemplo, y).

3 Encontrar sus dependencias



Identificar las entradas de las que depende esa salida.

No necesitas entender CÓMO una salida depende de sus entradas, sólo CUÁLES son esas entradas.

Dependencias: ¿cómo las encontramos?



Los usuarios

Son quienes mejor conocen las dependencias entre entradas y salidas.



Cuidado con los usuarios

A menudo saben DEMASIADO: hay que ser selectivo. Querrán explicar CÓMO dependen los resultados de las entradas; no necesitas saberlo.



Otras fuentes

Especificación de requisitos, documentación del sistema y el presupuesto del proyecto.

SECCIÓN 03

Tipos de entrada: ROV, GV y SNM

03

Clasificar las entradas determina qué tipo de prueba aplicar

Tres tipos de entrada

ROV

Variable de sólo resultados

Influye en el valor de la salida, pero no en el camino que sigue el programa.

CÓMO PROBAR

Pruebas de valor único + valores límite + fuera de rango

GV

Variable de puerta

Decide qué ruta o cálculo se ejecuta (aparece en condiciones).

CÓMO PROBAR

Cubrir todas las rutas (prueba de rutas)

SNM

Variable que no debería importar

No debería afectar a la salida que se está probando.

CÓMO PROBAR

Sin pruebas; sólo una verificación preliminar

Dependencias y entradas: caso 1

```
Total = amount * tax_rate
```



¿Cuál es la dependencia de la salida Total con respecto a la entrada amount (cantidad)?

`amount`

ROV Variable de sólo resultados

Dependencias y entradas: caso 2

```
if (amount <= 1800)
    Total = amount
else if (amount > 1800 AND amount < 15000)
    Total = amount * tax_rate[1]
else
    Total = amount * tax_rate[2]
```



Estamos probando la salida Total. ¿Qué tipo de entrada es cada una?

`amount (cantidad)`

GV Variable de puerta: decide la rama

`tax_rate (tasa de impuesto)`

ROV Variable de sólo resultados

Dependencias y entradas: caso 3

```
Dialled_area_code = <código de área ingresado>
Phone_number = <número destino ingresado>
User_area_code = <obtenido automáticamente>
Full_destination = concatenate(Dialled_area_code,
Phone_number)
Call_cost_rate = ComputeRate(User_area_code,
Dialled_area_code)
```



¿Qué tipo de entrada (SNM, ROV o GV) es Dialled_area_code?

`Dialled_area_code`

ROV Variable de sólo resultados

¿Cómo sabemos que identificamos bien las dependencias?

1. Para cada salida, seleccione todas las entradas que no deberían importar (SNM).
2. Fije el resto de variables en un mismo valor (aleatorio) dentro de su rango válido.
3. Ejecute el código una vez y guarde la respuesta.
4. Ejecute pruebas que varíen aleatoriamente las SNM dentro de sus rangos válidos.
5. Compare los resultados: si las SNM no importan, la salida siempre debe ser la misma.
6. Registre las variables que cambian las salidas y verifique si deberían o no importar.

Ejercicio: ¿ROV, GV o SNM?

EJERCICIO

Salida que probamos: el costo de envío

```
double costoEnvio(double peso, String destino, int cantidad, String colorCaja) {
    double tarifa;
    if (destino.equals("internacional")) tarifa = 25;
    else if (peso ⩽ 1.0)                 tarifa = 3;
    else                                 tarifa = 3 + (peso - 1.0) * 2;
    return tarifa * cantidad;
}
```

Entrada	Tipo	Cómo probarla
`destino`	**GV**	Cubrir las rutas: nacional e internacional
`peso`	**GV**	Decide la rama (⩽ 1 kg o más): probar ambas rutas
`cantidad`	**ROV**	Valor límite y fuera de rango
`colorCaja`	**SNM**	Solo verificación preliminar

Truco: si la variable aparece en una **condición** (un if), es de puerta. Si solo entra en el cálculo, es de sólo resultados. Si no aparece, no debería importar: para la SNM se fija todo lo demás, se varía el color y la salida debe ser idéntica.

PANORAMA

Técnicas de pruebas funcionales



CAJA NEGRA

Pruebas funcionales

Valor límite TEMA DE HOY

Clases de equivalencia

Tablas de decisión



CAJA BLANCA

Pruebas estructurales

Pruebas de rutas

Pruebas de flujo de datos

SECCIÓN 04

Prueba de valor límite

04

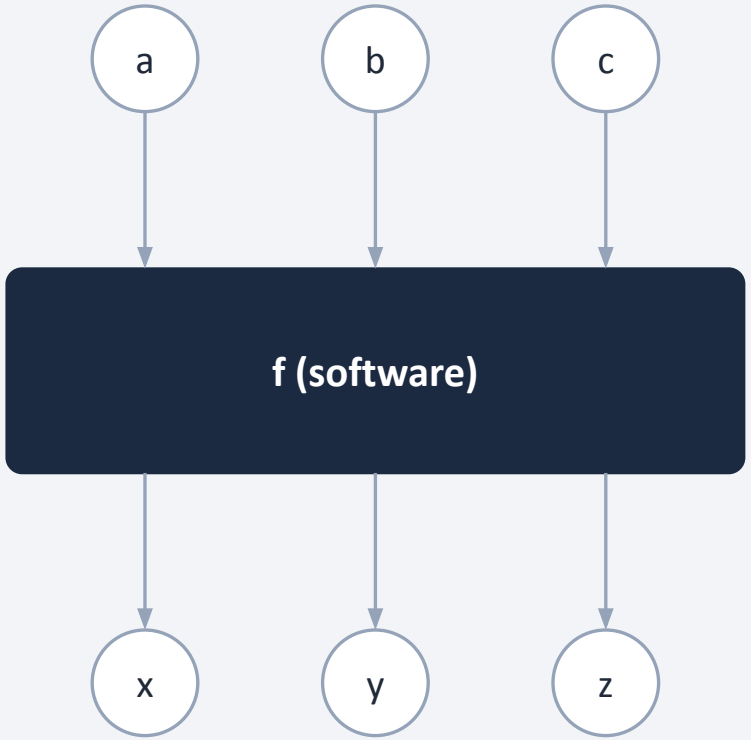
Los errores se concentran en los extremos del dominio de entrada

El software como una función

Cualquier software que se vaya a probar puede considerarse como una función que asocia sus entradas con sus salidas.

$$f : X \rightarrow Y$$

X = dominio (entradas) · Y = rango (salidas)



Ejemplo: clasificación de la presión arterial

DOMINIO (X)

Entradas

⟨x1 = sistólica, x2 = diastólica⟩



FUNCIÓN

f

X ⟶ Y



RANGO (Y)

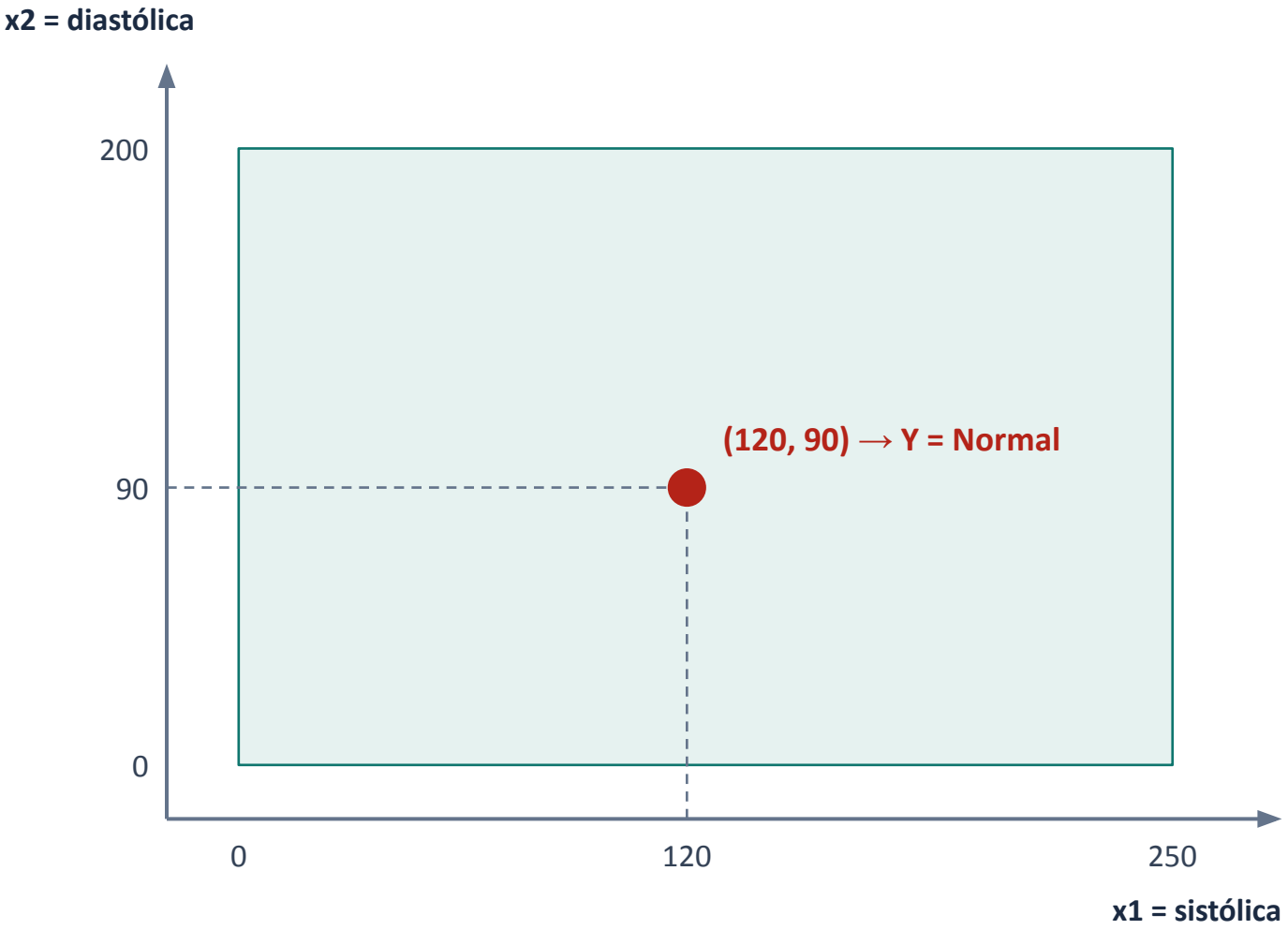
Salidas

{ Baja, Normal, Alta }



Ejemplo: con x1 = 120 y x2 = 90 la función devuelve Y = Normal.

Dominio de entrada de dos variables



Dominio de entrada

- x1 (sistólica): de 0 a 250
- x2 (diastólica): de 0 a 200
- Cada punto del rectángulo es una combinación válida de entradas.
- Los límites (0, 200, 250) son donde se concentran los errores.

Tres categorías de valores de prueba



1. **Dentro del rango**
   Valores típicos o nominales.

2. **En los límites del rango**
   Los extremos: mínimo y máximo.

3. **Fuera del rango**
   Valores no válidos (“ilegales”).

Adaptado de Eric J. Braude, Ingeniería de software: una perspectiva orientada a objetos (Wiley, 2001).

¿Por qué el análisis del valor límite?



Se producen más errores en los límites del dominio de entrada que en el “centro”.

Por esta razón se desarrolló el BVA como técnica de prueba.

Cuatro variantes

1. **Normal** Valores válidos · una variable extrema a la vez
2. **Robusta** Agrega valores no válidos (fuera de rango)
3. **Peor caso** Varias variables extremas a la vez
4. **Robusta en el peor caso** Valores no válidos + varias variables extremas

Prueba del valor límite normal

Los errores tienden a ocurrir cerca de los valores extremos de una variable de entrada.

Por ejemplo, una condición de bucle que prueba < cuando debería probar ≤:

```
for (i = 0; i < n; i++)    // ¿o i <= n?
```



Un error de “uno más / uno menos” sólo se detecta con valores justo en el límite.

Valores que se utilizan

min	Como mínimo
min+	Justo por encima del mínimo
nom	Un valor nominal
max−	Justo por debajo del máximo
max	En su máximo

¿Por qué los errores viven en los límites?

EJEMPLO

```
// Requisito: pueden votar las personas de 18 años o más
boolean puedeVotar(int edad) {
    return edad > 18;      // defecto: debía ser ⩾
}
```

Edad probada	Esperado	Obtenido	
10	false	false	**pasa**
30	true	true	**pasa**
18	**true**	**false**	**FALLA**

Los valores «del centro» (10, 30) nunca revelan el defecto. Solo el valor **justo en el límite** (18) lo encuentra.

Es el clásico error de «uno más / uno menos»: `>` en vez de `⩾`, o `i < n` en vez de `i ⩽ n`.

Con valor límite normal (edad de 18 a 120) se prueban: **18, 19, 69, 119, 120**. El 18 está incluido por diseño.

Normal con una variable

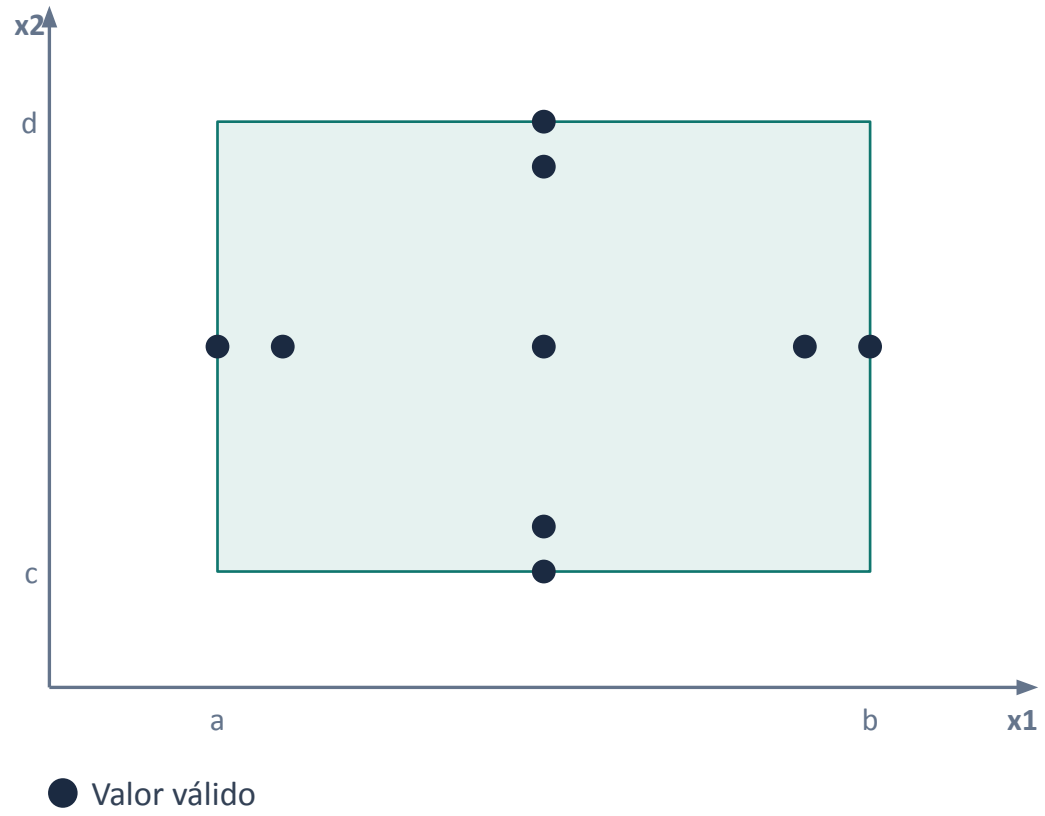


Se prueba cada valor característico de la variable: 5 casos de prueba.

4n + 1 = 5 pruebas

VARIANTE 1 · n = 2

Normal con dos variables



4n + 1 = 9 pruebas

- Una variable toma sus valores extremos mientras la otra permanece en su valor nominal.
- El punto central (nom, nom) se comparte entre ambas variables.

Normal con n variables

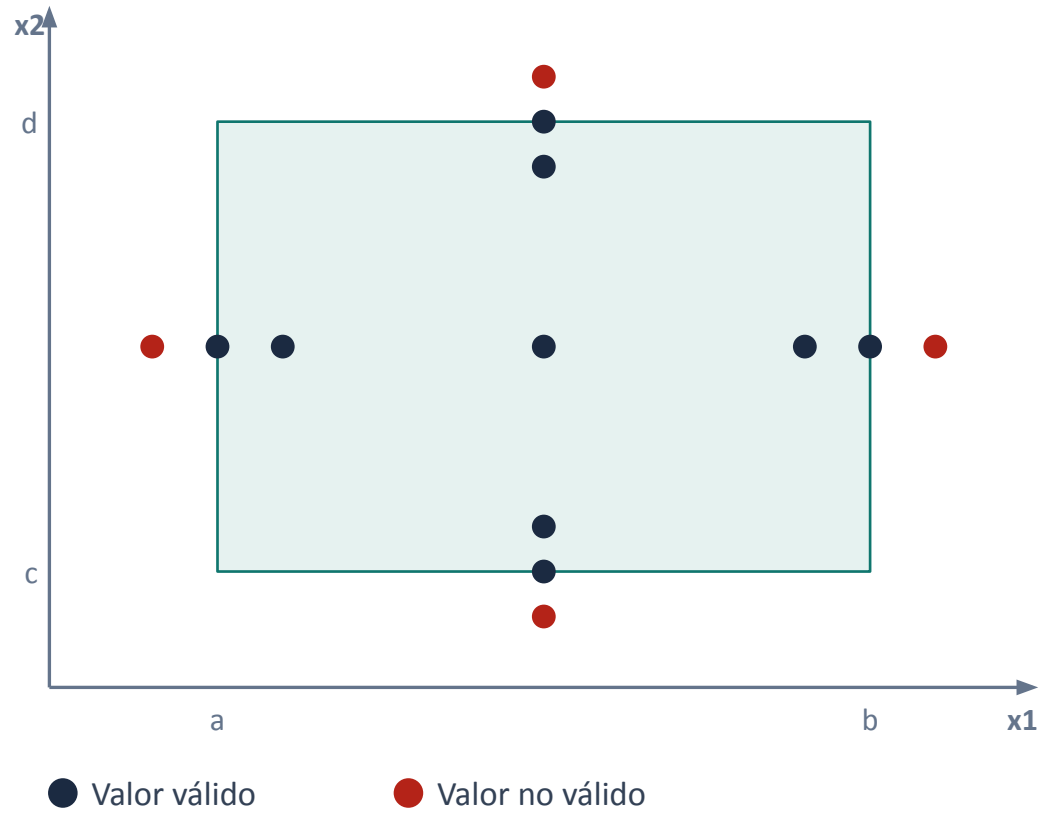
Cada variable aporta 4 valores (min, min+, max−, max) mientras las demás permanecen en su valor nominal; se suma 1 caso con todas las variables en su valor nominal.

Variables (n)	Fórmula	Casos de prueba
1	4(1) + 1	5
2	4(2) + 1	9
3	4(3) + 1	13
4	4(4) + 1	17

Supuesto de “falla única”: los fallos rara vez son el resultado de la aparición simultánea de dos o más fallas.

VARIANTE 2

Prueba robusta de valor límite



6n + 1 = 13 pruebas (n = 2)

- Es una extensión de la prueba de valor límite normal.
- A los cinco valores se agrega uno ligeramente mayor que el máximo (max+) y uno ligeramente menor que el mínimo (min−).
- Permite comprobar cómo el sistema maneja entradas no válidas.

Prueba de valor límite en el peor caso



Varias variables extremas a la vez

Se interesa por lo que sucede cuando más de una variable tiene un valor extremo simultáneamente.



¿Cuándo usarla?

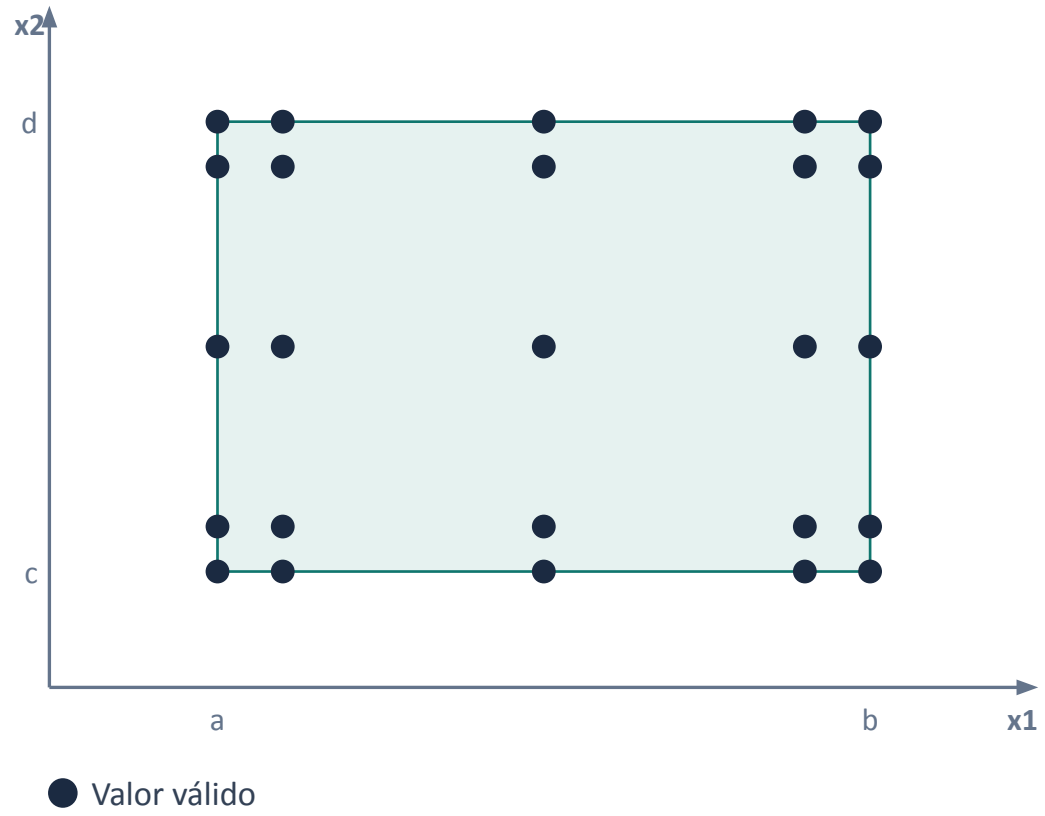
Cuando las variables tienen interacciones complejas y el impacto de un fallo es grande.



Rechaza el supuesto de “falla única” del análisis normal: combina todos los valores de todas las variables (producto cartesiano).

VARIANTE 3

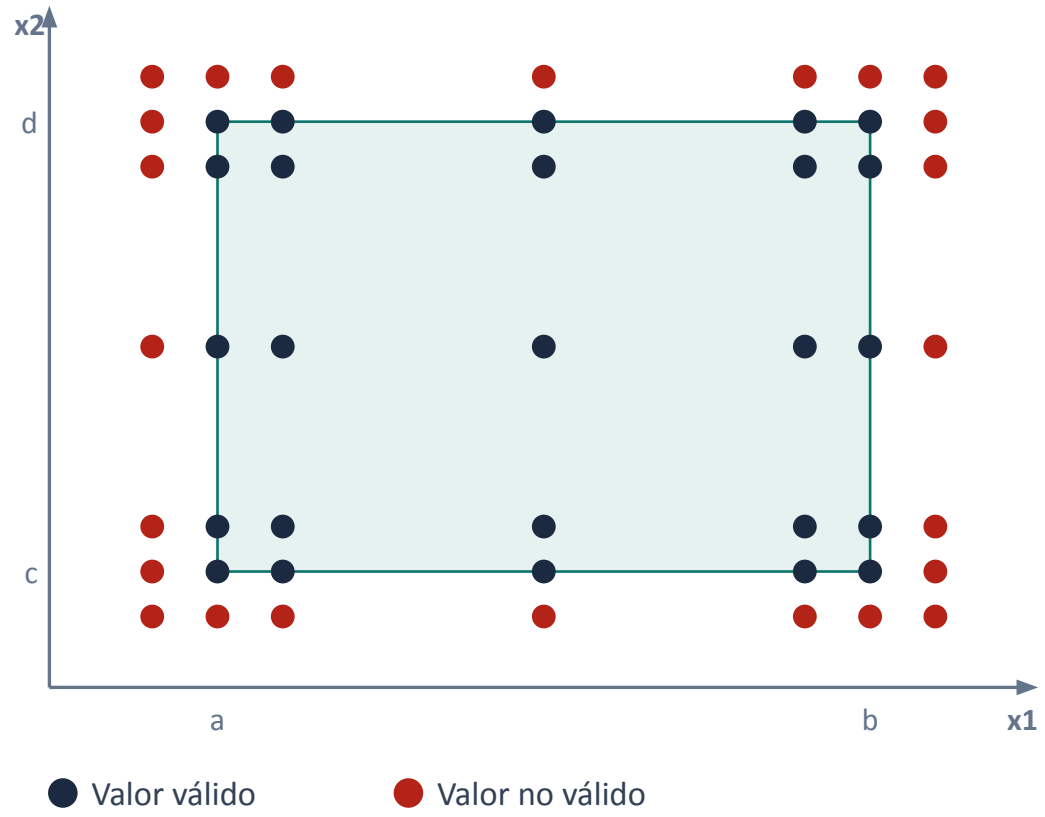
Peor caso con dos variables



5^n = 25 pruebas (n = 2)

- Se combinan los cinco valores de cada variable con los cinco de la otra.
- El número de pruebas crece de forma exponencial: con 3 variables ya son 125.

Robusta en el peor caso

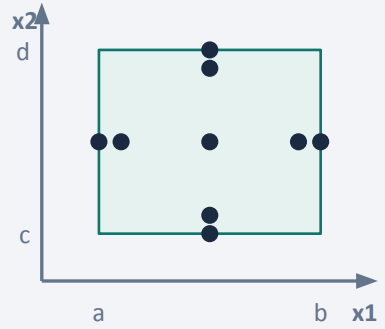
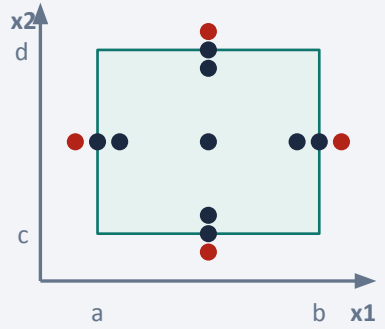
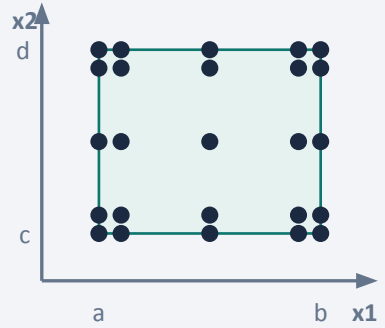
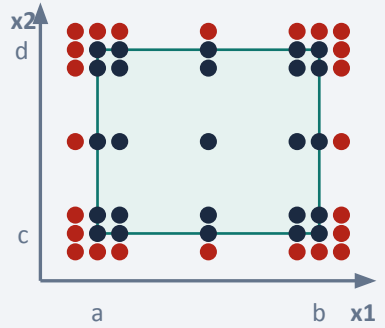


7^n = 49 pruebas (n = 2)

- Combina los siete valores (incluidos min− y max+) de cada variable.
- También conocida como prueba paranoica.
- Es la variante más exhaustiva y la más costosa.

RESUMEN VISUAL

Comparación de las cuatro variantes

	SÓLO VALORES VÁLIDOS	VÁLIDOS Y NO VÁLIDOS
FALLA ÚNICA	 **Normal** $4n + 1$	 **Robusta** $6n + 1$
PEOR CASO	 **Peor caso** 5^n	 **Robusta peor caso** 7^n

Ejercicio: valor límite normal con dos variables

EJERCICIO

Préstamo: **edad** de 18 a 65 (nominal 40) y **plazo** de 6 a 60 meses (nominal 30). ¿Qué casos genera el valor límite normal?

#	Edad	Plazo
1	18	30
2	19	30
3	40	30
4	64	30
5	65	30
6	40	6
7	40	7
8	40	59
9	40	60

4n + 1 = 4·2 + 1 = 9 casos. Una variable toma min, min+, max- y max mientras la otra queda en su valor nominal; el caso (40, 30) se comparte.

Robusta ($6n + 1 = 13$): agrega (17, 30), (66, 30), (40, 5) y (40, 61).

Peor caso ($5^n = 25$): los 5 valores de la edad × los 5 del plazo.
Robusta peor caso: $7^2 = 49$.

CRECIMIENTO SEGÚN EL NÚMERO DE VARIABLES

¿Cuántos casos de prueba se necesitan?

Variante	Fórmula	n = 1	n = 2	n = 3	n = 5
Normal	4n + 1	5	9	13	21
Robusta	6n + 1	7	13	19	31
Peor caso	5^n	5	25	125	3 125
Robusta peor caso	7^n	7	49	343	16 807

Las variantes de falla única crecen de forma lineal; las de peor caso, de forma exponencial. Elija según el riesgo y el presupuesto de pruebas.

Ejercicio: ¿cuántos casos con 4 variables?

EJERCICIO

Variante	Fórmula	n = 4
Normal	$4n + 1$	**17**
Robusta	$6n + 1$	**25**
Peor caso	5^n	**625**
Robusta peor caso	7^n	**2 401**

Las variantes de falla única crecen de forma **lineal**; las de peor caso, de forma **exponencial**.

¿Cuál elegir?

- **Normal**: variables independientes, supuesto de falla única.
- **Robusta**: además interesa cómo se manejan entradas inválidas.
- **Peor caso**: las variables interactúan y una falla sería grave.
- **Robusta peor caso** («paranoica»): sistemas críticos, como software médico.

No sirve sola con variables enumeradas (colores), booleanas o dependientes entre sí (día, mes y año bisiesto).

CASO PRÁCTICO

Ejemplo: calificaciones de estudiantes

Calificaciones válidas

Números enteros de 0 a 10.

Límites a controlar

0 (límite inferior) y 10 (límite superior).

Calificaciones no válidas

Menores que 0 o mayores que 10.

Variable de entrada: calificación (n = 1). Dominio válido: { 0, 1, 2, …, 10 }.

CASO PRÁCTICO

Valores límite de la calificación



Etiqueta	Significado	Valor
min−	Justo por debajo del límite inferior	−1
min (LB)	Límite inferior	0
nom	Valor nominal	5
max (UB)	Límite superior	10
max+	Justo por encima del límite superior	11

Datos de prueba para la calificación

Variante	Fórmula	Casos	Valores de prueba
Normal	4n + 1	5	0, 1, 5, 9, 10
Robusta	6n + 1	7	−1, 0, 1, 5, 9, 10, 11
Peor caso	5^n	5	0, 1, 5, 9, 10
Robusta peor caso	7^n	7	−1, 0, 1, 5, 9, 10, 11

Con una sola variable no hay combinaciones: el peor caso coincide con la normal (5 casos) y la robusta peor caso con la robusta (7 casos).

Ejercicio: valores límite de una variable

EJERCICIO

Un sistema acepta la **cantidad de entradas por compra: de 1 a 10** (enteros).

Variante	Fórmula	Casos	Valores
Normal	$4n + 1$	5	1, 2, 5, 9, 10
Robusta	$6n + 1$	7	0, 1, 2, 5, 9, 10, 11
Peor caso	5^n	5	1, 2, 5, 9, 10
Robusta peor caso	7^n	7	0, 1, 2, 5, 9, 10, 11

min = 1 · min+ = 2 · nom = 5 · max− = 9 · max = 10 · y en las robustas: min− = 0 y max+ = 11.

Con **una sola variable** no hay combinaciones: el peor caso coincide con la normal y la robusta peor caso con la robusta.

Otras técnicas complementarias



Prueba de valor especial

- El probador usa su conocimiento y experiencia del dominio para seleccionar valores.
- Es ad hoc: no se basa en ningún principio.
- Es el método menos sistemático y uniforme.
- Aporta valor como complemento de un método sistemático.



Pruebas aleatorias

- Se seleccionan valores aleatorios dentro de los rangos especificados.
- Pregunta clave: ¿cuántos valores son suficientes?
- Un criterio práctico: generar valores hasta cubrir al menos una vez cada clase de salida.

CUÁNDO NO ES SUFICIENTE

Limitaciones del análisis de valores límite

Funciona bien con…

Funciones de varias variables independientes que representan cantidades físicas acotadas.

Límites no obvios

Por ejemplo, ¿cuál es el límite superior de la edad?

Límites no apropiados

Por ejemplo, en variables booleanas.

Variables enumeradas

No tienen límites: “depósito”, “retiro”, “consulta”; colores “rojo”, “azul”…

Dependencias entre variables

Es inadecuado si las variables dependen entre sí (día, mes, año bisiesto).

Ignora la semántica

No tiene en cuenta la naturaleza de la función ni el significado de las variables.

Pruebas funcionales: conceptos clave



Casos de prueba

Especifican las entradas, la salida esperada del sistema y una declaración de lo que se está probando.



Datos de prueba

Son las entradas diseñadas para probar un sistema. A veces pueden generarse automáticamente.



Resultados esperados

Los casos de prueba no pueden generarse de forma totalmente automática: personas que entienden el sistema deben especificar los resultados esperados.



Ejecución automatizada

La ejecución sí puede automatizarse: los resultados se comparan automáticamente con los esperados.

Fuente: I. Sommerville, Ingeniería de Software.

PROCESO

Resumen: del dominio de entrada a los datos de prueba

01



Identificar variables

Entradas (x1, x2, …) y salidas.

02



Clasificar entradas

ROV, GV o SNM.

03



Definir el dominio

Límites mínimo y máximo de cada variable.

04



Elegir la variante

Normal, robusta, peor caso o robusta peor caso.

05



Generar datos

Calcular los valores y los casos de prueba.

Taller

Reunión de Scrum con su equipo de proyecto

1. Identificar las variables de entrada y salida.
2. Definir el dominio de entrada de cada variable.
3. Identificar los datos para la prueba con cada variante.

Variantes a aplicar

Normal	$4n + 1$
Robusta	$6n + 1$
Peor caso	5^n
Robusta peor caso	7^n

Tema 8

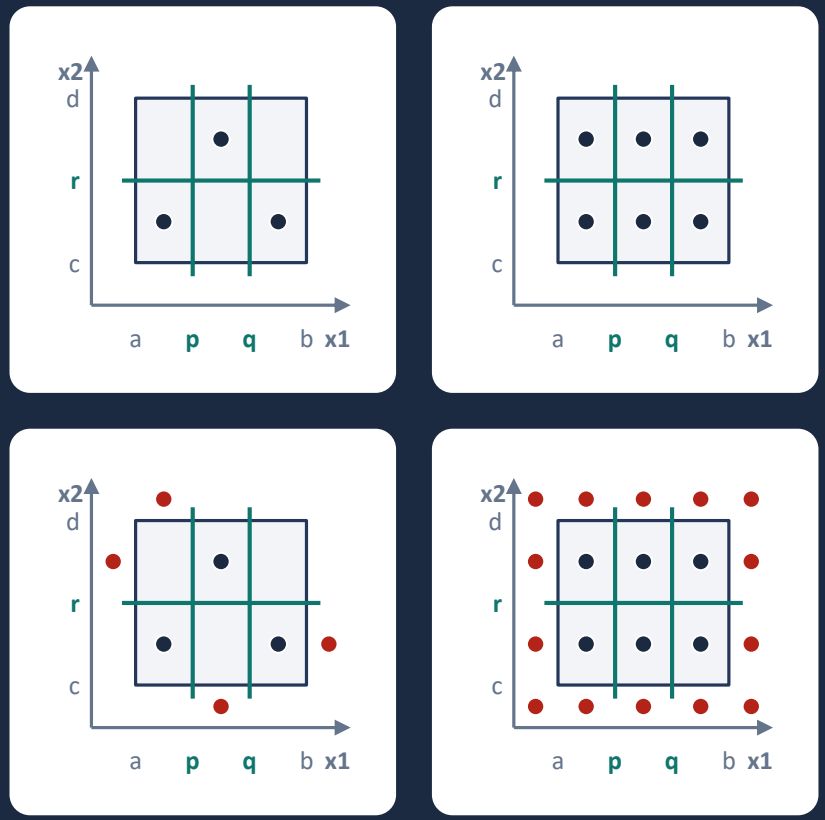
Clases de equivalencia

Pruebas funcionales: menos casos de prueba, sin perder cobertura

INGENIERÍA DE SOFTWARE II

Clases de equivalencia

Pruebas funcionales: menos casos de prueba, sin perder cobertura



CONTENIDO DE LA SESIÓN

Agenda

01

Repaso: valor límite, ROV y GV

02

Clases de equivalencia

03

Ejemplos

04

Pruebas débiles y fuertes

05

Taller

SECCIÓN 01

Repaso: valor límite, ROV y GV

01

Por qué el análisis del valor límite no basta para todas las variables

El software como una función

DOMINIO (X)

Entradas

⟨x1 = sistólica, x2 = diastólica⟩



FUNCIÓN

f

X → Y



RANGO (Y)

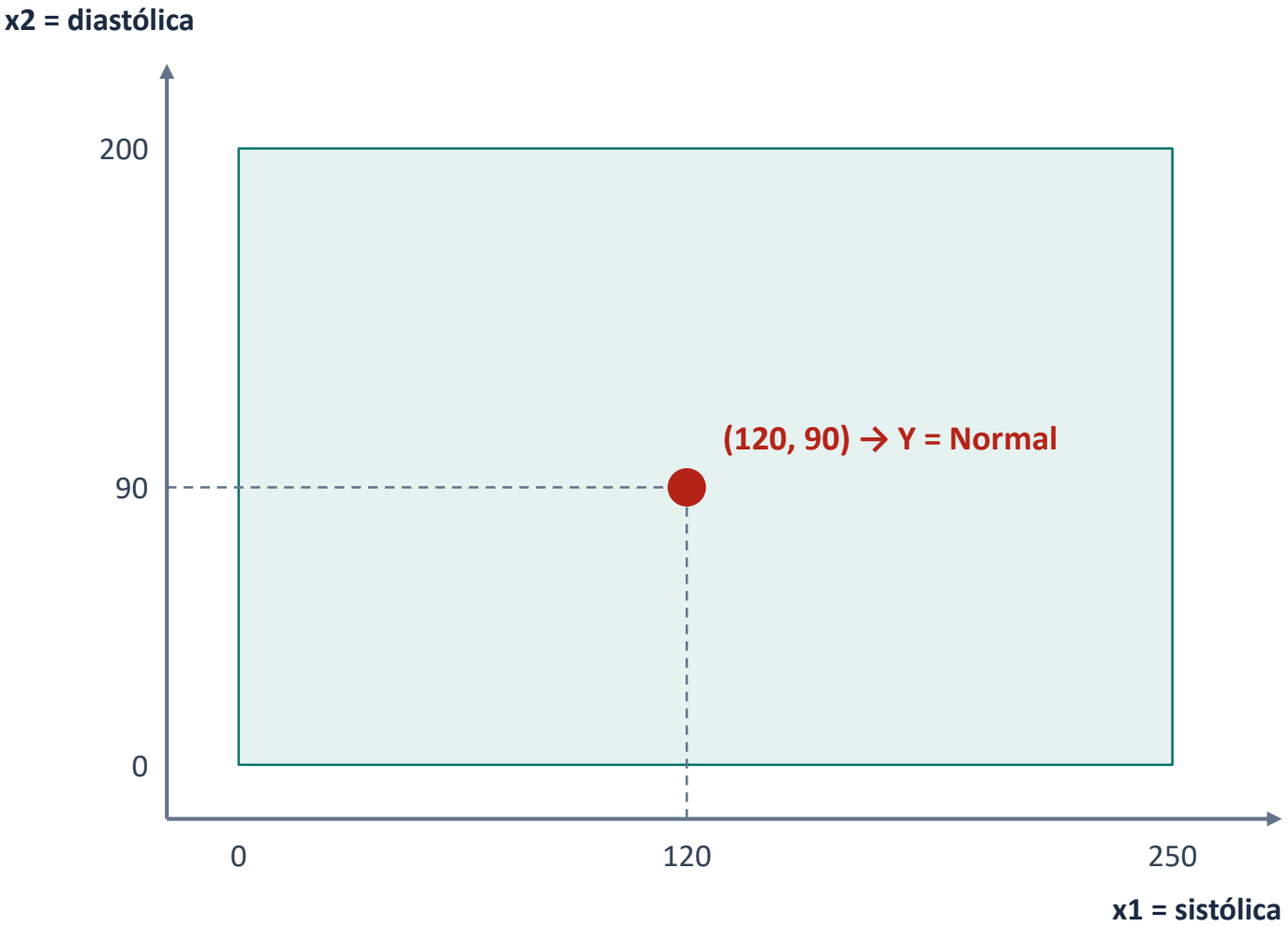
Salidas

{ Baja, Normal, Alta }



Ejemplo: clasificación de la presión arterial. Con x1 = 120 y x2 = 90, la función devuelve Y = Normal.

Dominio de entrada de dos variables



Dominio de entrada

- x1 (sistólica): de 0 a 250
- x2 (diastólica): de 0 a 200
- Cada punto del rectángulo es una combinación válida de entradas.

El supuesto oculto del valor límite

ROV

El análisis del valor límite (BVA) asume implícitamente que las variables son ROV (variables de sólo resultados).

Cuando se trata de GV (variables de puerta), este supuesto no se cumple.

Consecuencias del supuesto



Variables independientes

Las variables se tratan como si no dependieran unas de otras.



Una entrada a la vez

BVA se centra en los límites de una única entrada, sin considerar la combinación de varias.



Falla única

Es probable que se cumpla el supuesto de falla única.

Variables de puerta: varias rutas

```
if (amount <= 1800)
    Total = amount
else if (amount > 1800 AND amount < 15000)
    Total = amount * tax_rate[1]
else
    Total = amount * tax_rate[2]
```

Ruta 1

Ruta 2

Ruta 3

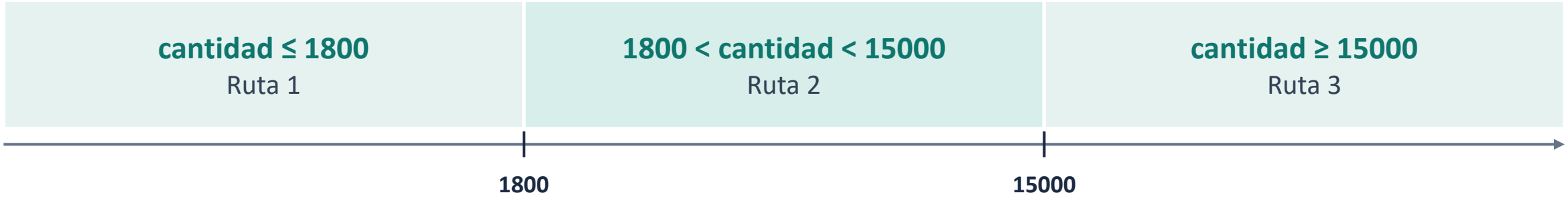
3

rutas distintas según el valor de amount

amount es una variable de puerta (GV): su valor decide qué cálculo se ejecuta.

Una variable, tres clases de equivalencia

Con una GV, una prueba de valor único es insuficiente. En el ejemplo necesitamos probar tres casos:



Decimos que la variable “cantidad” tiene tres clases de equivalencia.

Por el momento ignoramos los valores fuera de rango.

SECCIÓN 02

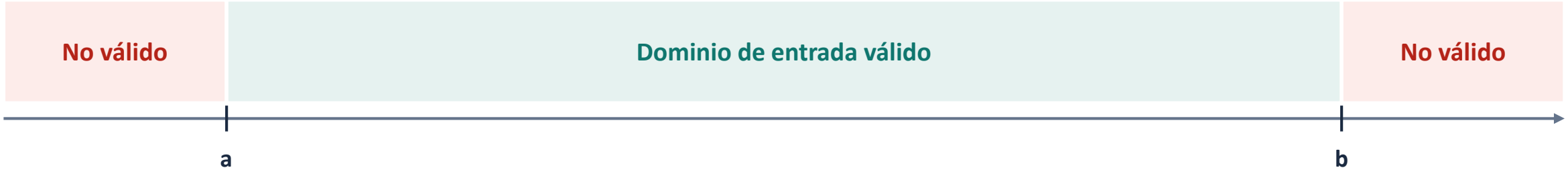
Clases de equivalencia

02

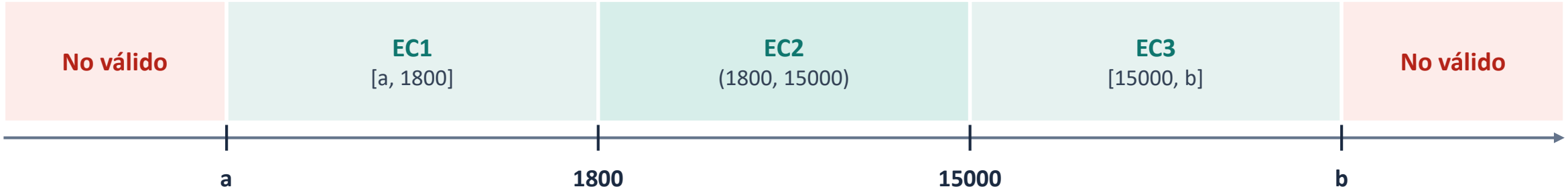
Agrupar entradas que el sistema trata de la misma manera

Del dominio a las clases de equivalencia

Valor límite: dominio de una variable



Clases de equivalencia: el dominio se divide según el comportamiento



DEFINICIÓN

¿Qué es una clase de equivalencia?

> Dos valores de entrada pertenecen a la misma clase de equivalencia si la especificación los trata de la misma forma, es decir, si **provocan que se ejecute la misma ruta.**

Mismo comportamiento

Si un valor de la clase detecta un error, los demás probablemente también.

Un representante basta

Se elige un valor dentro del intervalo para probar cada comportamiento.

Clases de equivalencia y datos de prueba

Clase	Ruta (salida)	Datos de prueba (entrada)	Valor de ejemplo
EC1	amount ≤ 1800	[a, 1800]	1000
EC2	1800 < amount < 15000	(1800, 15000)	8000
EC3	amount ≥ 15000	[15000, b]	20000

Las clases se definen a partir del rango (la salida o ruta), pero los valores que se prueban se toman del dominio (la entrada). Elija un valor dentro de cada intervalo.

¿PARA QUÉ SIRVEN?

Objetivos de las clases de equivalencia



Menos valores de entrada

Probar la menor cantidad posible de valores de entrada.



Sin pruebas redundantes

Detectar y eliminar las pruebas que no aportan información nueva.



Cuántas pruebas

Indicar cuántos casos de prueba son necesarios para probar un software.



Basta con probar un valor de cada clase: probar más sería redundante.

Completitud y no redundancia



Dominio de entrada A

Completitud

$$A = A_1 \cup A_2 \cup \ldots \cup A_8$$

Las clases cubren todo el dominio: ningún valor queda sin probar.

No redundancia

$$i \neq j \Rightarrow A_i \cap A_j = \emptyset$$

Las clases no se superponen: ningún valor se prueba dos veces.

De la salida a las clases: calificación → resultado

EJEMPLO

Una función recibe una nota de **0 a 100** y devuelve: **Reprobado** (0–59), **Bueno** (60–79) o **Excelente** (80–100).

Clase	Ruta (salida)	Datos de prueba (entrada)	Valor de ejemplo
CE1	Reprobado	[0, 59]	30
CE2	Bueno	[60, 79]	70
CE3	Excelente	[80, 100]	90
CE4	Error (no válida)	nota < 0	-5
CE5	Error (no válida)	nota > 100	105

Las clases se definen por la **salida** (qué ruta se ejecuta) y los valores se toman de la **entrada**.

La partición es **completa** (todo número cae en alguna clase) y **no redundante** (ninguno cae en dos).

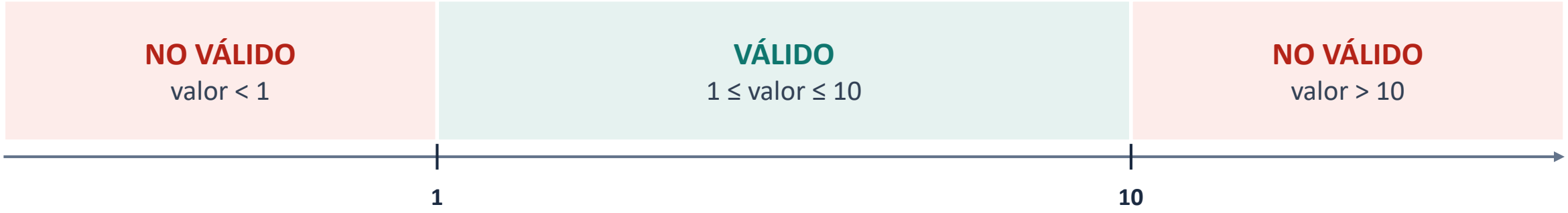
SECCIÓN 03

Ejemplos de clases de equivalencia

03

Rangos numéricos, conjuntos y un caso completo

Ejemplo 1: números del 1 al 10



CE1 **valor < 1**

No válida

CE2 **1 ≤ valor ≤ 10**

Válida

CE3 **valor > 10**

No válida

Resultado: 3 clases de equivalencia (1 válida y 2 no válidas).

Ejemplo 2: colores primarios

Dominio de colores:



Válido: el valor es un color primario.

CE	Descripción	Tipo
CE1	valor ∈ { amarillo, azul, rojo }	**Válida**
CE2	valor ∉ { amarillo, azul, rojo }	**No válida**

Ejercicio: clases de equivalencia de una contraseña

EJERCICIO

Regla: la contraseña debe tener **entre 8 y 16 caracteres**.

Clase	Descripción	Validez	Representante
CE1	longitud < 8	**No válida**	«abc12» (5)
CE2	8 ⩽ longitud ⩽ 16	**Válida**	«Uees2026x» (9)
CE3	longitud > 16	**No válida**	20 caracteres

Con clases de equivalencia bastan **3 casos**: uno por clase. Probar 9 y 12 caracteres sería redundante: los dos están en CE2.

Combinado con valor límite

Los límites de las clases son 8 y 16. Para reforzar, se prueban también **7, 8, 16 y 17**.

Las dos técnicas se complementan: las clases dicen **qué comportamientos** probar; el valor límite, **dónde** suelen esconderse los errores.

CASO COMPLETO

Ejemplo 3: sistema de un hostal en la selva ecuatoriana

```
function getPriceRoom(tipo, edad, genero)
```

Tipo de habitación
sencilla, doble, matrimonial, suite

Edad
entre 18 y 100

Género
F, M

REGLA DE NEGOCIO

Sólo para clientes de género femenino existe un 20 % de descuento en habitaciones sencillas y un 10 % en el resto de habitaciones.

¿Cuántas clases de equivalencia válidas y no válidas existen?

EJEMPLO 3 · SOLUCIÓN

Solución: clases de equivalencia del hostal

Variable	CE	Descripción	Validez
Tipo	CE1	valor ∈ { sencilla, doble, matrimonial, suite }	**Válida**
	CE2	valor ∉ { sencilla, doble, matrimonial, suite }	**No válida**
Edad	CE3	valor entero	**Válida**
	CE4	valor no entero	**No válida**
	CE5	18 ≤ valor ≤ 100	**Válida**
	CE6	valor < 18	**No válida**
	CE7	valor > 100	**No válida**
Género	CE8	valor ∈ { F, M }	**Válida**
	CE9	valor ∉ { F, M }	**No válida**

4 válidas

5 no válidas

EJEMPLO 3 · TABLA DE DECISIÓN

Descuentos según género y tipo de habitación

Las variables género y tipo son variables de puerta: juntas determinan qué descuento se aplica.

Género	Sencilla	Doble	Matrimonial	Suite
M (masculino)	0 %	0 %	0 %	0 %
F (femenino)	**20 %**	**10 %**	**10 %**	**10 %**

Hay 3 comportamientos distintos (0 %, 10 % y 20 %): cada uno debe quedar cubierto por al menos un caso de prueba.

SECCIÓN 04

Pruebas débiles y fuertes

04

Cuántos casos de prueba se necesitan

DOS NIVELES DE COBERTURA

Pruebas débiles y fuertes

DÉBIL

Cobertura de código

Número mínimo de pruebas

Cada línea de código se ejecuta al menos una vez.

FUERTE

Cobertura de ruta

Número máximo de pruebas

Se prueban todas las combinaciones de clases (todas las rutas).

En ambos casos se excluyen los valores fuera de rango.

ESPECIFICACIÓN

Problema de ejemplo: cálculo de la corrección

1. La corrección se determina según la Base, si la Base es nueva y el valor de Tipo.
2. Si la Base es menor que 10 000, se usa el Método 1. Si está entre 10 000 y 50 000, el Método 2. En otro caso, el Método 3.
3. Cada método usa procedimientos que dependen de si la Base es nueva o anterior (“Nuevo” = S o N).
4. Tipo puede ser A, B, C o X. Cada método usa el Tipo para decidir qué proceso aplicar.

Clases de cada variable

Base	Nuevo	Tipo
< 10 000	S	A
10 000 – 50 000	N	B
> 50 000		C
		X
3 clases	**2 clases**	**4 clases**

¿Cuántos casos de prueba?

ALTERNATIVA 1 · PRUEBAS FUERTES

Todas las combinaciones

`Base × Nuevo × Tipo`

3 × 2 × 4 = 24

- Se denomina cobertura de ruta.
- Equivale a configurar las “puertas” en todas las combinaciones posibles.

ALTERNATIVA 2 · PRUEBAS DÉBILES

Número mínimo de casos

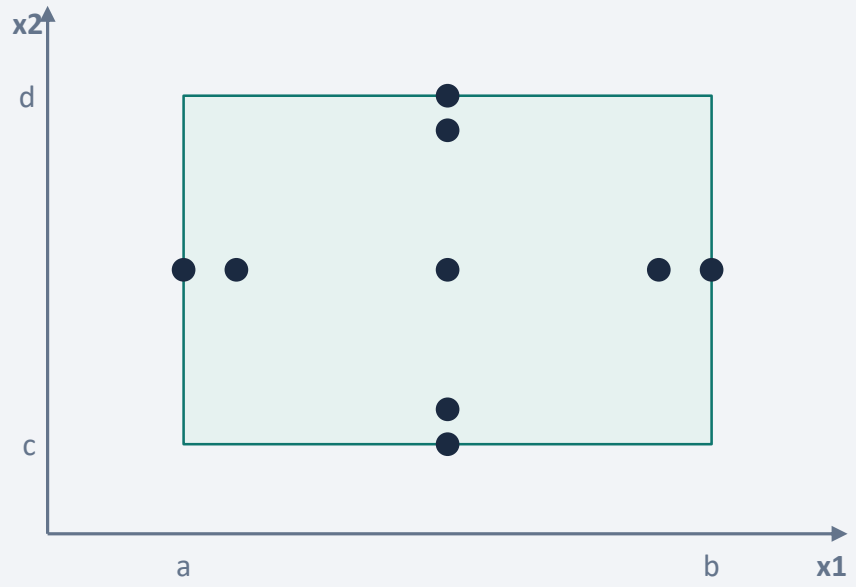
`max(Base, Nuevo, Tipo)`

max(3, 2, 4) = 4

- Se denomina cobertura de código.
- Asegura “visitar” cada línea de código una vez, sin considerar cómo se llegó allí ni a dónde se va después.

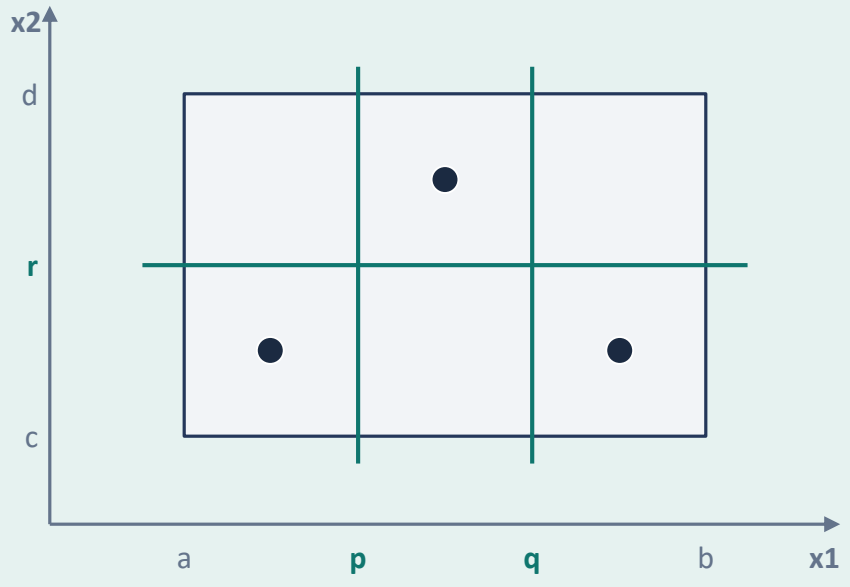
Valor límite vs. clases de equivalencia

Valor límite normal



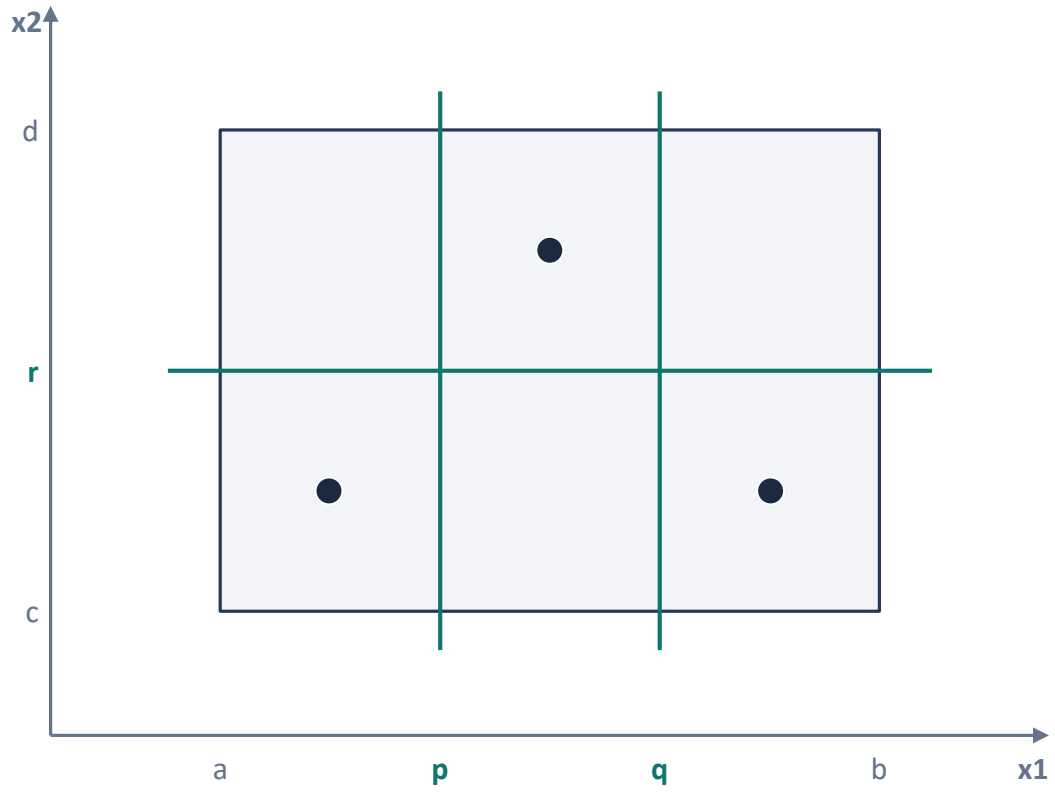
Valores válidos cerca de los límites · 9 casos

Clase de equivalencia normal débil



x1: 3 clases · x2: 2 clases → max(3, 2) = 3 casos

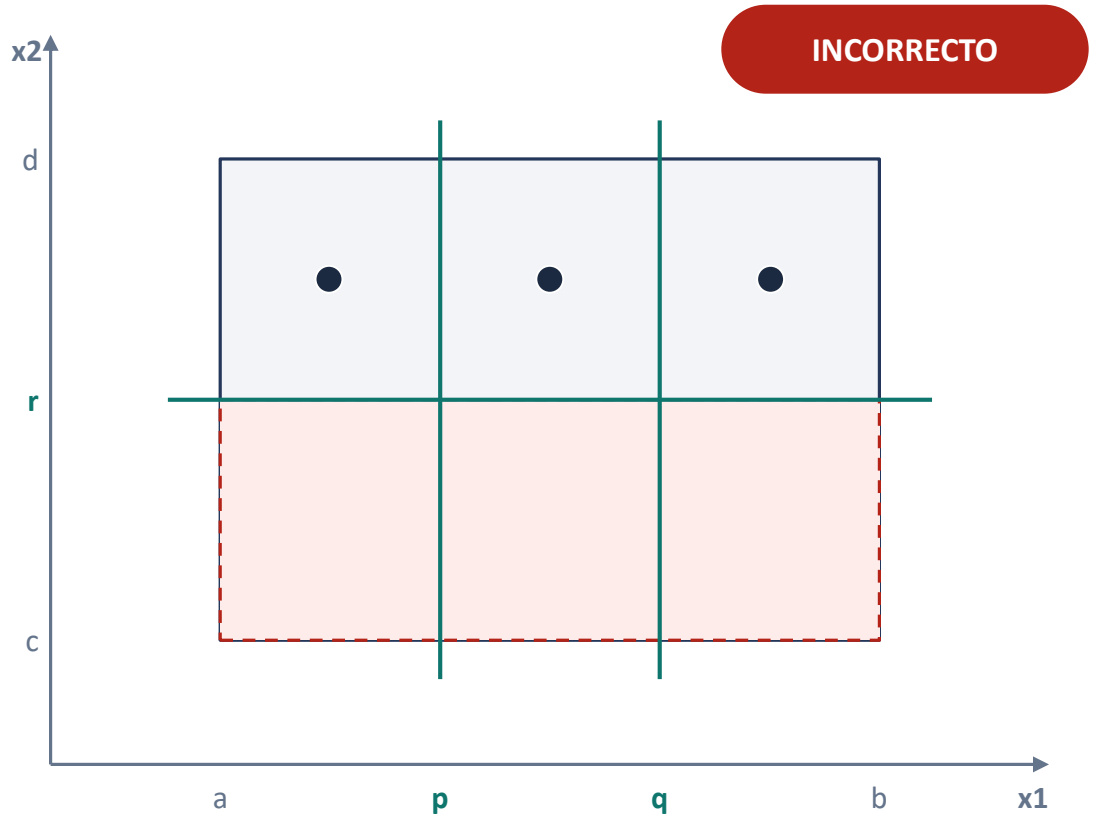
Clase de equivalencia normal débil



3 casos de prueba

- Se usa un valor de cada clase de equivalencia (intervalo) en un caso de prueba.
- x1 tiene 3 clases: [a, p), [p, q), [q, b].
- x2 tiene 2 clases: [c, r), [r, d].
- Número de casos = la mayor cantidad de clases de una variable.

Error frecuente en la prueba débil



x1: bien cubierta

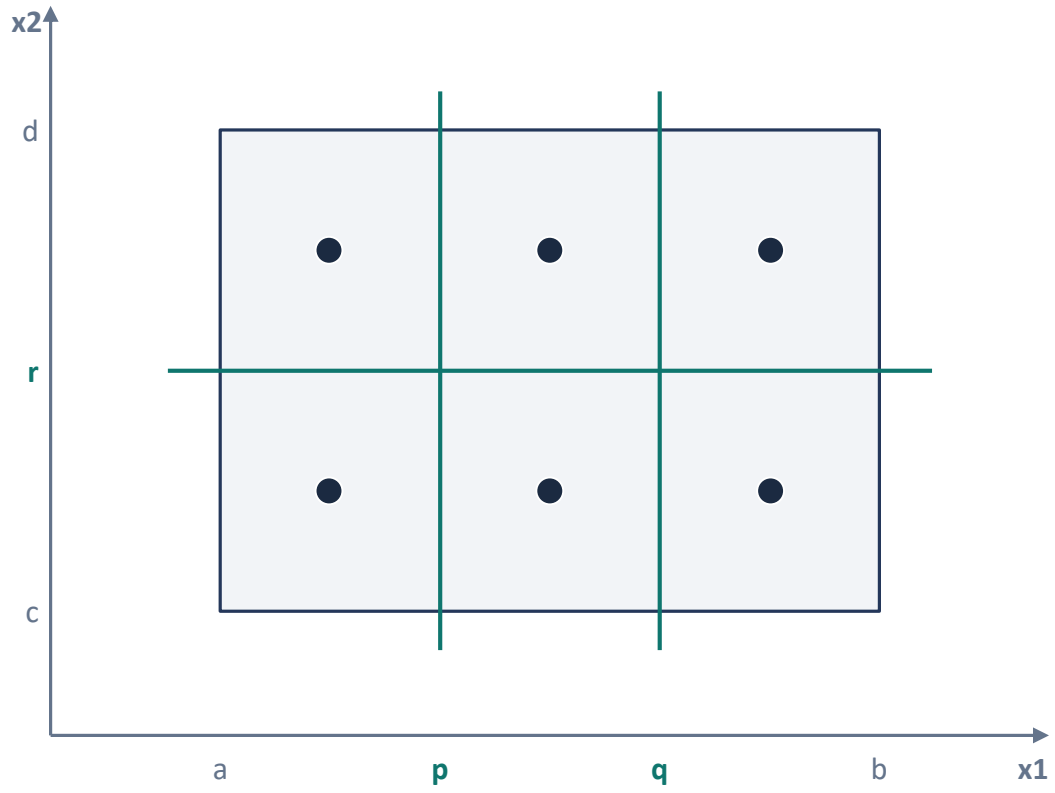
Hay un caso en cada una de sus 3 clases: (a–p), (p–q), (q–b).

x2: clase sin probar

Los 3 casos están en (r–d); la clase (c–r) nunca se prueba.

Toda clase de todas las variables debe tener al menos un caso de prueba.

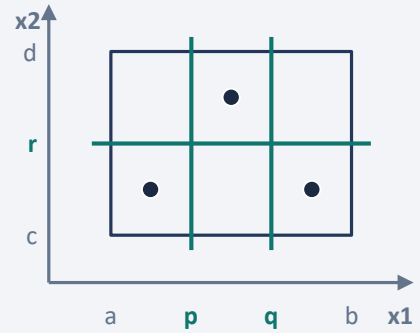
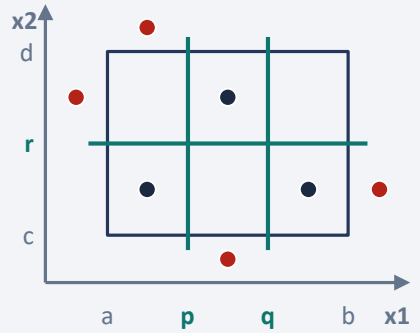
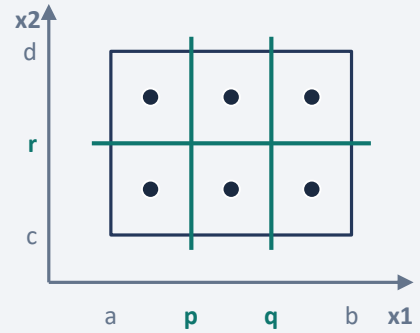
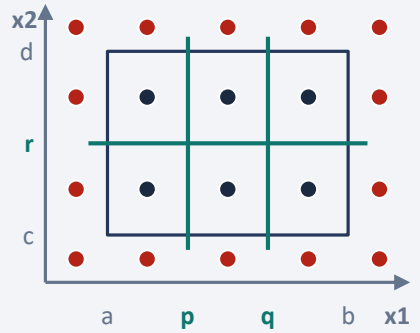
Clase de equivalencia normal fuerte



3 × 2 = 6 casos

- Se basa en el supuesto de fallas múltiples.
- Necesita un caso de prueba por cada elemento del producto cartesiano de las clases de equivalencia.
- Cubre todas las combinaciones de clases: cobertura completa.

Las cuatro variantes

	SÓLO VÁLIDOS	VÁLIDOS Y NO VÁLIDOS
DÉBIL	 **Débil normal** **3 casos** `max(3, 2)`	 **Débil robusta** **7 casos** `3 + 2 + 2`
FUERTE	 **Fuerte normal** **6 casos** `3 × 2`	 **Fuerte robusta** **20 casos** `(3+2) × (2+2)`

¿Cómo calcular el número de casos?

Variante	Supuesto	Fórmula	Ejemplo (3 y 2 clases)
Débil normal	Falla única	$\max(n_i)$	$\max(3, 2) = 3$
Fuerte normal	Falla múltiple	$\prod n_i$	$3 \times 2 = 6$
Débil robusta	Falla única	$\max(n_i)$ + no válidos de cada variable	$3 + 2 + 2 = 7$
Fuerte robusta	Falla múltiple	$\prod (n_i + 2)$	$5 \times 4 = 20$



En las variantes robustas, cada variable aporta 2 clases no válidas adicionales: por debajo y por encima de su rango.

Ejercicio: débil normal vs. fuerte normal

EJERCICIO

monto tiene 3 clases: [0, 1800], (1800, 15000), [15000, 50000]. **antigüedad** del cliente tiene 2: [0, 5) y [5, 40] años.

Débil normal	monto	antig.
1	1 000	2
2	8 000	10
3	20 000	2

max(3, 2) = 3 casos. Cada clase de cada variable aparece al menos una vez.

Fuerte normal	monto	antig.
1	1 000	2
2	1 000	10
3	8 000	2
4	8 000	10
5	20 000	2
6	20 000	10

3 × 2 = 6 casos: el producto cartesiano.

Error frecuente en la débil: poner los 3 casos con antigüedad 10 deja la clase [0, 5) sin probar.

Las variantes robustas, con el mismo ejemplo

EJEMPLO

Débil robusta · 3 + 2 + 2 = 7 casos

Los 3 casos de la débil normal, más un caso por cada clase **no válida**, con lo demás en valores válidos:

Caso	monto	antig.
4	-50	2
5	60 000	2
6	8 000	-1
7	8 000	45

Fuerte robusta · (3 + 2) × (2 + 2) = 20 casos

Cada variable suma sus 2 clases no válidas (por debajo y por encima de su rango) y se combinan todas: 5 × 4 = 20.

Variante	Fórmula	Aquí
Débil normal	$\max(n_i)$	3
Fuerte normal	$\prod n_i$	6
Débil robusta	$\max(n_i)$ + no válidas	7
Fuerte robusta	$\prod (n_i + 2)$	20

Debilidades de las clases de equivalencia



El uso de la equivalencia fuerte a veces puede causar redundancia.

¿En qué casos?



Cuando se ignoran las dependencias entre las variables de entrada y de salida.

Ejemplo: en el hostal, el tipo de habitación sólo importa si el género es F; combinar los 4 tipos con M genera casos que prueban el mismo comportamiento (0 %).

Cuándo la equivalencia fuerte es redundante

EJEMPLO

Género	Sencilla	Doble	Matrimonial	Suite
M	0 %	0 %	0 %	0 %
F	20 %	10 %	10 %	10 %

Fuerte normal: 2 géneros × 4 tipos = **8 casos**. Pero solo hay **3 comportamientos**: 0 %, 10 % y 20 %.

Con M, los 4 tipos prueban lo mismo (0 %). Un conjunto suficiente: **(M, sencilla) → 0 %**, **(F, sencilla) → 20 %** y **(F, doble) → 10 %**.

La debilidad: la equivalencia fuerte ignora las **dependencias** entre variables. El tipo de habitación solo importa cuando el género es F.

SECCIÓN 05

Taller

05

Préstamos hipotecarios

TRABAJO INDIVIDUAL

Ejercicio: préstamos hipotecarios



Monto a prestar

Desde $50 000 hasta $500 000



Tasa de interés

Monto	Tasa permitida
hasta $150 000	7 % a 9 %
mayor a $150 000	9,5 % a 10,75 %

CONSIGNA

Indique cuáles son las clases de equivalencia válidas y no válidas de cada variable de entrada.



La salida del sistema es la tabla de amortización del préstamo. Asuma que las variables de entrada son las indicadas.

Solución

Monto

CE	Descripción	Validez
CE1	monto < $50 000	**No válida**
CE2	$50 000 ≤ monto ≤ $500 000	**Válida**
CE3	monto > $500 000	**No válida**
CE9	monto no es un número real	**No válida**
CE11	monto es un número real	**Válida**

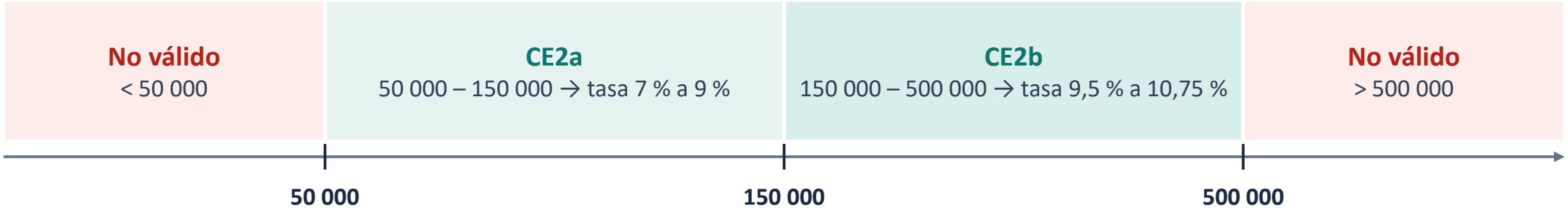
Tasa de interés

CE	Descripción	Validez
CE4	tasa < 7 %	**No válida**
CE5	7 % ≤ tasa ≤ 9 %	**Válida**
CE6	9 % < tasa < 9,5 %	**No válida**
CE7	9,5 % ≤ tasa ≤ 10,75 %	**Válida**
CE8	tasa > 10,75 %	**No válida**
CE10	tasa no es un número real	**No válida**
CE12	tasa es un número real	**Válida**

PARA PROFUNDIZAR

Observación: el monto también es una GV

El rango de tasa válido depende del monto. Por eso conviene dividir la clase válida del monto en dos:



Así se detectan casos que la solución por variable independiente no cubre: por ejemplo, monto = $100 000 con tasa = 10 % (cada valor es válido por separado, pero la combinación no lo es).

Taller resuelto: casos de prueba del préstamo hipotecario

EJERCICIO

Caso	Monto	Tasa	Clases que cubre	Esperado
1	$100 000	8 %	CE2a, CE5, CE11, CE12	Tabla de amortización
2	$300 000	10 %	CE2b, CE7	Tabla de amortización
3	$40 000	8 %	CE1	Rechazado
4	$600 000	10 %	CE3	Rechazado
5	$100 000	6 %	CE4	Rechazado
6	$100 000	9,2 %	CE6	Rechazado
7	$300 000	11 %	CE8	Rechazado
8	«abc»	8 %	CE9	Rechazado
9	$100 000	«x»	CE10	Rechazado

Los casos **válidos** cubren varias clases válidas a la vez. Los **no válidos** llevan una sola clase no válida cada uno, para saber qué causó el rechazo.

Como el monto es una **variable de puerta** para la tasa, se agrega el caso **$100 000 con 10 %**: cada valor es válido por separado, pero la combinación no. Esperado: rechazado.